

Introdução ao método dos elementos de contorno

Notas de aula — curso 30 h

Lucas S. Campos

Materiais complementares:

[Apostila](#) · [Entregas](#) · [Playlist YouTube](#)

Código: [BEM_gmsh](#)

Sumário

1	Apresentação	5
1.1	Objetivos desta aula	5
1.2	O problema reduz para o contorno	5
1.3	O BEM em uma figura	6
1.4	Quando o BEM costuma valer a pena	7
1.5	Notação mínima (aula 1)	7
1.6	Da integração por partes ao contorno	8
1.7	Laplace 1D, em camadas	9
1.8	Exemplos numéricos (Julia + Plots)	10
1.9	Roteiro	12
1.10	Exercícios	12
1.11	Preparar o Julia desta aula	13
1.12	Extra (fora da trilha da aula 1) — radiação	14
2	Glossário e notação	14
2.1	Domínio e contorno	14
2.2	Problema de potencial (Laplace / Poisson)	14
2.3	Elasticidade 2D	15
2.4	Condições de contorno (CDC)	15
2.5	Integração e erro	15
2.6	Siglas	16
2.7	Gráficos no curso	16
3	Interpolação	16
3.1	Objetivos	16
3.2	Mapa do capítulo	16
3.3	Interpolação polinomial	16
3.4	Exercícios	18
3.5	Continuando com interpolação	18
3.6	Interpolação por polinômios por partes	19
3.7	Exercício	19
3.8	Estabilidade da interpolação polinomial	20
3.9	Fenômeno de Runge	20
3.10	Integração numérica	23
3.11	Integrais singulares ou quasi-singulares	26
3.12	Desafio	30
4	Equações diferenciais	31
4.1	Objetivos	31
4.2	Mapa do capítulo	31
4.3	PVI escalar	31
4.4	Método de Euler	32
4.5	Runge–Kutta de 4ª ordem	33
4.6	Sistemas e redução de ordem	34
4.7	Matrizes de diferenças finitas	36
4.8	Método das linhas: equação do calor	37
4.9	Equação da onda (método das linhas)	38
4.10	Ponte com o BEM	40
4.11	Exercícios	45
4.12	Extra: multi-passo AB4 e Houbolt	47

5	Indo para 2D	49
5.1	Cálculo do perímetro de figuras planas	49
5.2	Área	51
6	Gmsh e malhas para o BEM	56
6.1	Por que Gmsh no BEM?	56
6.2	Instalação rápida	56
6.3	Anatomia de um arquivo .geo	56
6.4	Convenção de condições de contorno	57
6.5	Do arquivo .msh ao BEMdata	58
6.6	Gerando geometria em Julia (API Gmsh)	58
6.7	Propriedades geométricas via contorno	59
6.8	ONELAB: Gmsh como interface do solver	60
6.9	Boas práticas de malha para o BEM	60
6.10	Exercícios	60
6.11	Leitura e código de referência	61
7	Laplace 2D	61
8	Mapa do BEM (leia isto antes da álgebra)	61
9	Formulação	62
9.1	Passos 1–3: da PDE à equação integral	62
9.2	Passo 4: discretização $\rightarrow$ matrizes H e G	63
9.3	Passos 5–6: CDCs e o sistema $Ax = b$	64
9.4	Passo 7: pontos internos	65
10	Exemplo resolvido ponta a ponta (BEM_gmsh)	66
11	Montagem densa vs. H-matriz	67
11.1	Exercícios	67
11.2	Desafio	69
12	Medidas de erro	69
12.1	Erro médio	69
12.2	Erro máximo	69
12.3	Norma L_2 do erro	70
12.4	No BEM_gmsh	70
13	Poisson 2D	70
14	MECID / DIBEM	70
14.1	Exercícios	72
14.2	Gravação	74
15	Elasticidade 2D	74
16	Equações importantes	74
16.1	Código (BEM_gmsh)	76
16.2	Exercícios	77
17	Propgeo 3D	80
17.1	Exercício	80
17.2	Material complementar	81
18	Trabalhos finais	81
18.1	Regras gerais	81
18.2	Proposta A — Mecânica da fratura com Dual BEM (trinca + K_I + propagação)	82
18.3	Proposta B — Multirregião, interfaces e materiais contrastantes	83
18.4	Proposta C — Dinâmica no contorno: ondas / calor transiente com análise de esquema . . .	84
18.5	Proposta D — H-matrizes, custo e fidelidade em malhas grandes	85
18.6	Proposta E — Contato elástico por BEM de semi-espaço / semi-plano	86

18.7	Cronograma sugerido (a partir da 2ª metade do curso)	87
18.8	Integridade e uso de IA	87
18.9	Escolha orientada	87
19	Viga de Euler (extra)	89
19.1	Teoria de vigas	89
20	Efeitos transientes	92
21	Desafio	93

1 Apresentação

1.1 Objetivos desta aula

Ao final, você deve ser capaz de:

1. Explicar, em uma frase, o que o método dos elementos de contorno (BEM / MEC) faz de diferente em relação a métodos de domínio (diferenças finitas, elementos finitos).
2. Citar duas situações em que o BEM costuma ser atrativo e duas em que ele é desconfortável.
3. Partir de $T'' = 0$ em um intervalo, chegar ao sistema $HT = GQ$ nas pontas e aplicar condições de contorno.
4. Calcular T em pontos **internos** a partir só dos dados de contorno já resolvidos.
5. Resolver no Julia (com Plots) os dois casos-modelo e o exercício de convecção.

1.2 O problema reduz para o contorno

Considere uma barra (ou a espessura de uma grande placa) no intervalo

$$\Omega = (x_0, x_f), \quad \Gamma = \{x_0, x_f\}.$$

Em regime permanente, sem geração de calor, a temperatura $T(x)$ obedece à equação de Laplace 1D

$$\frac{d^2 T}{dx^2} = 0,$$

com **duas** condições de contorno escolhidas entre temperatura e fluxo

$$Q(x) := \frac{dT}{dx}$$

nas extremidades. Exemplos:

- Dirichlet: $T(x_0)$ e $T(x_f)$ dados;
- Neumann / mista: uma temperatura e um fluxo, ou dois fluxos compatíveis com o balanço.

Em 1D o contorno Γ é o par de pontas. A pergunta do BEM fica literal:

Se a física no interior está toda amarrada pelo operador diferencial, será que basta conhecer o que acontece em Γ ?

Para Laplace 1D a resposta analítica é óbvia (T é reta). O objetivo é outro: montar a **mesma** lógica que, em 2D/3D, transforma um problema de domínio em um problema só de contorno.

1.3 O BEM em uma figura

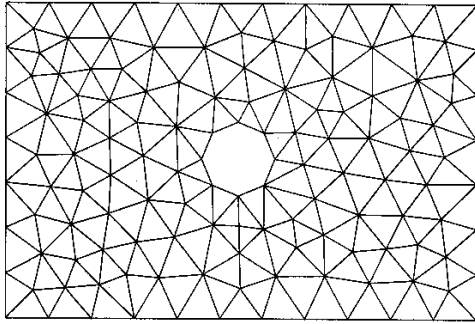


Ideia-chave (guarde esta frase):

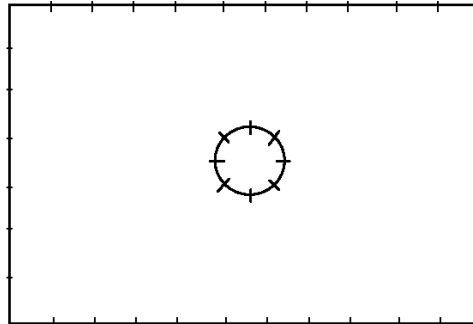
Identidade integral + solução fundamental $\Rightarrow$ equações cujas incógnitas vivem no contorno Γ .

Comparação rápida com métodos de domínio:

Aspecto	FEM / diferenças (domínio)	BEM (contorno)
O que se malha	todo o Ω	só $\Gamma = \partial\Omega$
Incógnitas primárias	nos nós do volume/área	nos nós do contorno
Interior	obtido junto com a solução	pós-processamento (avaliar a identidade num ponto interno)
Matriz típica	esparsa (interação local)	cheia e, em geral, não simétrica
Ingrediente extra	funções de forma no domínio	solução fundamental do operador



(FEM Discretization: 228 Elements)



(BEM Discretization: 44 Elements)

Em 2D, malhar só a curva que cerca a peça reduz uma dimensão da discretização. Em troca, cada ponto de contorno “enxerga” todos os outros: a matriz deixa de ser esparsa. Não existe almoço grátis.

1.4 Quando o BEM costuma valer a pena

Vantagens típicas

1. Só o contorno é discretizado: menos geometria para preparar quando Ω é grande e a física é linear e homogênea.
2. Domínios infinitos ou semi-infinitos (solo, escoamento externo, espaço aberto) entram com naturalidade via solução fundamental adequada — sem “truncar caixas” enormes.
3. Valores internos são calculados **depois**, só onde interessam (útil em laços de otimização).
4. Boa resolução de concentrações de tensão / gradientes em bordas, fendas e cargas concentradas, quando a formulação está bem posta.

Limitações típicas

1. Matrizes cheias e não simétricas: custo e memória crescem rápido se a implementação for ingênua.
2. É preciso conhecer (ou saber derivar) a **solução fundamental** do operador. Nem todo material / não-linearidade tem SF simples.
3. Estruturas muito delgadas e alguns acoplamentos são chatas de tratar só com BEM clássico.
4. Não-linearidade ou heterogeneidade no **domínio** em geral reintroduz integrais de volume (ou truques equivalentes).

Por que ainda se ensina pouco na graduação

1. Formulação matemática mais densa na entrada (integrais, SF, singularidades).
2. Menos códigos didáticos curtos do que no universo do FEM.
3. Singularidades nas integrais exigem cuidado numérico.
4. Mudar a ideia padrão dos métodos de domínio tem um custo envolvido.

Este curso ataca sobretudo formulação passo a passo, integração de termos delicados e a passagem 1D $\rightarrow$ 2D.

1.5 Notação mínima (aula 1)

Alinhada ao glossário do curso, com um atalho só para o 1D:

Símbolo	Significado aqui
$\Omega = (x_0, x_f)$	domínio 1D
$\Gamma = \{x_0, x_f\}$	contorno
n	normal exterior a Ω : $n(x_0) = -1$, $n(x_f) = +1$

T	temperatura / potencial
$Q = dT/dx$	derivada espacial (atalho 1D)
x_d	ponto fonte (onde “colocamos” a SF)
$T^*(x, x_d), Q^* = \partial T^*/\partial x$	solução fundamental e sua derivada
H, G	matrizes de influência: $HT = GQ$

Armadilha de notação. No restante do curso o fluxo de contorno do código é

$$q := -k \frac{\partial T}{\partial n}.$$

Em 1D, $\partial T/\partial n = n, Q$. Com $k = 1$, $q = -nQ$. Nesta aula trabalhamos com Q para a álgebra ficar transparente; quando formos ao 2D, a incógnita de contorno volta a ser q .

1.6 Da integração por partes ao contorno

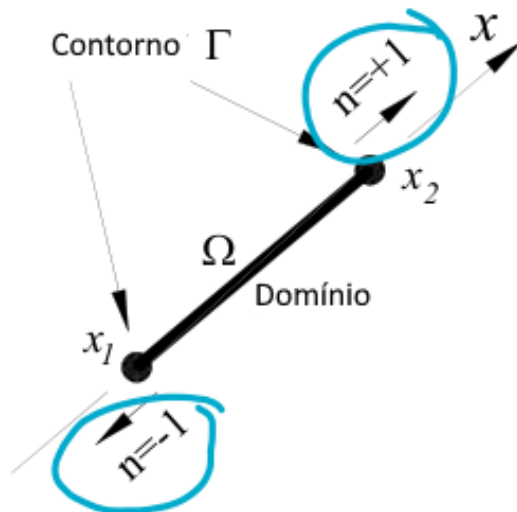
Sejam u e v funções regulares em $[x_1, x_2]$. Integração por partes:

$$\int_{x_1}^{x_2} u(x)v'(x) dx = [u(x)v(x)]_{x_1}^{x_2} - \int_{x_1}^{x_2} v(x)u'(x) dx.$$

O termo avaliado nas pontas é o contorno 1D. Com a normal exterior $n(x_1) = -1, n(x_2) = +1$,

$$[uv]_{x_1}^{x_2} = u(x_2)v(x_2)n(x_2) + u(x_1)v(x_1)n(x_1) = \sum_{x \in \Gamma} u(x)v(x)n(x),$$

que em dimensão maior se escreve $\int_{\Gamma} uv n d\Gamma$ (ou $\int_{\Gamma} uv \mathbf{n} \cdot d\Gamma$).



Leitura operacional:

1. Integração por partes **transfere** derivadas de uma função para a outra.
2. Cada transferência produz termos de contorno.
3. No BEM repetimos o processo até o operador diferencial cair inteiro sobre a função peso (a SF). O número de integrações acompanha a ordem do operador (Laplace: 2; biarmônico/viga: 4; ...).

1.7 Laplace 1D, em camadas

1.7.1 Camada 0 — problema forte

$$\frac{d^2 T}{dx^2} = 0 \quad \text{em} \quad (x_0, x_f),$$

mais duas CDC entre $\{T(x_0), T(x_f), Q(x_0), Q(x_f)\}$.

1.7.2 Camada 1 — resíduo ponderado (ainda sem escolher o peso)

Para uma função peso T^* suficientemente regular,

$$\int_{x_0}^{x_f} T^* \frac{d^2 T}{dx^2} dx = 0.$$

Duas integrações por partes levam a

$$\int_{x_0}^{x_f} T^* T'' dx = [T^* Q - T Q^*]_{x_0}^{x_f} - \int_{x_0}^{x_f} T (T^*)'' dx.$$

1.7.3 Camada 2 — escolha da função peso: solução fundamental

Em vez de um peso polinomial arbitrário, o BEM escolhe T^* especial:

$$-\frac{d^2 T^*(x, x_d)}{dx^2} = \delta(x - x_d).$$

Aqui δ é a delta de Dirac: a integral de $f(x)\delta(x - x_d)$ devolve $f(x_d)$ se o intervalo cobre x_d , e 0 caso contrário. Intuição: $T^*(\cdot, x_d)$ é a resposta em todo o eixo de uma fonte unitária em x_d , para o operador $-d^2/dx^2$.

Por que isso ajuda? O termo de domínio vira amostragem do campo:

$$\int_{x_0}^{x_f} T (T^*)'' dx = - \int_{x_0}^{x_f} T(x) \delta(x - x_d) dx = -T(x_d) \quad (\text{se } x_d \in (x_0, x_f)).$$

Com $T'' = 0$, a identidade colapsa para uma relação **só com valores de contorno** e com o valor $T(x_d)$.

1.7.4 Camada 3 — SF explícita em 1D

Uma SF conveniente é

$$T^*(x, x_d) = -\frac{1}{2} |x - x_d|, \quad Q^*(x, x_d) = \frac{\partial T^*}{\partial x} = -\frac{1}{2} \text{sign}(x - x_d).$$

Confira: longe de x_d , T^* é linear por partes, logo $(T^*)'' = 0$; o “salto” da derivada em x_d produz a delta. Em 1D, $T^*(x_d, x_d) = 0$ — a singularidade é branda. Em 2D a SF de Laplace envolve $\log r$ e a integração exige mais cuidado (aulas seguintes).

Substituindo e reorganizando, para x_d no intervalo obtém-se a equação integral de contorno

$$-T(x_d) = T(x_f)Q^*(x_f, x_d) - T^*(x_f, x_d)Q(x_f) - T(x_0)Q^*(x_0, x_d) + T^*(x_0, x_d)Q(x_0).$$

1.7.5 Camada 4 — colocação no contorno ($HT = GQ$)

O contorno só tem dois pontos. Colocamos a fonte em cada um deles: $x_d = x_0$ e $x_d = x_f$.

Valores da SF:

Quantidade	$x_d = x_0$	$x_d = x_f$
$T^*(x_0, x_d)$	0	$-\frac{x_f - x_0}{2}$
$T^*(x_f, x_d)$	$-\frac{x_f - x_0}{2}$	0
$Q^*(x_0, x_d)$	+1/2	+1/2
$Q^*(x_f, x_d)$	-1/2	-1/2

Equação em $x_d = x_0$:

$$-T(x_0) = -\frac{1}{2}T(x_f) + \frac{x_f - x_0}{2}Q(x_f) - \frac{1}{2}T(x_0).$$

Equação em $x_d = x_f$:

$$-T(x_f) = -\frac{1}{2}T(x_f) - \frac{1}{2}T(x_0) - \frac{x_f - x_0}{2}Q(x_0).$$

Reorganizando em matriz (com $T_0 = T(x_0)$, etc.):

$$\begin{pmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{pmatrix} \begin{pmatrix} T_0 \\ T_f \end{pmatrix} = (x_f - x_0) \begin{pmatrix} 0 & -0.5 \\ 0.5 & 0 \end{pmatrix} \begin{pmatrix} Q_0 \\ Q_f \end{pmatrix}.$$

Ou seja,

$$HT = GQ, \quad H = \begin{pmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{pmatrix}, \quad G = (x_f - x_0) \begin{pmatrix} 0 & -0.5 \\ 0.5 & 0 \end{pmatrix}.$$

Leitura que você vai reencontrar em 2D:

- cada **linha** de H e G corresponde a uma colocação (um x_d);
- cada **coluna** corresponde à influência de um grau de liberdade de contorno (T ou Q naquele nó);
- H multiplica temperaturas; G multiplica fluxos Q .

Condições de contorno. Das quatro quantidades (T_0, T_f, Q_0, Q_f) , **duas** são dados e duas são incógnitas. Move-se para a esquerda tudo o que é desconhecido e para a direita tudo o que é conhecido, até obter $Ax = b$ com $A \ 2 \times 2$.

1.7.6 Camada 5 — pontos internos (pós-processamento)

Com T e Q já conhecidos **nas pontas**, a mesma identidade com $x_d \in (x_0, x_f)$ devolve $T(x_d)$ sem resolver outro sistema. Para a SF deste capítulo:

$$T(x_d) = \frac{1}{2}(T_0 + T_f) + \frac{x_d - x_0}{2}Q_0 + \frac{x_d - x_f}{2}Q_f.$$

Isso materializa a vantagem “interior sob demanda”: o sistema linear é só de contorno; o campo interno é avaliação.

1.8 Exemplos numéricos (Julia + Plots)

Ambiente mínimo desta aula:

```
using Pkg
Pkg.add("Plots") # uma vez por ambiente
using Plots, LinearAlgebra
```

Matrizes H e G no intervalo $[x_0, x_f]$:

```
function bemld_matrices(x0, xf)
    L = xf - x0
    H = [0.5 -0.5; -0.5 0.5]
    G = L * [0.0 -0.5; 0.5 0.0]
    return H, G, L
end
```

Montagem a partir de $HT - GQ = 0$: colunas de incógnitas vão para A ; termos conhecidos vão para b .

```
"""Resolve H*T = G*Q com CDC mistas.
`T_data` / `Q_data`: valor numérico se conhecido, `NaN` se incógnita.
Em cada nó, prescreva exatamente uma entre T e Q."""
function solve_bemld(H, G, T_data, Q_data)
    n = 2
    T_known = .!isnan.(T_data)
    Q_known = .!isnan.(Q_data)
    @assert count(T_known) + count(Q_known) == n
    @assert all(T_known .!= Q_known)

    A = zeros(n, n)
    b = zeros(n)
    col_of_T = zeros{Int, n}
    col_of_Q = zeros{Int, n}
    col = 0

    for i in 1:n
        if T_known[i]
            b .= H[:, i] * T_data[i]
        else
            col += 1
            col_of_T[i] = col
            A[:, col] .+= H[:, i]
        end
    end
    for i in 1:n
        if Q_known[i]
            b .+= G[:, i] * Q_data[i]
        else
            col += 1
            col_of_Q[i] = col
            A[:, col] .-= G[:, i] # -G * Q_desconhecido
        end
    end

    x = A \ b
    T = zeros(n)
    Q = zeros(n)
    for i in 1:n
        T[i] = T_known[i] ? T_data[i] : x[col_of_T[i]]
        Q[i] = Q_known[i] ? Q_data[i] : x[col_of_Q[i]]
    end
    return T, Q, A, b, x
end
```

```
end
```

```
T_interior(xd, x0, xf, T, Q) =  
    0.5 * (T[1] + T[2]) + (xd - x0)/2 * Q[1] + (xd - xf)/2 * Q[2]
```

1.8.1 Caso 1 — Dirichlet

$$T(0) = 100, \quad T(1) = 0 \quad \Rightarrow \quad T_{\text{exata}(x)} = 100 - 100x, \quad Q = -100.$$

```
x0, xf = 0.0, 1.0  
H, G, L = bem1d_matrices(x0, xf)  
  
T_data = [100.0, 0.0] # T conhecido nos dois nós  
Q_data = [NaN, NaN] # Q desconhecido  
T, Q, A, b, x = solve_bem1d(H, G, T_data, Q_data)  
@show T Q # Q ≈ [-100, -100]  
  
xs = range(x0, xf; length=100)  
T_num = [T_interior(xd, x0, xf, T, Q) for xd in xs]  
T_exato = @. 100 - 100 * xs  
@show maximum(abs, T_num - T_exato)  
  
plot(xs, T_exato; label="exato", xlabel="x", ylabel="T",  
      title="Caso 1 — Dirichlet", lw=2)  
plot!(xs, T_num; label="BEM (interior)", ls=:dash, lw=2)
```

1.8.2 Caso 2 — mista (um dado em cada ponta)

$$T(0) = 100, \quad Q(1) = 0 \quad \Rightarrow \quad T \equiv 100, \quad Q \equiv 0.$$

```
T_data = [100.0, NaN]  
Q_data = [NaN, 0.0]  
T, Q, A, b, x = solve_bem1d(H, G, T_data, Q_data)  
@show T Q # T ≈ [100, 100], Q ≈ [0, 0]
```

Caso 2b (variante). $T(0) = 100, Q(0) = 0$ também fecha algebricamente e devolve $T_f = 100, Q_f = 0$. Serve para ver que bastam **dois dados independentes** entre os quatro slots — não necessariamente um em cada lado.

1.9 Roteiro

1. Operador no domínio $\rightarrow$ resíduo ponderado.
2. Integrar por partes até o operador cair na função peso.
3. Escolher o peso = SF ($-L^*T^* = \delta$).
4. Domínio vira $T(x_d)$; sobram termos só em Γ .
5. Colocar x_d nos nós de contorno $\rightarrow HT = GQ$.
6. Aplicar CDC $\rightarrow Ax = b$.
7. Interior: reavaliar a identidade com $x_d \in \Omega$.

1.10 Exercícios

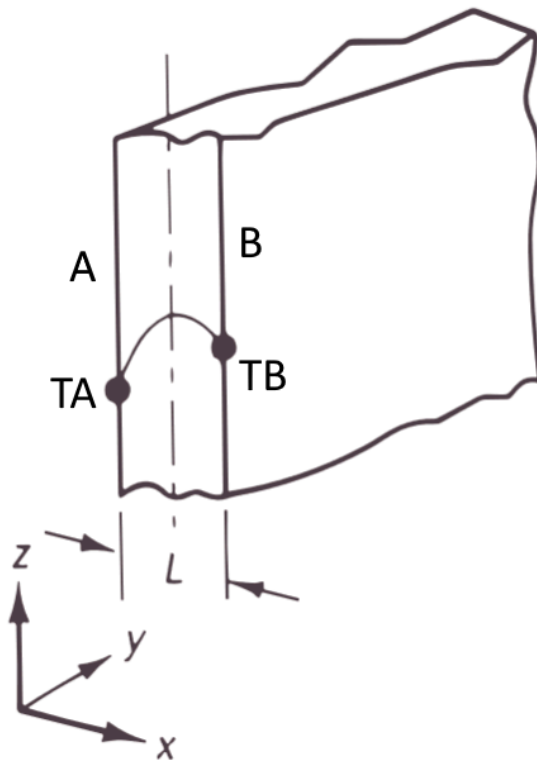
1. **Convecção (Robin) à direita.** No extremo direito, $Q_f = h(T_f - T_\infty)$ com $T_\infty = 20^\circ\text{C}$ e $h = 3$ (unidades consistentes do modelo 1D). À esquerda, $T_0 = 100^\circ\text{C}$. Como Q_f depende de T_f , substitua

essa relação **antes** de montar $Ax = b$ (a linha correspondente mistura colunas de H e G). Resolva analítica e numericamente. Compare $T(x)$ e os fluxos nas pontas.

2. **Fonte uniforme (Poisson 1D).** Com geração b constante, a identidade ganha um termo de domínio

$$-T(x_d) = \dots(\text{termos de contorno})\dots + \int_{x_0}^{x_f} T^*(x, x_d) b \, dx.$$

Para b constante a integral é analítica. Considere a placa de espessura $L = 2$ cm, $k = 1$ W/(m·K), $b = 1000$ kW/m³, faces a $T_A = 100$ °C e $T_B = 200$ °C. Atente às unidades (L em metros). Compare com



$$T(x) = \left[\frac{T_B - T_A}{L} + \frac{b}{2k}(L - x) \right] x + T_A.$$

Nota: a equação forte passa a envolver b e k ; mantenha a mesma convenção de sinal da SF e do termo de domínio.

3. **Perguntas de checagem (sem código).**

- Por que a matriz do BEM, em geral, é cheia?
- O que precisa existir para o BEM “clássico” de um operador linear?
- Se só T_0 e T_f são dados, o sistema devolve o quê?

1.11 Preparar o Julia desta aula

```
winget install julia -s msstore
julia --version
```

```
using Pkg
Pkg.add("Plots")
using Plots
```

Gráficos desta aula usam **Plots.jl**. Capítulos seguintes reintroduzem malha, Gmsh e o fluxo de trabalho completo quando a geometria deixar de ser um intervalo.

1.12 Extra (fora da trilha da aula 1) — radiação

Condição não linear de radiação entre superfícies:

$$q_n = \kappa f_s f_\epsilon (u^4 - u_R^4) = \underbrace{\kappa f_s f_\epsilon (u^2 + u_R^2)}_{h_r(u)} (u + u_R)(u - u_R),$$

com $\kappa \approx 5.699 \times 10^{-8} \text{ W/(m}^2 \cdot \text{K}^4)$ (Stefan–Boltzmann), $0 \leq f_s \leq 1$ fator de forma e $0 < f_\epsilon \leq 1$ emissividade. Parece convecção com h_r dependente da própria temperatura: resolve-se o problema linear, atualiza-se h_r , itera-se até a variação de u ficar pequena.

Projeto opcional **depois** de dominar Robin linear — não é pré-requisito da aula 1.

2 Glossário e notação

Este capítulo fixa a **notação padrão** das notas. Quando um capítulo legado usar outro símbolo, a tabela abaixo prevalece.

2.1 Domínio e contorno

Símbolo	Significado
Ω	domínio (área em 2D, volume em 3D)
$\Gamma = \partial\Omega$	contorno (orientado; normal saindo de Ω)
$\mathbf{n} = (n_x, n_y)$	normal unitária exterior
$\mathbf{x} = (x, y)$	ponto de campo (integração)
$\mathbf{x}_d$ ou p	ponto fonte (colocação)
$r = \mathbf{x} - \mathbf{x}_d $	distância fonte–campo
$N_k(\xi)$	função de forma no elemento de contorno
J	jacobiano da transformação para $\xi \in [-1, 1]$

2.2 Problema de potencial (Laplace / Poisson)

Usamos **temperatura/potencial** T e **fluxo de contorno** q com a convenção do BEM_gmsh:

$$q := -k \frac{\partial T}{\partial n}.$$

Símbolo	Significado
T	potencial / temperatura
q	fluxo no contorno, $q = -k \partial T / \partial n$
k	condutividade (Laplace: <code>Laplace(k)</code>)
f	fonte de domínio em Poisson: $\nabla^2 T = f$
T^*, q^*	solução fundamental e sua derivada normal
H, G	matrizes de influência: $HT = Gq$ (+ termo de domínio)

M	matriz de domínio (DIBEM / “massa”)
$A, \mathbf{x}, \mathbf{b}$	sistema final após CDCs: $A\mathbf{x} = \mathbf{b}$

Em textos de mecânica dos fluidos o potencial costuma ser φ ou u ; **nestas notas de potencial térmico** preferimos T . Nos exercícios de membrana, a deflexão é w e a equação é a de Poisson em w .

2.3 Elasticidade 2D

Símbolo	Significado
u_i ou $\mathbf{u} = (u_x, u_y)$	deslocamento
t_i ou $\mathbf{t}$	tração no contorno, $t_i = \sigma_{ij}n_j$
$\sigma_{ij}, \varepsilon_{ij}$	tensão e deformação
E, ν	módulo de Young e coeficiente de Poisson
$\mu = E/(2(1 + \nu))$	módulo de cisalhamento
U_{ij}, T_{ij}	SF de deslocamento e de tração (Kelvin)
H, G	blocos 2×2 por nó: análogo vetorial de $HT = Gq$

Não use ν (nu) e v (velocidade ou função peso) no mesmo parágrafo sem deixar claro. A função peso dos resíduos é v ou T^* ; o Poisson do material é sempre ν .

2.4 Condições de contorno (CDC)

Tipo	O que é prescrito
Dirichlet	T (potencial) ou u_i (deslocamento)
Neumann	q (fluxo) ou t_i (tração)
Mista	Dirichlet em parte de Γ , Neumann no resto
Robin / convecção	$q = h(T - T_\infty)$ – combinação linear

No Gmsh / BEM_gmsh: Laplace " $0;T$ " ou " $1;q$ "; elasticidade " $tx;ux;ty;uy$ " (ver capítulo **Gmsh e malhas**).

2.5 Integração e erro

Símbolo	Significado
n ou N	número de nós / pontos de colocação no contorno
n_{elem}	número de elementos de contorno
$n_{\text{pg}} / \text{npg}$	pontos de Gauss por elemento
ε_u	erro relativo (médio, máximo ou L_2 – capítulo Medidas de erro)
<code>rel_error(dad)</code>	erro relativo agregado no BEM_gmsh

2.6 Siglas

Sigla	Significado
BEM / MEC	Boundary Element Method / Método dos Elementos de Contorno
MEF / FEM	Método dos Elementos Finitos
SF	solução fundamental
CDC	condição de contorno
DIBEM	Dual Integration Boundary Element Method – termo de domínio por RBF
RBF	função de base radial
H-matriz	matriz hierárquica (compressão de blocos de H e G)
PVI	problema de valor inicial

2.7 Gráficos no curso

Salvo menção em contrário, os scripts usam **Plots.jl**:

```
using Plots
plot(x, y; xlabel="x", ylabel="T", label="numérico", lw=2)
```

Funções de visualização do repositório de código (quando usadas) podem ter backend próprio; nestas notas o padrão didático é Plots.

3 Interpolação

Gráficos deste capítulo usam **Plots.jl**.

3.1 Objetivos

1. Montar e resolver o sistema de Vandermonde para interpolação polinomial.
2. Reconhecer quando o polinômio global oscila (Runge) e quando splines por partes são preferíveis.
3. Comparar nós equidistantes e nós de Chebyshev pelo erro máximo em escala log.
4. Aplicar trapézio e Gauss–Legendre e ler a ordem de convergência no gráfico.
5. Suavizar integrandos quase-singulares com transformação (sinh / Monegato).

3.2 Mapa do capítulo

1. Interpolação polinomial (Vandermonde, exemplo China)
2. Limites do polinômio global e splines por partes
3. Estabilidade, Runge e nós de Chebyshev
4. Integração numérica (trapézio, Gauss)
5. Integrais singulares / quase-singulares
6. Desafio (aleta — ponte para BEM 1D)

3.3 Interpolação polinomial

Suponha que queremos conhecer a população às vezes entre os anos do censo ou estimar populações futuras. Uma técnica é encontrar um polinômio que passa por todos os pontos de dados.

Dado n pontos $(t_1, y_1), \dots, (t_n, y_n)$, onde os t_i são todos distintos, o problema **de interpolação polinomial** é encontrar um polinômio p de grau menor que n tal que $p(t_i) = y_i$ para todos i .

O problema de interpolação polinomial tem uma solução única. Uma vez encontrado o polinômio interpolador, ele pode ser avaliado em qualquer lugar para estimar ou prever valores.

3.3.1 Interpolação como um sistema linear

Dados os dados (t_i, y_i) por $i = 1, \dots, n$, buscamos um polinômio

$$p(t) = c_1 + c_2 t + c_3 t^2 + \dots + c_n t^{n-1},$$

de tal modo que $y_i = p(t_i)$ para todos i . Essas condições são usadas para determinar os coeficientes $c_1, \dots, c_n$:

$$\begin{aligned} c_1 + c_2 t_1 + \dots + c_{n-1} t_1^{n-2} + c_n t_1^{n-1} &= y_1, \\ c_1 + c_2 t_2 + \dots + c_{n-1} t_2^{n-2} + c_n t_2^{n-1} &= y_2, \\ c_1 + c_2 t_3 + \dots + c_{n-1} t_3^{n-2} + c_n t_3^{n-1} &= y_3, \\ &\vdots \\ c_1 + c_2 t_n + \dots + c_{n-1} t_n^{n-2} + c_n t_n^{n-1} &= y_n. \end{aligned}$$

Essas equações formam um sistema linear para os coeficientes c_i :

$$\begin{bmatrix} 1 & t_1 & \dots & t_1^{n-2} & t_1^{n-1} \\ 1 & t_2 & \dots & t_2^{n-2} & t_2^{n-1} \\ 1 & t_3 & \dots & t_3^{n-2} & t_3^{n-1} \\ \vdots & \vdots & & \vdots & \vdots \\ 1 & t_n & \dots & t_n^{n-2} & t_n^{n-1} \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix},$$

ou simplesmente, $\mathbf{V}\mathbf{c} = \mathbf{y}$. A matriz $\mathbf{V}$ é de um tipo especial chamado de Vandermonde.

A interpolação polinomial pode, portanto, ser formulada como um sistema linear de equações com uma matriz de Vandermonde. Criamos dois vetores para dados sobre a população da China. O primeiro tem os anos dos dados do censo e o outro tem a população, em milhões de pessoas.

```
year = [1982, 2000, 2010, 2015];
pop = [1008.18, 1262.64, 1337.82, 1374.62];
t = year .- 1980.0
y = pop;
V = [ t[i]^j for i=1:4, j=0:3 ]
#4x4 Matrix{Float64}:
#1.0  2.0  4.0  8.0
#1.0  20.0 400.0 8000.0
#1.0  30.0 900.0 27000.0
#1.0  35.0 1225.0 42875.0
c = V \ y
#4-element Vector{Float64}:
# 962.2387878787877
# 24.12775468975476
# -0.592262049062053
# 0.006843867243867301

using Polynomials
p = Polynomial(c)
p(2005-1980)
```

O valor oficial da população para 2005 foi 1303,72, então nosso resultado é bastante bom.

```
using Plots
tt = range(0, 35; length=500)
scatter(t, y; label="real", xlabel="anos desde 1980",
        ylabel="população (milhões)", title="População da China",
        legend=:topleft)
plot!(tt, p.(tt); label="interpolante", lw=2)
```

3.4 Exercícios

- Suponha que você queira interpolar os pontos $(-1,0)$, $(0,1)$, $(2,0)$, $(3,1)$ e $(4,2)$ por um polinômio de grau o mais baixo possível. **(a)** 🧑‍🔬 Qual é o grau máximo necessário desse polinômio? **(b)** 🧑‍🔬 Escreva um sistema linear de equações para os coeficientes do polinômio interpolador. **(c)** 📖 Use Julia para resolver numericamente o sistema em (b).
- (a)** 🧑‍🔬 Suponha que você quer encontrar um polinômio cúbico p tal que $p(-1) = -2$, $p'(-1) = 1$, $p(1) = 0$, e $p'(1) = -1$. (Isso é conhecido como um *Interpolador de Hermite*.) Escreva um sistema linear de equações para os coeficientes de p . **(b)** 📖 Use Julia para resolver o sistema linear na parte (a), e faça um gráfico de p sobre $-1 \leq x \leq 1$.

3.5 Continuando com interpolação

Dado $n + 1$ pontos distintos (t_0, y_0) , (t_1, y_1) , ..., (t_n, y_n) , com $t_0 < t_1 < \dots < t_n$ chamados de nós, o problema de interpolação é encontrar uma função $p(x)$, chamada de interpolante, tal que $p(t_k) = y_k$ para $k = 0, \dots, n$.

Aqui t_k são os nós e x denota a variável independente contínua.

Nas fórmulas os nós costumam ir de 0 a n . Em Julia os vetores começam em 1: o nó matemático t_k é `t[k+1]`.

Os polinômios são o primeiro candidato óbvio a servir como funções interpolando. Eles são fáceis de trabalhar e, vimos que um sistema linear de equações pode ser usado para determinar os coeficientes de um polinômio que passa por todos os membros de um conjunto de pontos. No entanto, não é difícil encontrar exemplos para os quais a interpolação polinomial leva a resultados inutilizáveis.

Aqui estão alguns pontos que poderíamos considerar ser observações de uma função desconhecida em $[-1,1]$.

```
using Plots
n = 5
t = range(-1, 1; length=n+1)
y = @. t^2 + t + 0.05*sin(20*t)
scatter(t, y; label="dados", legend=:topleft)
```

O interpolante polinomial, calculado usando o `fit`, parece muito bom.

```
using Plots, Polynomials
p = fit(t, y, n)
xx = range(-1, 1; length=400)
scatter(t, y; label="dados", legend=:topleft)
plot!(xx, p.(xx); label="interpolante", lw=2)
```

Mas agora considere um conjunto diferente de pontos gerados quase exatamente da mesma maneira.

```
n = 18
t = range(-1, 1; length=n+1)
y = @. t^2 + t + 0.05*sin(20*t)
scatter(t, y; label="dados", legend=:topleft)
```

Os pontos em si não têm nada de especial. Mas observe o que acontece com o interpolante polinomial.

```
p = fit(t, y, n)
x = range(-1, 1; length=1000)
scatter(t, y; label="dados", legend=:topleft)
plot!(x, p.(x); label="interpolante", lw=2)
```

Certamente deve haver funções que são mais representativas desses pontos!

Ideia-chave. Polinômio global de grau alto em nós equidistantes pode oscilar violentamente entre os dados (ainda que passe por todos os pontos). Grau baixo **por partes** (spline) costuma representar melhor o mesmo conjunto.

3.6 Interpolação por polinômios por partes

Para manter pequenos graus de polinomial enquanto interpolam grandes conjuntos de dados, escolheremos interpolantes dos polinômios por partes. Especificamente, o interpolante p deve ser um polinômio em cada subintervalo $[t_{k-1}, t_k]$ para $k = 1, \dots, n$.

Geralmente, designamos antecipadamente um grau máximo para cada parte polinomial de $p(x)$.

```
using Plots, Interpolations

# `t` estritamente crescente (nós). Linear e cúbico bastam para contrastar
# com o polinômio global; o grau 2 não é first-class em Interpolations.jl.
p1 = LinearInterpolation(t, y)           # grau 1 (por partes)
p3 = CubicSplineInterpolation(t, y)      # grau 3 (spline cúbica)
xx = range(first(t), last(t); length=400)
scatter(t, y; label="dados", legend=:topleft)
plot!(xx, p1.(xx); label="linear por partes", lw=2)
plot!(xx, p3.(xx); label="cúbico por partes", lw=2)
```

3.7 Exercício

1. Os dois vetores a seguir definem uma forma geométrica.

```
x = [ 0, 0.51, 0.96, 1.06, 1.29, 1.55, 1.73, 2.13, 2.61,
      2.19, 1.76, 1.56, 1.25, 1.04, 0.58, 0 ]
y = [ 0, 0.16, 0.16, 0.43, 0.62, 0.48, 0.19, 0.18, 0,
      -0.12, -0.12, -0.29, -0.30, -0.15, -0.16, 0 ]
```

Podemos considerar x e y como funções de um parâmetro s , com os pontos sendo valores dados em $s = 0, 1, \dots, 15$ (grade uniforme).

(a) Interpole com spline cúbica (`CubicSplineInterpolation` ou `BSpline(Cubic(...))`) em $x(s)$ e $y(s)$ e plote a curva $(x(s), y(s))$.

(b) O canto à esquerda corresponde a juntar $s = 0$ com $s = 15$. Use contorno **periódico** no parâmetro (grade uniforme):

```
using Interpolations
s = 0:15
itpx = scale(interpolate(x, BSpline(Cubic(Periodic(OnGrid())))), s)
itpy = scale(interpolate(y, BSpline(Cubic(Periodic(OnGrid())))), s)
ss = range(0, 15; length=400)
plot(itpx.(ss), itpy.(ss); label="cúbica periódica", lw=2, aspect_ratio=1)
```

Compare com a spline **sem** Periodic (condição natural).

3.8 Estabilidade da interpolação polinomial

Escolhemos uma função em relação ao intervalo $[0, 1]$.

```
using Plots, Polynomials
f = x -> sin(exp(2*x))
t = (0:6) ./ 6
y = f.(t)
p = fit(t, y)
xx = range(0, 1; length=400)
plot(xx, f.(xx); label="função", title="Interpolante equidistante, n=6",
      legend=:bottomleft, lw=2)
scatter!(t, y; label="nós")
plot!(xx, p.(xx); label="interpolante", lw=2, ls=:dash)
```

Isso parece bom. Queremos rastrear o comportamento do erro à medida que N aumenta. Estimaremos o erro no interpolante contínuo, amostrando-o em um grande número de pontos e tomando a norma máxima.

```
using LinearAlgebra, Polynomials, Plots
n = 5:5:60; err = zeros(size(n))
x = range(0,1,length=2001) # pontos para medir o erro
for (i,n) in enumerate(n)
    t = (0:n)/n
    y = f.(t)
    p = fit(t, y)
    err[i] = norm((@. f(x) - p(x)), Inf)
end
using Plots
plot(n, err; yscale=:log10, xlabel="n", ylabel="max error",
      title="Erro de interpolação para nós equidistantes",
      marker=:circle, label=false, lw=2)
```

O erro diminui inicialmente como seria de esperar, mas começa a crescer. Ambas as fases ocorrem a taxas exponenciais em n , ou seja, $O(k^n)$, aparecendo linear em um gráfico semi-log.

3.9 Fenômeno de Runge

A decepcionante perda de convergência é um sinal de mau condicionamento devido ao uso de nós igualmente espaçados.

```
using Plots, LinearAlgebra
f = x -> 1/(x^2 + 16)
xx = range(-1, 1; length=400)
plot(xx, f.(xx); title="Função teste", label=false, lw=2)
```

Essa função possui infinitamente muitas derivadas contínuas em toda a linha real e parece fácil de aproximar em $[-1, 1]$. Começamos fazendo interpolação polinomial equispacada para alguns pequenos valores de n .

```
using Plots, Polynomials
x = range(-1, 1; length=2501)
plt = plot(xlabel="x", ylabel="|f-p|", yscale=:log10, title="Erro para graus baixos",
           ylims=(1e-20, 1), legend=:bottomright)
for n in 4:4:12
    tt = range(-1, 1; length=n+1)
    pp = fit(tt, f.(tt))
    plot!(plt, x, abs.(f.(x) .- pp.(x)); label="grau $n", lw=2)
end
plt
```

A convergência até agora parece bastante boa, embora não seja uniformemente. No entanto, observe o que acontece à medida que continuamos aumentando o grau.

```
plt = plot(xlabel="x", ylabel="|f-p|", yscale=:log10, title="Erro para graus altos",
           ylims=(1e-20, 1), legend=:bottomright)
for n in @. 12 + 15*(1:3)
    tt = range(-1, 1; length=n+1)
    pp = fit(tt, f.(tt))
    plot!(plt, x, abs.(f.(x) .- pp.(x)); label="grau $n", lw=2)
end
plt
```

A convergência no meio não pode ficar melhor do que a precisão da máquina em relação aos valores da função. Portanto, a manutenção da lacuna crescente entre o centro e as extremidades empurra as curvas de erro exponencialmente rapidamente nas extremidades, destruindo a convergência.

A observação da instabilidade é o **fenômeno de Runge**: nós igualmente espaçados + grau polinomial crescente $\Rightarrow$ erro explode perto das extremidades do intervalo.

Ideia-chave. A falha não é “polinômios são ruins”, e sim **nós equidistantes em grau alto**. Redistribuir nós (Chebyshev) troca um pouco de precisão no centro por estabilidade global.

Uma família de nós que fornece convergência estável para a interpolação polinomial é os pontos Chebyshev do segundo tipo definido por:

$$t_k = -\cos\left(\frac{k\pi}{n}\right), \quad k = 0, \dots, n.$$

```

x = range(-1, 1; length=2001)
plt = plot(xlabel="x", ylabel="|f-p|", yscale=:log10, title="Erro com os pontos
de Chebyshev",
           ylims=(1e-20, 1), legend=:bottomright)
for n in [4, 10, 16, 40]
    tt = [-cos(pi * k / n) for k in 0:n]
    pp = fit(tt, f.(tt))
    plot!(plt, x, abs.(f.(x) .- pp.(x)); label="grau $n", lw=2)
end
plt

```

A partir do grau 16, o erro está dentro da precisão de máquina e ele permanece lá à medida que n aumenta.

3.9.1 Exercício

1. Para cada caso, calcule o interpolante polinomial usando n nós de Chebyshev do segundo tipo em $[-1, 1]$ por $n = 4, 8, 12, \dots, 60$. Em cada valor de n , calcule o erro (ou seja, $\max |p(x) - f(x)|$) avaliado em 4000 valores de x . Usando uma escala log-linear, plote o erro em função de n e, em seguida, determine uma boa aproximação à constante k de $O(n^{-k})$.

(a) $f(x) = 1/(25x^2 + 1)$ (b) $f(x) = \tanh(5x + 2)$ (c) $f(x) = \cosh(\sin x)$ (d) $f(x) = \sin(\cosh x)$

1. **Equidistante vs Chebyshev (lado a lado).** Considere

$$f(x) = \frac{1}{1 + 25x^2}$$

em $[-1, 1]$. Para $n = 8, 16, 32$:

- (a) monte o interpolante polinomial com **nós equidistantes**;
- (b) monte o interpolante com **nós de Chebyshev do segundo tipo** $t_k = -\cos(k\pi/n)$.

Em cada n , estime $|f - p|_\infty$ em pelo menos 2000 pontos de teste. Entregue:

1. um gráfico de f , p_{eq} e p_{Cheb} para $n = 16$;
2. um gráfico (eixo y em escala log) de $|f - p|_\infty$ vs n para as duas famílias de nós;
3. uma frase respondendo: **em que regime o nó equidistante ainda é aceitável?**

```

using Plots, Polynomials
f = x -> 1/(1 + 25x^2)
x_test = range(-1, 1; length=2501)
ns = [8, 16, 32]
err_eq = Float64[]
err_ch = Float64[]
for n in ns
    t_eq = range(-1, 1; length=n+1)
    t_ch = [-cos(pi * k / n) for k in 0:n]
    p_eq = fit(collect(t_eq), f.(t_eq))
    p_ch = fit(t_ch, f.(t_ch))
    push!(err_eq, maximum(abs, f.(x_test) .- p_eq.(x_test)))
    push!(err_ch, maximum(abs, f.(x_test) .- p_ch.(x_test)))
end
# exemplo de figura para n = 16
n = 16
t_eq = range(-1, 1; length=n+1)
t_ch = [-cos(pi * k / n) for k in 0:n]

```

```

p_eq = fit(collect(t_eq), f.(t_eq))
p_ch = fit(t_ch, f.(t_ch))
xx = range(-1, 1; length=800)
plot(xx, f.(xx); label="f", lw=2, title="n = 16")
plot!(xx, p_eq.(xx); label="equidistante", ls=:dash, lw=2)
plot!(xx, p_ch.(xx); label="Chebyshev", ls=:dot, lw=2)
plot(ns, err_eq; yscale=:log10, marker=:circle, label="equidistante",
      xlabel="n", ylabel="||f-p||∞", lw=2)
plot!(ns, err_ch; marker=:square, label="Chebyshev", lw=2)

```

3.10 Integração numérica

A primitiva de e^x é simples, isso torna a avaliação de $\int_{-1}^1 e^x dx$ pelo teorema fundamental trivial.

```

using FastGaussQuadrature, LinearAlgebra
exato = exp(1)-exp(-1)

x, w = gausslegendre(3)
#([-0.7745966692414834, 0.0, 0.7745966692414834], [0.5555555555555556,
0.8888888888888888, 0.5555555555555556])

f(x) = exp(x)

In = dot(w, f.(x))
exato-In

```

A abordagem numérica é muito robusta. Por exemplo, $e^{\sin x}$ não tem primitiva conhecida. Mas numericamente, sua integral não é mais difícil de ser calculada.

```

f(x) = exp(sin(x))
In = dot(w, f.(x))

```

Quando você olha para os gráficos dessas funções, o que é notável é que uma dessas áreas é muito simples, enquanto o outro é analiticamente muito difícil. Do ponto de vista numérico, eles são praticamente o mesmo problema.

```

using Plots
xx = range(-1, 1; length=400)
p1 = plot(xx, exp.(xx); fillrange=0, fillalpha=0.25, label=false,
          xlabel="x", ylabel="exp(x)", ylims=(0, 2.7), lw=2)
p2 = plot(xx, exp.(sin.(xx)); fillrange=0, fillalpha=0.25, label=false,
          xlabel="x", ylabel="exp(sin(x))", ylims=(0, 2.7), lw=2)
plot(p1, p2; layout=(2, 1), size=(700, 500))

```

A integração numérica (**quadratura**) combina valores do integrando amostrados em nós. Nesta seção, primeiro usamos nós igualmente espaçados:

$$t_i = a + ih, \quad h = \frac{b-a}{n}, \quad i = 0, \dots, n.$$

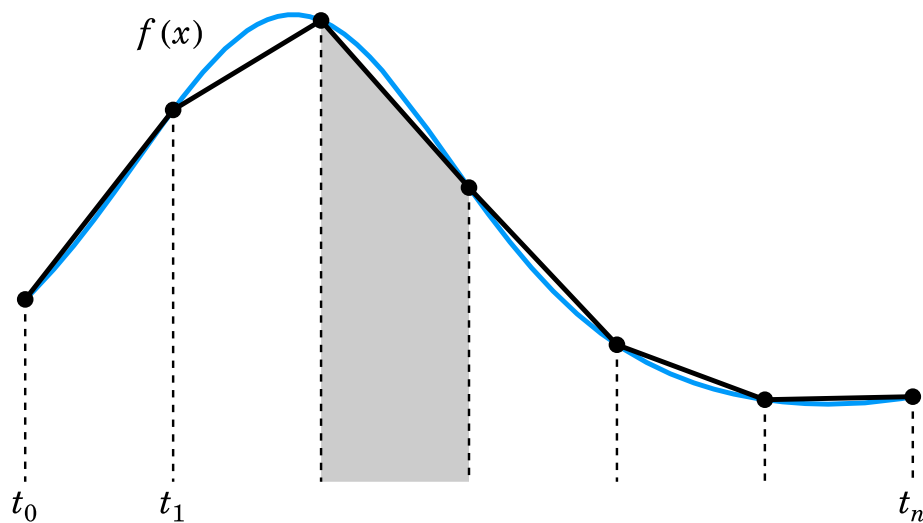
A Integração numérica consiste em uma lista de pesos $w_0, \dots, w_N$ escolhidos de modo que:

$$\int_a^b f(x) dx \approx h \sum_{i=0}^n w_i f(t_i) = h[w_0 f(t_0) + w_1 f(t_1) + \dots w_n f(t_n)].$$

Uma maneira direta de derivar fórmulas de integração é encontrar um interpolante e operar exatamente nele.

3.10.1 Regra do trapézio

Uma das fórmulas de integração mais importantes resulta da integração do interpolante linear por partes. Geometricamente, a fórmula trapezoidal de áreas de trapézoides que se aproximam da região sob a curva $y = f(x)$



Usando áreas de triângulos, é trivial derivar que

$$w_i = \begin{cases} 1, & i = 1, \dots, n-1, \\ \frac{1}{2}, & i = 0, n. \end{cases}$$

```

"""
    trapezoidal(f,a,b,n)

    Aplique a fórmula de integração do trapezoidal no intervalo [a, b], quebrado
    em partes iguais. Retorna a estimativa, um vetor de nós e um vetor de valores do
    integrando nos nós.
"""

function trapezoidal(f,a,b,n)
    h = (b-a)/n
    t = range(a,b,length=n+1)
    y = f.(t)
    T = h * ( sum(y[2:n]) + 0.5*(y[1] + y[n+1]) )
    return T,t,y
end

```

Vamos aproximar $f(x) = e^{\sin 7x}$ no intervalo $[0, 2]$.


```

f = x -> exp(sin(7*x));
a = 0; b = 2;

Integral = 2.6632197827615394 #exato
T,t,y = trapezoidal(f,a,b,40)
@show (T,Integral-T);

n = [ 10^n for n in 1:5 ]
err = []
for n in n
    T,t,y = trapezoidal(f,a,b,n)
    push!(err,Integral-T)
end

foreach(args -> println(args[1], " ", args[2]), zip(n, err))

```

Cada aumento em um fator de 10 em n reduz o erro em um fator de cerca de 100, o que é consistente com a convergência de segunda ordem.

3.10.2 Quadratura de Gauss

Vamos considerar a fórmula de integração numérica genérica:

$$\int_{-1}^1 f(x)dx \approx \sum_{k=1}^n w_k f(t_k) = Q_n[f],$$

Como existem n nós e n pesos disponíveis para escolher, parece plausível esperar que a quadratura consiga representar exatamente os polinômios de grau $m = 2n - 1$, e essa intuição acaba sendo correta.

Vamos testar isso na integral:

$$\int_{-1}^1 \frac{1}{1+4x^2} dx = \arctan(2).$$

```

f = x->1/(1+4*x^2);
exato = atan(2);

n = 8:4:96
errT = zeros(size(n))
errG = zeros(size(n))
for (k,n) in enumerate(n)
    errT[k] = abs(exato - trapezoidal(f,-1,1,n)[1])
    x, w = gausslegendre(n)
    errG[k] = abs(exato - dot(w, f.(x)))
end

errT[iszero.(errT)] .= NaN
errG[iszero.(errG)] .= NaN
using Plots
plot(collect(n), errT; yscale=:log10, xlabel="nós", ylabel="erro",
     title="integração numérica", ylims=(1e-16, 1),
     marker=:circle, label="trapézio", lw=2)
plot!(collect(n), errG; marker=:circle, label="Gauss-Legendre", lw=2)

```

E agora com uma integral com um integrando mais pontudo:

$$\int_{-1}^1 \frac{1}{1+16x^2} dx = \frac{1}{2} \arctan(4).$$

```
f = x->1/(1+16*x^2);
exato = atan(4)/2;

n = 8:4:96
errT = zeros(size(n))
errG = zeros(size(n))
for (k,n) in enumerate(n)
    errT[k] = abs(exato - trapezoidal(f,-1,1,n)[1])
    x, w = gausslegendre(n)
    errG[k] = abs(exato - dot(w, f.(x)))
end

errT[iszero.(errT)] .= NaN
errG[iszero.(errG)] .= NaN
using Plots
plot(collect(n), errT; yscale=:log10, xlabel="n", ylabel="erro",
     title="integração numérica", ylims=(1e-16, 1),
     marker=:circle, label="trapézio", lw=2)
plot!(collect(n), errG; marker=:circle, label="Gauss-Legendre", lw=2)
```

3.10.3 Exercícios

- Usando a mudança de variável: $z = \phi(x) = a + (b-a)\frac{(x+1)}{2}$ a integral pode ser reescrita como: $\int_a^b f(z) dz = \frac{b-a}{2} \int_{-1}^1 f(\phi(x)) dx$. Sabendo disso, use a quadratura de Gauss para integrar $\int_{\pi/2}^{\pi} x^2 \sin 8x dx = -\frac{3\pi^2}{32}$. Mostre resultados para diferentes valores de n até que se obtenha uma convergência de 10 dígitos.
- Integre numericamente a função $f(x) = -x \log(|x - 1/2|)$ no intervalo $[0, 1/2]$ usando a quadratura de Gauss. Use 5, 10, 15 e 20 pontos de Gauss.

$$I_a = \int_0^{1/2} -x \log\left(\left|x - \frac{1}{2}\right|\right) dx = \frac{1}{16}(3 + 2 \log 2) = 0,274143$$

3.11 Integrais singulares ou quasi-singulares

A última função integrada tem uma singularidade quando $x = 1/2$. Mesmo com essa singularidade, e tendendo a infinito nesse ponto, essa função pode ser integrada analiticamente. Numericamente é interessante usar técnicas especiais para tratar de integrais desses tipo.

Considere uma integral do tipo:

$$I_s = \int_{-1}^1 \frac{g(\xi)}{(\xi - a)^2 + b^2} d\xi$$

Uma ideia para avaliar numericamente integrais singulares ($b = 0$) ou quasi-singulares ($b \neq 0$) é encontrar uma transformação cujo Jacobiano seja zero ou bem próximo de zero no ponto singular. Por exemplo, para integrais quasi-singulares podemos usar:

$$\xi = a + b \sinh(\mu s - \eta)$$

onde

$$\mu = \frac{1}{2} \left\{ \operatorname{arcsinh} \left(\frac{1+a}{b} \right) + \operatorname{arcsinh} \left(\frac{1-a}{b} \right) \right\}$$

$$\eta = \frac{1}{2} \left\{ \operatorname{arcsinh} \left(\frac{1+a}{b} \right) - \operatorname{arcsinh} \left(\frac{1-a}{b} \right) \right\}$$

e seu jacobiano é dado por:

$$\frac{d\xi}{ds} = b\mu \cosh(\mu s - \eta)$$

Aplicando essa transformação, obtém-se:

$$I_s = \int_{-1}^1 \frac{g(s(\xi))}{(s(\xi) - a)^2 + b^2} \frac{d\xi}{ds} ds$$

Essa transformação anula (ou reduz) o Jacobiano perto da singularidade, suaviza o integrando e acelera a convergência da quadratura — o mesmo espírito das integrais singulares do BEM.

Ponte com o BEM. Soluções fundamentais geram núcleos $\log r$, $1/r$, etc. no contorno. Transformações do tipo sinh / Monegato–Sloan (interior) / Sato (extremo) são o dia a dia da montagem de H e G .

```
function sinhtrans(u, a, b)
    mu = 1 / 2 * (asinh((1 + a) / b) + asinh((1 - a) / b))
    eta = 1 / 2 * (asinh((1 + a) / b) - asinh((1 - a) / b))

    x = a .+ b * sinh.(mu * u .- eta)
    J = b * mu * cosh.(mu * u .- eta)
    x, J
end
```

```
using Plots

"""
    plot_quasising_sinh(a=0.0, b=0.05; g=nothing, n=800)

Mostra o integrando quase-singular em  $\xi \in [-1,1]$ 

 $f(\xi) = g(\xi) / ((\xi - a)^2 + b^2)$ 

e o integrando *transformado* na variável  $s \in [-1,1]$ 

 $f(\xi(s)) \cdot |d\xi/ds|$ 

com a transformação `sinhtrans`. O pico alto e estreito vira uma curva bem mais regular (o Jacobiano anula-se perto da quase-singularidade).
"""
function plot_quasising_sinh(a=0.0, b=0.05; g=nothing, n=800)
    g === nothing && (g = xi -> 1.0)
    f(xi) = g(xi) / ((xi - a)^2 + b^2)
```

```

ξ = range(-1, 1; length=n)
s = range(-1, 1; length=n)
ξs, J = sinhtrans(collect(s), a, b)
before = f.(ξ)
after = @. f(ξs) * abs(J)

p1 = plot(ξ, before; lw=2, label="f(ξ)",
          xlabel="ξ", ylabel="integrando",
          title="Antes (quase-singular)", legend=:topright)
vline!(p1, [a]; ls=:dash, color=:gray, label="a = $a")

p2 = plot(s, after; lw=2, label="f(ξ(s)) · |J|",
          xlabel="s", ylabel="integrando transformado",
          title="Depois (× Jacobiano sinh)", legend=:topright)
# marca s* tal que ξ(s*) ≈ a (J mínimo)
_, i = findmin(abs.(ξs .- a))
vline!(p2, [s[i]]; ls=:dash, color=:gray, label="ξ(s) ≈ a")

plot(p1, p2; layout=(1, 2), size=(900, 350))
end

# mapa ξ(s) e Jacobiano
x = range(-1, 1; length=100)
xt, jac = sinhtrans(x, 0.5, 0.01)
p1 = plot(collect(x), xt; xlabel="s", ylabel="ξ", label=false, lw=2)
p2 = plot(collect(x), jac; xlabel="s", ylabel="dξ/ds", label=false, lw=2)
plot(p1, p2; layout=(2, 1), size=(700, 500))

# integrando antes × depois
plot_quasising_sinh(0.0, 0.05)
plot_quasising_sinh(0.5, 0.02) # pico deslocado

```

aplicando essa técnica para a integral do último exemplo pode-se observar uma redução significativa do erro para a mesma quantidade de pontos.

```

f = x->1/(1+16*x^2);
exato = atan(4)/2;

n = 8

x, w = gausslegendre(n)
xt, J = sinhtrans(x, 0, 1/4)
errG = abs(exato - dot(w, f.(x)))
errGt = abs(exato - dot(w, J.*f.(xt)))

```

1. **Quase-singular + sinhtrans (tabela b ↓).** Considere

$$I(b) = \int_{-1}^1 \frac{1}{\xi^2 + b^2} d\xi, \quad a = 0,$$

com valor exato

$$I(b) = \left(\frac{1}{b}\right) \left[\arctan\left(\frac{1}{b}\right) + \arctan\left(\frac{1}{b}\right) \right] = \left(\frac{2}{b}\right) \arctan\left(\frac{1}{b}\right).$$

Para $b \in \{10^{-1}, 10^{-2}, 10^{-3}\}$ e Gauss–Legendre com $n \in \{8, 16, 32\}$:

1. calcule o erro absoluto **sem** transformação (integrando amostrado direto em ξ);
2. calcule o erro **com** `sinhtrans(s, 0, b)`, usando o integrando $f(\xi(s)) |J(s)|$;
3. monte uma tabela $(b, n, e_{\text{cru}}, e_{\text{sinh}})$;
4. para o menor b , gere `plot_quasising_sinh(0, b)` e comente o achatamento do pico.

Ponte BEM: b pequeno imita fonte **perto** do elemento (integral quase-singular na montagem de H e G).

```
using FastGaussQuadrature, LinearAlgebra, Plots
Iex(b) = (2/b) * atan(1/b)
f(ξ, b) = 1/(ξ^2 + b^2)

bs = (1e-1, 1e-2, 1e-3)
ns = (8, 16, 32)
println("b\t n\t err_cru\t err_sinh")
for b in bs, n in ns
    s, w = gausslegendre(n)
    # sem transformação
    e_cru = abs(Iex(b) - dot(w, f.(s, b)))
    # com sinhtrans (a = 0)
    ξ, J = sinhtrans(s, 0.0, b)
    e_sinh = abs(Iex(b) - dot(w, f.(ξ, b) .* abs.(J)))
    println("$b\t $n\t $e_cru\t $e_sinh")
end
plot_quasising_sinh(0.0, 1e-3)
```

3.11.1 Exercício

1. Integre de novo $f(x) = -x \log(|x - 1/2|)$ em $[0, 1/2]$ com Gauss, agora usando a transformação abaixo. A singularidade está no **extremo** $x = 1/2$: mapeie o intervalo para $[-1, 1]$ (afim) e chame Monegato com $s0 = 1$ (ramo Sato). Compare graus $m = 3, 4, 5$ e $n = 5, 10, 15, 20$ pontos de Gauss com o valor I_a do exercício anterior. (Opcional: repita com uma singularidade **interior** artificial, p.ex. estendendo o intervalo, e graus **ímpares** $q = 3, 5$.)

```
"""
    Monegato(t, s0, q=5)

Transformação em `t ∈ [-1,1]` → `s ∈ [-1,1]`, com Jacobiano `ds/dt`.

- `|s0| < 1` (interior): Monegato–Sloan de grau *ímpar* `q ≥ 3`.
- `s0 ≈ ±1` (extremo): Sato de grau `q ≥ 2` (par ou ímpar).

Retorna `(s, ds)`.
"""
function Monegato(t, s0, q::Integer=5; atol=1e-14)
    t = float(t)
    s0 = float(s0)
    # --- extremo: Sato ---
    if isapprox(s0, 1; atol=atol)
        q ≥ 2 || throw(ArgumentError("Sato no extremo +1 exige q ≥ 2"))
        # Φ(t) = 1 - (1-t)^q / 2^(q-1)    (suaviza s = +1)
        u = @. 1 - t
    end
```

```

s = @. 1 - u^q / 2^(q - 1)
ds = @. q * u^(q - 1) / 2^(q - 1)
return s, ds
elseif isapprox(s0, -1; atol=atol)
q ≥ 2 || throw(ArgumentError("Sato no extremo -1 exige q ≥ 2"))
# espelho: suaviza s = -1
u = @. 1 + t
s = @. -1 + u^q / 2^(q - 1)
ds = @. q * u^(q - 1) / 2^(q - 1)
return s, ds
end
# --- interior: Monegato-Sloan ---
(-1 < s0 < 1) || throw(ArgumentError("s0 deve estar em (-1,1) ou ±1"))
(q ≥ 3 && isodd(q)) || throw(ArgumentError(
    "Monegato-Sloan interior exige grau ímpar q = 3,5,7,... (recebeu q=$q)"))
aq = (1 + s0)^(1 / q)
bq = (1 - s0)^(1 / q)
δ = (1 / 2)^q * (aq + bq)^q
t0 = (aq - bq) / (aq + bq)
u = @. t - t0
s = @. s0 + δ * u^q
ds = @. q * δ * u^(q - 1)
return s, ds
end

```

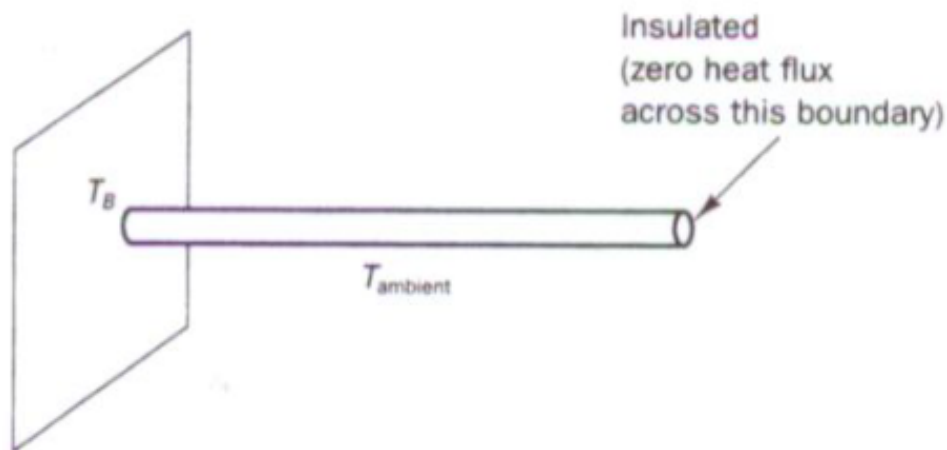
1. Use 10 pontos de Gauss para calcular as integrais abaixo. Compare com a solução analítica. Quais integrais tiveram os maiores erros? Por que?

$$= -0.666666666666667; \int_0^{\pi} (\sin(3x) \cdot \cos(2x) + 2x^3 + 3x \sin(x)) dx, 59.3293234777059; \int_0^1 \frac{1}{x+1} dx, \ln(2) = 0.6931471805599$$

3.12 Desafio

Considere o resfriamento de uma aleta circular por meio de transferência de calor por convecção ao longo de seu comprimento. A convecção dá origem a uma perda de calor ou termo de sumidouro dependente da temperatura na equação governante. Mostrada na Figura está uma aleta cilíndrica com área de seção transversal uniforme A . A base está a uma temperatura de 100C (TB) e a extremidade direita está isolada. A aleta está exposta a uma temperatura ambiente de 20C. A transferência de calor unidimensional nesta situação é governada por $\frac{d}{dx} \left(kA \frac{dT}{dx} \right) - hP(T - T_{\infty}) = 0$

onde h é o coeficiente de transferência de calor por convecção, P o perímetro, k a condutividade térmica do material e T_{∞} a temperatura ambiente. Calcule usando BEM a distribuição de temperatura ao longo da aleta e compare os resultados com a solução analítica fornecida por $\frac{T - T_{\infty}}{T_B - T_{\infty}} = \frac{\cosh[n(L-x)]}{\cosh(nL)}$



4 Equações diferenciais

Gráficos deste capítulo usam **Plots.jl**. Pacotes: DifferentialEquations, Plots, LinearAlgebra.

4.1 Objetivos

Ao final, você deve ser capaz de:

1. Formular um PVI escalar e resolvê-lo com DifferentialEquations e com Euler / RK4 próprios.
2. Medir ordem de convergência em gráfico log-log.
3. Reduzir uma EDO de 2ª ordem a um sistema de 1ª ordem.
4. Montar matrizes de diferenciação D_x , D_{xx} , impor Dirichlet e resolver um PVC 1D.
5. Integrar no tempo o sistema semi-discreto (método das linhas) para a equação do calor — em MDF e em BEM 1D.
6. Explicar a forma $M\dot{T} + HT = GQ$ e extrair um PVI nas incógnitas de contorno.

4.2 Mapa do capítulo

1. PVI e o solucionador Tsit5
2. Euler (ordem 1) e estudo de erro
3. Runge-Kutta 4
4. Sistemas e redução de ordem
5. Matrizes de diferenças finitas e PVC estacionário
6. Método das linhas: difusão (MDF)
7. Equação da onda (método das linhas)
8. Ponte com o BEM e método das linhas no contorno
9. Exercícios
10. (Extra) multi-passo AB4 e Houbolt

4.3 PVI escalar

Quantidades que mudam no tempo (ou ao longo de um parâmetro) são modeladas por equações diferenciais. Condições suplementares fecham o problema: no **problema de valor inicial** (PVI) tudo é prescrito em um único valor da variável independente — em geral o tempo.

PVI escalar de primeira ordem:

$$u'(t) = f(t, u(t)), \quad a \leq t \leq b,$$

$$u(a) = u_0.$$

- t : variável independente; u : dependente.

- Se $f(t, u) = g(t) + h(t)u$, a EDO é **linear**; caso contrário, **não linear**.
- Solução: função $u(t)$ que satisfaz a EDO e a condição inicial.
- Notação no tempo: às vezes $\dot{u}$ em vez de u' .

Ideia-chave. Métodos de PVI **avancam** a partir de $u(a)$: a cada passo usam f (a inclinação) para construir u em tempos futuros. A qualidade depende da **ordem**, da **estabilidade** e do tamanho de passo h .

4.3.1 Com DifferentialEquations.jl

Exemplo: $u' = \sin((u + t)^2)$, $t \in [0, 4]$, $u(0) = 1$.

A API pede $f(u, p, t)$ — o argumento p carrega parâmetros constantes (mesmo que não usemos).

```
using DifferentialEquations, Plots, LinearAlgebra

f = (u, p, t) -> sin((t + u)^2)
u0 = 1.0
tspan = (0.0, 4.0)

ivp = ODEProblem(f, u0, tspan)
sol = solve(ivp, Tsit5())

plot(sol.t, sol.u; xlabel="t", ylabel="u(t)", label="solução", lw=2)
scatter!(sol.t, sol.u; label="nós adaptativos", ms=3)
```

O objeto `sol` é avaliável em qualquer t (`sol(1.0)`): por baixo há uma malha adaptativa e interpolação. O restante do capítulo explica **como** se constroem valores discretos (t_i, u_i) — com passo fixo, para controlar ordem.

4.4 Método de Euler

Discretize o tempo em passos iguais:

$$t_i = a + ih, \quad h = \frac{b-a}{n}, \quad i = 0, \dots, n.$$

Seja $\hat{u}(t)$ a solução **exata** e $u_i \approx \hat{u}(t_i)$ a aproximação numérica.

Ideia: no intervalo $[t_i, t_{i+1}]$, a inclinação do interpolante linear por partes é $\frac{u_{i+1} - u_i}{h}$. Iguale a $f(t_i, u_i)$:

$$u_{i+1} = u_i + hf(t_i, u_i).$$

Isso é o **método de Euler explícito** (ordem 1).

```
"""
    euler(ivp, n) -> t, U

Euler explícito com n passos. `U[i]` é o estado em t[i]
(escalar ou vetor, conforme `ivp.u0`).
"""
function euler(ivp, n)
    a, b = ivp.tspan
    h = (b - a) / n
    t = [a + i*h for i in 0:n]
```



```

u0 = ivp.u0 isa Number ? float(ivp.u0) : float.(ivp.u0)
U = Vector{typeof(u0)}(undef, n + 1)
U[1] = u0
for i in 1:n
    U[i+1] = U[i] + h * ivp.f(U[i], ivp.p, t[i])
end
return t, U
end

```

4.4.1 Exemplo e convergência

Mesma EDO, agora $u(0) = -1$:

```

f = (u, p, t) -> sin((t + u)^2)
ivp = ODEProblem(f, -1.0, (0.0, 4.0))

t20, u20 = euler(ivp, 20)
t50, u50 = euler(ivp, 50)
u_ref = solve(ivp, Tsit5(); reltol=1e-14, abstol=1e-14)

plot(t20, u20; marker=:circle, label="Euler n=20", xlabel="t", ylabel="u", lw=2)
plot!(t50, u50; marker=:circle, label="Euler n=50", lw=2)
plot!(t50, u_ref.(t50); color=:black, lw=2, label="referência")

```

Estudo de erro em norma do máximo nos nós:

```

ns = [round{Int, 5 * 10^k} for k in 0:0.5:3]
err_E = Float64[]
for n in ns
    t, U = euler(ivp, n)
    push!(err_E, maximum(abs, u_ref.(t) .- U))
end

plot(ns, err_E; xscale=:log10, yscale=:log10, marker=:circle,
    label="Euler", xlabel="n", ylabel="||e||∞", lw=2)
plot!(ns, err_E[1] .* (ns[1] ./ ns); ls=:dash, label="O(1/n)")

```

Ideia-chave — ordem. Se o erro global se comporta como $O(h^p) = O(n^{-p})$, no plano $\log n \times \log \text{erro}$ a reta tem inclinação $-p$. Euler tem $p = 1$: multiplicar n por 10 reduz o erro cerca de $10\times$.

4.5 Runge–Kutta de 4ª ordem

Euler usa **uma** avaliação de f por passo. Métodos de Runge–Kutta (RK) combinam várias avaliações (**estágios**) no intervalo $[t_i, t_{i+1}]$ para subir a ordem.

O RK clássico de ordem 4:

$$\begin{aligned}
k_1 &= hf(t_i, u_i), \\
k_2 &= hf\left(t_i + \frac{h}{2}, u_i + \frac{k_1}{2}\right), \\
k_3 &= hf\left(t_i + \frac{h}{2}, u_i + \frac{k_2}{2}\right), \\
k_4 &= hf(t_i + h, u_i + k_3), \\
u_{i+1} &= u_i + \frac{k_1 + 2k_2 + 2k_3 + k_4}{6}.
\end{aligned}$$

```

"""RK4 clássico, n passos. Aceita estado escalar ou vetorial."""
function rk4(ivp, n)
    a, b = ivp.tspan
    h = (b - a) / n
    t = [a + i*h for i in 0:n]
    u0 = ivp.u0 isa Number ? float(ivp.u0) : float.(ivp.u0)
    U = Vector{typeof(u0)}(undef, n + 1)
    U[1] = u0
    for i in 1:n
        ui, ti = U[i], t[i]
        k1 = h * ivp.f(ui, ivp.p, ti)
        k2 = h * ivp.f(ui + k1/2, ivp.p, ti + h/2)
        k3 = h * ivp.f(ui + k2/2, ivp.p, ti + h/2)
        k4 = h * ivp.f(ui + k3, ivp.p, ti + h)
        U[i+1] = ui + (k1 + 2k2 + 2k3 + k4)/6
    end
    return t, U
end

```

Compare ordens no mesmo PVI:

```

err_R = Float64[]
for n in ns
    t, U = rk4(ivp, n)
    push!(err_R, maximum(abs, u_ref.(t) .- U))
end

plot(ns, err_E; xscale=:log10, yscale=:log10, marker=:circle, label="Euler p≈1",
     lw=2)
plot!(ns, err_R; marker=:square, label="RK4 p≈4", lw=2)
plot!(ns, err_E[1] .* (ns[1] ./ ns); ls=:dash, label="O(n⁻¹)")
plot!(ns, err_R[1] .* (ns[1] ./ ns).^4; ls=:dot, label="O(n⁻⁴)")

```

Euler melhorado (RK de ordem 2, um estágio intermediário em $t_i + h/2$) fica como leitura opcional; a trilha do curso usa Euler (referência de ordem 1) e RK4 (cavalo de batalha explícito).

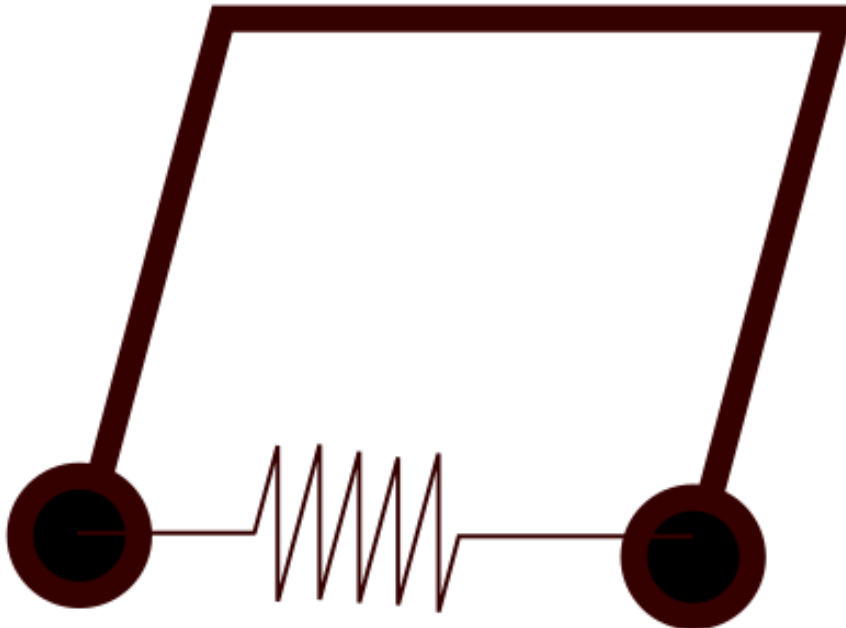
4.6 Sistemas e redução de ordem

Poucas aplicações são escalares. O mesmo Euler/RK4 vale para $\mathbf{u} \in \mathbb{R}^d$ se f devolver um vetor:

$$\mathbf{u}_{i+1} = \mathbf{u}_i + hf(t_i, \mathbf{u}_i) \quad (\text{Euler})$$

EDOs de ordem m viram sistemas de 1ª ordem introduzindo derivadas inferiores como novas incógnitas.

4.6.1 Pêndulos acoplados (2ª ordem → 1ª)



$$\begin{aligned}\theta_1'' + \gamma\theta_1' + (g/L)\sin\theta_1 + k(\theta_1 - \theta_2) &= 0, \\ \theta_2'' + \gamma\theta_2' + (g/L)\sin\theta_2 + k(\theta_2 - \theta_1) &= 0.\end{aligned}$$

Com $u = (\theta_1, \theta_2, \theta_1', \theta_2')$:

$$\begin{aligned}u_{1'} &= u_3, \quad u_{2'} = u_4, \\ u_{3'} &= -\gamma u_3 - (g/L)\sin u_1 + k(u_2 - u_1), \\ u_{4'} &= -\gamma u_4 - (g/L)\sin u_2 + k(u_1 - u_2).\end{aligned}$$

```
function couple(u, p, t)
    γ, L, k = p
    g = 9.8
    du = similar(u)
    du[1] = u[3]
    du[2] = u[4]
    du[3] = -γ*u[3] - (g/L)*sin(u[1]) + k*(u[2] - u[1])
    du[4] = -γ*u[4] - (g/L)*sin(u[2]) + k*(u[1] - u[2])
    return du
end

u0 = [1.25, -0.5, 0.0, 0.0]
tspan = (0.0, 50.0)
γ, L = 0.0, 0.5

sol0 = solve(ODEProblem(couple, u0, tspan, [γ, L, 0.0]), Tsit5())
sol1 = solve(ODEProblem(couple, u0, tspan, [γ, L, 1.0]), Tsit5())

plot(sol0.t, [u[1] for u in sol0.u]; label="θ1, k=0", xlims=(20, 50), lw=2)
plot!(sol0.t, [u[2] for u in sol0.u]; label="θ2, k=0", lw=2)
```

```
plot!(sol1.t, [u[1] for u in sol1.u]; label="θ1, k=1", ls=:dash, lw=2)
plot!(sol1.t, [u[2] for u in sol1.u]; label="θ2, k=1", ls=:dash, lw=2,
      xlabel="t", title="Pêndulos acoplados")
```

Receita. Para cada variável de ordem máxima m , crie m componentes $(y, y', \dots, y^{(m-1)})$. Relações cinemáticas $y' = v$, etc., + a EDO original fecham o sistema. Dimensão = soma das ordens.

4.7 Matrizes de diferenças finitas

Até aqui a variável independente era o **tempo**. No espaço, para um PVC ou para semi-discretizar uma EDP, aproximamos d/dx e d^2/dx^2 por **matrizes de diferenciação**.

Nós uniformes em $[a, b]$:

$$x_i = a + ih, \quad h = \frac{b-a}{n}, \quad i = 0, \dots, n.$$

Queremos $\mathbf{g} \approx f'(\mathbf{x})$ com $\mathbf{g} = D_x \mathbf{f}$. Usamos diferenças centradas de ordem 2 no interior e unilaterais de ordem 2 nos extremos:

$$D_x = \frac{1}{h} \begin{pmatrix} -3/2 & 2 & -1/2 & & \\ -1/2 & 0 & 1/2 & & \\ & \ddots & \ddots & \ddots & \\ & & -1/2 & 0 & 1/2 \\ & & 1/2 & -2 & 3/2 \end{pmatrix}.$$

Segunda derivada (ordem 2):

$$D_{xx} = \frac{1}{h^2} \begin{pmatrix} 2 & -5 & 4 & -1 & \\ 1 & -2 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & -2 & 1 \\ -1 & 4 & -5 & 2 \end{pmatrix}.$$

```
"""
    diffmat(n, xspan) -> x, Dx, Dxx

n = número de *intervalos* (n+1 nós) em xspan = (a,b).
Diferenças de ordem 2 (centradas no interior).
"""

function diffmat(n, xspan)
    a, b = xspan
    h = (b - a) / n
    x = [a + i*h for i in 0:n]
    # Dx
    dp = fill(0.5/h, n)
    dm = fill(-0.5/h, n)
    Dx = diagm(-1 => dm, 1 => dp)
    Dx[1, 1:3] = [-1.5, 2, -0.5] / h
    Dx[n+1, n-1:n+1] = [0.5, -2, 1.5] / h
    # Dxx
    d0 = fill(-2/h^2, n+1)
```

```

dp2 = ones(n) / h^2
Dxx = diagm(-1 => dp2, 0 => d0, 1 => dp2)
Dxx[1, 1:4] = [2, -5, 4, -1] / h^2
Dxx[n+1, n-2:n+1] = [-1, 4, -5, 2] / h^2
return x, Dx, Dxx
end

```

4.7.1 PVC estacionário (Laplace 1D)

$T''(x) = 0$ em $(-1, 1)$, $T(-1) = 100$, $T(1) = 0$. Solução exata: $T(x) = 50(1 - x)$.

Impor Dirichlet **substituindo linhas** de D_{xx} :

```

n = 40
x, Dx, Dxx = diffmat(n, (-1.0, 1.0))
A = copy(Dxx)
A[1, :] .= 0; A[1, 1] = 1
A[end, :] .= 0; A[end, end] = 1
rhs = zeros(n + 1)
rhs[1] = 100
rhs[end] = 0
T = A \ rhs
T_ex = @. 50 * (1 - x)

plot(x, T_ex; label="exato", lw=2, xlabel="x", ylabel="T")
scatter!(x, T; label="MDF", ms=3)
@show maximum(abs, T - T_ex)

```

Ideia-chave. $D_{xx}\mathbf{T} = \mathbf{b}$ é um sistema algébrico esparsos nos nós do **domínio**. No BEM 1D da apresentação, o sistema vive só nas **pontas** e usa $HT = GQ$. Mesma física, densidades de informação diferentes.

4.8 Método das linhas: equação do calor

EDP parabólica 1D:

$$u_t = \kappa u_{xx},$$

com $\kappa > 0$. Fixe a malha em x , substitua $u_{xx} \approx D_{xx}\mathbf{u}(t)$ e obtenha um **PVI grande** no tempo — o **método das linhas**.

Condições: $u(-1, t) = 0$, $u(1, t) = 2$, e

$$u(x, 0) = 1 + \sin(\pi x/2) + 3(1 - x^2)e^{-4x^2}.$$

```

function heat_rhs!(du, u, p, t)
    Dxx, κ, ua, ub = p
    u[1] = ua
    u[end] = ub
    mul!(du, Dxx, u)
    du .*= κ
    du[1] = 0
    du[end] = 0
end

```

```

    return nothing
end

n = 100
x, Dx, Dxx = diffmat(n, (-1.0, 1.0))
κ = 1.0
ua, ub = 0.0, 2.0
u0 = @. 1 + sin(pi*x/2) + 3*(1 - x^2)*exp(-4*x^2)
u0[1] = ua; u0[end] = ub

prob = ODEProblem(heat_rhs!, u0, (0.0, 0.75), (Dxx, κ, ua, ub))
sol = solve(prob, Tsit5())

plt = plot(xlabel="x", ylabel="u(x,t)", title="Calor / difusão", legend=:topleft)
for tt in 0:0.1:0.7
    plot!(plt, x, sol(tt); label="t=$tt", lw=2)
end
plt

```

Animação opcional (GIF local) ou vídeo do repositório:

```

anim = @animate for tt in range(0, 0.75; length=60)
    plot(x, sol(tt); xlabel="x", ylabel="u", ylims=(0, 4.2),
        legend=false, lw=2, title="t=$(round(tt; digits=3))")
end
gif(anim, "calor.gif"; fps=15)

```

Vídeo pronto: [boundaries-heat.mp4](#)

Os integradores do início do capítulo (Euler, RK4, Tsit5) aplicam-se a esse sistema; a estabilidade explícita exige $\Delta t = O(h^2)$ para o calor — por isso solucionadores **adaptativos** ou implícitos ajudam.

4.9 Equação da onda (método das linhas)

EDP hiperbólica 1D (corda / acústica unidimensional):

$$u_{tt} = c^2 u_{xx},$$

com velocidade de onda $c > 0$. Diferente do calor, a informação propaga com **velocidade finita** c e a energia (em domínio isolado) se conserva no contínuo.

4.9.1 Redução a 1ª ordem no tempo

Como no pêndulo, introduza a velocidade $v = u_t$:

$$u_t = v, \quad v_t = c^2 u_{xx}.$$

No espaço, $u_{xx} \approx D_{xx} \mathbf{u}$. O estado do PVI fica o **par** de vetores

$$\mathbf{y} = \begin{pmatrix} \mathbf{u} \\ \mathbf{v} \end{pmatrix} \in \mathbb{R}^{2(n+1)}$$

(ou só nós interiores, se as BC fixarem u nos extremos). Então

$$\dot{\mathbf{y}} = \begin{pmatrix} \mathbf{v} \\ c^2 D_{xx} \mathbf{u} \end{pmatrix}$$

com BC aplicadas em $\mathbf{u}$ (e $\dot{u} = v = 0$ nos extremos se Dirichlet homogêneo constante).

Checklist para implementar.

1. $\mathbf{x}$, $\mathbf{Dx}$, $\mathbf{Dxx} = \text{diffmat}(n, (a,b))$.
2. Estado $\mathbf{y} = \text{vcat}(\mathbf{u}, \mathbf{v})$ com $\text{length}(\mathbf{u})=\text{length}(\mathbf{v})=n+1$.
3. No RHS: separar $\mathbf{u} = \mathbf{y}[1:n+1]$, $\mathbf{v} = \mathbf{y}[n+2:\text{end}]$; impor BC em $\mathbf{u}$ (e zerar $\mathbf{v}$ nos nós de Dirichlet fixo); $\mathbf{du} = \mathbf{v}$, $\mathbf{dv} = c^2 * (\mathbf{Dxx}*\mathbf{u})$.
4. Integrar com Tsit5 (ou RK4 próprio).
5. CFL prático: com passo **fixo** explícito, $\Delta t \leq h/c$; com Tsit5 adaptativo o passo se ajusta, mas malha grossa demais ainda dispersa a onda.

4.9.2 Problema-modelo com solução exata

$$\begin{aligned}u_{tt} &= c^2 u_{xx}, \quad 0 < x < 1, \quad t > 0, \\u(0, t) &= u(1, t) = 0, \\u(x, 0) &= \sin(\pi x), \quad u_t(x, 0) = 0.\end{aligned}$$

Modo normal: $u_{\text{exata}}(x, t) = \sin(\pi x) \cos(c\pi t)$ (verifique BC, CI e a EDP).

Esqueleto MDF:

```
function wave_rhs!(dy, y, p, t)
    Dxx, c2, nnode = p
    u = @view y[1:nnode]
    v = @view y[nnode+1:end]
    du = @view dy[1:nnode]
    dv = @view dy[nnode+1:end]
    # Dirichlet homogêneo
    u[1] = 0.0
    u[end] = 0.0
    v[1] = 0.0
    v[end] = 0.0
    du .= v
    mul!(dv, Dxx, u)
    dv .*= c2
    du[1] = 0.0
    du[end] = 0.0
    dv[1] = 0.0
    dv[end] = 0.0
    return nothing
end

function solve_wave_mdf(; n=100, c=1.0, tspan=(0.0, 2.0))
    x, Dx, Dxx = diffmat(n, (0.0, 1.0))
    nnode = length(x)
    u0 = sin.(pi .* x)
    v0 = zeros(nnode)
    y0 = vcat(u0, v0)
    prob = ODEProblem(wave_rhs!, y0, tspan, (Dxx, c^2, nnode))
    sol = solve(prob, Tsit5(); reltol=1e-8, abstol=1e-8)
    return sol, x
end
```

BEM (só a **forma**, para o exercício): no contorno, algo como

$$M\mathbf{u}'' + H\mathbf{u} = G\mathbf{q}$$

com q ligado a $\partial_n u$; o método das linhas no contorno seria um sistema de 2ª ordem no tempo (ou 1ª ordem em $(u, \dot{u})$ nas faces). No E5 pede-se o MDF completo e apenas o esboço matricial BEM.

4.10 Ponte com o BEM

4.10.1 Duas semi-discretizações da mesma física

A equação do calor 1D $T_t = \kappa T_{xx}$ pode ser atacada de dois jeitos depois de discretizar o **espaço**:

Aspecto	MDF (domínio)	BEM (contorno)
Incógnitas primárias	T em todos os nós de $[x_0, x_f]$	T e/ou $Q = \partial T / \partial x$ só em x_0, x_f
Operador espacial	D_{xx} esparsa	matrizes cheias H, G da identidade integral
Tempo	$\dot{\mathbf{T}} = \kappa D_{xx} \mathbf{T}$ (+ BC)	$M\dot{\mathbf{T}} + H\mathbf{T} = G\mathbf{Q}$
Interior	já está no vetor de estado	pós-processamento pela fórmula de representação

Ideia-chave. O **método das linhas** é o mesmo em ambos: depois do espaço, sobra um PVI (ou DAE) no tempo. O que muda é **quem** são as incógnitas e **quais** matrizes multiplicam T, Q e $\dot{T}$.

4.10.2 De $HT = GQ$ para o calor

Na aula 1 (estacionário), $HT = GQ$ nas pontas ($n_c = 2$). Com $T_t = \kappa T''$ vem o domínio

$$\int_{x_0}^{x_f} T^*(x, x_d), \left(\dot{T}(x) / \kappa \right) dx.$$

4.10.3 Cada Gauss é fonte interior: bloco ($n_c + n_i$)

Gauss-Legendre em $[x_0, x_f]$: nós x_j^g , pesos w_j ($j = 1..n_i$). Cada x_j^g tem **dois** papéis:

1. ponto de **campo** na quadratura de $\dot{T}$;
2. ponto **fonte** (colocação interior, $c = 1$).

Colocações:

$$x_d = (x_0, x_f, x_1^g, \dots, x_{n_i}^g), \quad N = n_c + n_i.$$

Aproximação nodal por Gauss:

$$\int T^*(\dot{T}/\kappa) dx \approx \sum_{j=1}^{n_i} (w_j/\kappa) T^*(x_j^g, x_d^{\{(p)\}}) \dot{T}(x_j^g).$$

Isso define o bloco retangular $M^g \in \mathbb{R}^{N \times n_i}$. O sistema **quadrado** $(n_c + n_i) \times (n_c + n_i)$ do método das linhas fecha com as incógnitas de contorno livres (Q e/ou $\dot{T}$ nos extremos não prescritos) empilhadas às $\dot{T}^g$:

$$z = (Q_0, \dot{T}_f, \dot{T}(x_1^g), \dots, \dot{T}(x_{n_i}^g)) \in \mathbb{R}^N$$

no problema misto T_0 fixo ($\dot{T}_0 = 0$) e $Q_f = 0$. As N colocações fornecem $Az = r$.

Ideia-chave. Montar M não é “somar T^* nos Gauss e jogar numa diagonal”. Cada Gauss gera uma **linha** (fonte interior) e uma **coluna** (amostra de $\dot{T}$).

4.10.4 Código: bem1d_M e RHS ($n_c + n_i$)

```
using LinearAlgebra, Plots, FastGaussQuadrature, DifferentialEquations

Tstar(x, xd) = -0.5 * abs(x - xd)
Qstar(x, xd) = x == xd ? 0.0 : -0.5 * sign(x - xd)

function bem1d_HG(x0, xf)
    L = xf - x0
    H = [0.5 -0.5; -0.5 0.5]
    G = L * [0.0 -0.5; 0.5 0.0]
    return H, G, L
end

"""
    bem1d_M(x0, xf; κ=1.0, nq=6)

BEM 1D com `nq` Gauss como fontes/interiores.
N = nc + ni = 2 + nq colocações `xd = [x0, xf, xg...]\`.

Retorna:
- `Mg` : N×ni, Mg[p,j] = w[j]/κ * T*(xg[j], xd[p])
- `Hb, Gb` : N×2 (contorno clássico nas linhas 1:2; representação nas interiores)
"""
function bem1d_M(x0, xf; κ=1.0, nq=6)
    nc = 2
    ξ, ŵ = gausslegendre(nq)
    jac = (xf - x0) / 2
    xg = collect(@. (x0 + xf)/2 + jac * ξ)
    w = collect(ŵ .* jac)
    ni = nq
    xd = vcat([x0, xf], xg)
    N = nc + ni

    Mg = zeros(N, ni)
    for p in 1:N, j in 1:ni
        Mg[p, j] = w[j] / κ * Tstar(xg[j], xd[p])
    end

    H, G, _ = bem1d_HG(x0, xf)
    Hb = zeros(N, 2)
    Gb = zeros(N, 2)
    Hb[1:2, :] .= H
    Gb[1:2, :] .= G
    # interior p: T(xd) + Hb·Tb - Gb·Qb + Mg·Ṫg = 0
    # partindo de
    # -T = Tf Q*(xf) - T*(xf) Qf - T0 Q*(x0) + T*(x0) Q0 + ∑ Mg Ṫg
    for p in 3:N
        xdp = xd[p]
```

```

    Hb[p, 1] = -Qstar(x0, xdp)
    Hb[p, 2] = Qstar(xf, xdp)
    Gb[p, 1] = -Tstar(x0, xdp)
    Gb[p, 2] = Tstar(xf, xdp)
end
return (; x0, xf, xg, w, xd, Mg, Hb, Gb, nc, ni, N, κ)
end

```

Equações em cada colocação $p = 1..N$ (vetor $T^b = (T_0, T_f)$, $Q^b = (Q_0, Q_f)$, $T^g, \dot{T}^g$):

- contorno ($p = 1, 2$): $(HT^b - GQ^b)_p + (M^g \dot{T}^g)_p = 0$;
- interior ($p = 2 + i$): $T_i^g + (H_{p,:}^b T^b - G_{p,:}^b Q^b) + (M^g \dot{T}^g)_p = 0$.

Incógnitas (BC mista: T_0 fixo, $Q_f = 0$). O sistema **reduzido** de ordem $N - 1$ resolve $z = (Q_0, \dot{T}^g) \in \mathbb{R}^{n_i+1}$ com as colocações em x_0 e em todos os Gauss (linhas 1 e $3..N$). Como o Gauss aberto não coloca massa em x_f , define-se $\dot{T}_f$ pelo $\dot{T}$ no Gauss mais próximo de x_f (no código: `argmax(xg)`). Para acoplar $\dot{T}_f$ à massa de fato, use Gauss–Lobatto ou DIBEM.

4.10.5 Método das linhas (implementação)

Estado $y = (T_f, T_1^g, \dots, T_{n_i}^g) \in \mathbb{R}^{N-1}$.

A matriz A do sistema em $z = (Q_0, \dot{T}^g)$ depende só da malha (Gb, Mg), **não** de y . Fatora-se **uma vez**; a cada RHS só monta o residual $r(y)$ e faz $z = F \setminus r$.

```

"""Fator LU de A (constante no tempo). Linhas de colocação: x0 + Gauss."""
function bem_heat_factor(mesh)
    (; Gb, Mg, ni, N) = mesh
    rows = vcat(1, 3:N)          # N-1 equações
    nz = 1 + ni
    A = zeros(nz, nz)
    for (eq, p) in enumerate(rows)
        A[eq, 1] = -Gb[p, 1]      # coluna Q0
        A[eq, 2:end] .= Mg[p, :]  # colunas Tg
    end
    return lu(A), rows
end

"""RHS: só r(y); resolve com fator pré-computado F."""
function bem_heat_rhs!(dy, y, p, t)
    mesh, TL, F, rows = p
    (; Hb, nc, ni, xg) = mesh
    Tf = y[1]
    Tg = y[2:end]
    @assert length(Tg) == ni

    r = zeros(1 + ni)
    for (eq, pidx) in enumerate(rows)
        r[eq] = -(Hb[pidx, 1] * TL + Hb[pidx, 2] * Tf)
        if pidx > nc
            r[eq] -= Tg[pidx - nc]
        end
    end
end

```



```

# --- T(L,t): BEM x analítico ---
ts = range(tspan...; length=80)
TR_bem = [solB(t)[1] for t in ts]
TR_ex = [T_exact(L, t) for t in ts]

p1 = plot(ts, TR_ex; lw=2, label="analítico", xlabel="t", ylabel="T(L,t)",
          title="Ponta direita")
plot!(p1, ts, TR_bem; lw=2, ls=:dash, label="BEM + linhas (nq=$(mesh.ni))")

# --- perfil em t fixo ---
t_snap = 0.25
xp, Tp = profile_at(solB, mesh, TL, t_snap)
xx = range(0, L; length=200)
p2 = plot(xx, T_exact.(xx, t_snap); lw=2, label="analítico",
          xlabel="x", ylabel="T", title="Perfil t = $t_snap")
plot!(p2, xp, Tp; marker=:circle, lw=2, ls=:dash, label="BEM (nós)")

plot(p1, p2; layout=(1, 2), size=(900, 350))

# --- erros ---
err_TR = maximum(abs, TR_bem .- TR_ex)
err_prof = maximum(abs, Tp .- T_exact.(xp, t_snap))
@show err_TR err_prof

# --- estudo com nq ---
println("nq\tterr_T(L,t)\terr_perfil(t=$t_snap)")
for nq in (2, 4, 6, 8, 12)
    soln, mn = solve_bem_heat(; L=L, κ=κ, TL=TL, tspan=tspan, nq=nq,
                              T0 = x -> sin(λ0 * x))
    eL = maximum(abs, [soln(t)[1] - T_exact(L, t) for t in ts])
    xq, Tq = profile_at(soln, mn, TL, t_snap)
    ep = maximum(abs, Tq .- T_exact.(xq, t_snap))
    println("$nq\t$t$eL\t$t$ep")
end

```

Leitura esperada: com o modo fundamental nas BC certas, o erro em $T(L, t)$ e no perfil cai ao aumentar n_q (mais fontes interiores = melhor M^g), até saturar pelo modelo 1D/SF estacionária e pela amarração de $\dot{T}_f$. Compare sempre com a curva $e^{-(\pi/2)^2 t}$ na ponta ($T(L, 0) = 1$).

Por que essa CI? $\sin(\pi x/(2L))$ some em $x = 0$ e tem derivada nula em $x = L$, logo respeita as BC para todo t na solução exata. Isso isola o erro da **semi-discretização BEM**, não de BC incompatíveis.

Checklist.

1. $N = n_c + n_i$ colocações: 2 extremos + todos os Gauss.
2. $M^g: N \times n_i$, $M_{pj}^g = w_j \kappa^{-1} T^*(x_j^g, x_d^p)$ — coluna = fonte de domínio no Gauss j .
3. $H^b, G^b: N \times 2$.
4. Fator LU de A ($N - 1$) **uma vez**; cada RHS só monta $r(y)$ e faz $F \setminus r$ para $(Q_0, \dot{T}^g)$.
5. 2D: o mesmo papel dos Gauss é o dos pontos interiores DIBEM.

4.10.7 Checklist MDF × BEM (linhas)

1. Espaço $\rightarrow$ PVI/DAE no tempo.
2. BEM: estado = contorno livre + T em cada Gauss (fonte interior).
3. Matriz espacial densa de ordem $\sim n_c + n_i$, **fatorada uma vez**; residual a cada passo.
4. Integrador no tempo intercambiável (Tsit5, RK4, Houbolt).
5. $n_i \uparrow$ enriquece domínio e tamanho do sistema.

4.11 Exercícios

Entregue para cada item: código Julia (Plots), figuras pedidas e **duas ou três frases** de interpretação. Use os solucionadores e rotinas do capítulo (euler, rk4, diffmat, bem1d_M, solve_bem_heat, ...).

1. **E1 — Ordem de Euler e RK4.** Considere o PVI

$$u' = -2tu, \quad u(0) = 2, \quad t \in [0, 2],$$

com solução exata $\hat{u}(t) = 2e^{-t^2}$.

- (a) Com euler e rk4, tome $n = 10 \cdot 2^d$ para $d = 1, \dots, 8$ e calcule o erro no tempo final

$$e(n) = |u_n - \hat{u}(t_n)|.$$

- (b) Em um único gráfico log-log ($n \times e(n)$), plote as duas curvas e sobreponha retas de referência $C_1 n^{-1}$ e $C_4 n^{-4}$ (ajuste C_1, C_4 no primeiro n de cada método).

- (c) Com base nas inclinações, confirme as ordens e diga a partir de qual n o RK4 deixa de ganhar (saturação por aritmética ou pela referência).

2. **E2 — De 2ª ordem a sistema.** O problema

$$u'' + 9u = 9t, \quad u(0) = 1, \quad u'(0) = 1, \quad t \in [0, 2\pi]$$

tem solução $\hat{u}(t) = t + \cos(3t)$.

- (a) Escreva o sistema de 1ª ordem em $y = (u, u')$ e a função $f(y, p, t)$ correspondente.

- (b) Integre com rk4 ($n = 200$) e com Tsit5 (DifferentialEquations). Reporte

$$e = |u(2\pi) - \hat{u}(2\pi)|$$

nos dois casos.

- (c) Plote $u(t)$ numérico (RK4) e $\hat{u}(t)$ no mesmo eixo. Onde o erro se concentra ao longo de t ?

3. **E3 — PVC 1D com diffmat.** Resolva

$$-T'' = 1, \quad x \in (0, 1), \quad T(0) = T(1) = 0,$$

cujas soluções exatas são $T(x) = x(1-x)/2$.

- (a) Use diffmat e imponha Dirichlet por substituição de linhas. Para $n \in \{10, 20, 40, 80\}$ (intervalos), calcule

$$e_\infty(h) = \max_i |T(x_i) - T_h(x_i)|, \quad h = 1/n.$$

- (b) Plote e_∞ vs h em escala log-log e estime a ordem p em $e = O(h^p)$.

- (c) Para $n = 40$, plote T exata e T_h juntas. O erro máximo ocorre no centro ou perto das bordas? Por quê, dada a ordem das fórmulas em D_{xx} ?

4. **E4 – MDF × BEM × solução analítica (calor 1D).** Considere o mesmo problema do exemplo do texto:

$$\begin{aligned}T_t &= T_{xx}, \quad 0 < x < 1, \\T(0, t) &= 0, \quad \partial_x T(1, t) = 0, \\T(x, 0) &= \sin(\pi x/2),\end{aligned}$$

com solução exata

$$T(x, t) = \sin(\pi x/2), e^{-(\pi/2)^2 t}.$$

MDF (método das linhas). Use `diffmat` em $[0, 1]$ com n intervalos, imponha $T(0, t) = 0$ e a Neumann $\partial_x T(1, t) = 0$ (substitua a última linha de D_{xx} por a linha correspondente de D_x , ou uma condição equivalente de ordem 2). Integre no tempo com `Tsit5` até $t = 0.5$.

BEM (método das linhas no contorno). Use `solve_bem_heat` / `bem1d_M` com n_q Gauss como fontes interiores (fator LU pré-computado).

- (a) **Erros em relação ao analítico.** Para MDF com $n \in \{40, 80, 160\}$ e BEM com $n_q \in \{2, 4, 6, 8, 12\}$, reporte

$$e_L = \max_{t \in [0, 0.5]} |T(1, t) - T_{\text{exata}}(1, t)|$$

e o erro máximo do perfil em $t = 0.25$. Organize em duas tabelas (MDF e BEM).

- (b) **Figuras.** Em $t = 0.25$, plote no **mesmo** gráfico: solução exata, perfil MDF (melhor n) e perfil BEM (melhor n_q). Em outro gráfico: $T(1, t)$ exata, MDF e BEM ao longo do tempo.

- (c) **Comparação MDF × BEM.** Com os números e as figuras:

- qual método atinge erro $< 10^{-3}$ em e_L com menos graus de liberdade no **estado** do PVI? (MDF: $\sim n - 1$ interiores; BEM: $1 + n_q$)
- o erro do BEM satura ao subir n_q ? O do MDF satura ao subir n ?
- comente o custo por passo: fator esparsa/aplicação de D_{xx} vs. `backsolve` denso $(N - 1) \times (N - 1)$ já fatorado.

- (d) **Pergunta teórica:** se $M^g \equiv 0$ no BEM, o que resta das equações de colocação? Por que some a dinâmica com a SF estacionária de Laplace?

5. **E5 – Propagação de onda (MDF + comparação; esboço BEM).**

Teoria mínima (releia a § “Equação da onda”). A EDP $u_{tt} = c^2 u_{xx}$ é de **segunda ordem no tempo**. Para usar `euler` / `rk4` / `Tsit5` (feitos para $y' = f(t, y)$), reduza a ordem:

$$v = u_t, \quad u_t = v, \quad v_t = c^2 u_{xx}.$$

No espaço, troque u_{xx} por $D_{xx} \mathbf{u}$ (`diffmat`). O estado do PVI é

$$\mathbf{y} = (\mathbf{u}, \mathbf{v}).$$

Em extremos com Dirichlet **fixo** $u = 0$, imponha também $v = u_t = 0$ nesses nós.

Problema.

$$\begin{aligned}u_{tt} &= c^2 u_{xx}, \quad 0 < x < 1, \quad 0 < t \leq 2, \\u(0, t) &= u(1, t) = 0, \\u(x, 0) &= \sin(\pi x), \quad u_t(x, 0) = 0,\end{aligned}$$

com $c = 1$ e solução exata

$$u_{\text{exata}}(x, t) = \sin(\pi x), \cos(\pi t).$$

(a) **Implementação MDF.** Usando o esqueleto `wave_rhs!` / `solve_wave_mdf` (ou equivalente seu) com `diffmat` e `Tsit5`, resolva para $n \in \{50, 100, 200\}$ intervalos. Em cada n , calcule

$$e_{\infty}(t) = \max_i |u_h(x_i, t) - u_{\text{exata}}(x_i, t)|$$

nos instantes $t \in \{0.5, 1.0, 1.5, 2.0\}$. Monte uma tabela (n, t, e_{∞}) .

(b) **Figuras.**

- Perfis $u(x, t)$ em $t = 0, 0.5, 1.0, 1.5$ (MDF com o melhor n) sobrepostos à exata (linhas tracejadas).
- Série temporal no ponto médio: $u(1/2, t)$ numérico vs $\cos(\pi t)$ (exata, pois $\sin(\pi/2) = 1$).

(c) **Dispersão e refinamento.** O erro cresce com t mesmo com BC e CI “perfeitas” no modo fundamental? Se sim, atribua a **dispersão numérica** de D_{xx} (a velocidade numérica do modo depende de h). Mostre que e_{∞} em $t = 2$ cai ao aumentar n e estime a ordem aparente em h .

(d) **Comparação qualitativa com o calor (E4).** Em uma frase cada: (i) o que acontece com a “energia” / amplitude do modo no calor vs na onda; (ii) restrição de passo no tempo explícito ($\Delta t = O(h^2)$ vs $O(h)$).

(e) **BEM — só estrutura (sem obrigar código).** Escreva a forma matricial esperada no contorno

$$M\mathbf{u}'' + H\mathbf{u} = G\mathbf{q}$$

e diga: o que são as incógnitas em cada extremo se $u(0, t) = u(1, t) = 0$? O método das linhas no contorno atuaria sobre quais componentes de $\mathbf{z}$? Em que o bloco M aqui difere do M^g do calor (segunda derivada temporal vs primeira)?

(f) **Opcional (Houbolt).** Com o extra do capítulo, integre o oscilador modal equivalente $U'' + (c\pi)^2 U = 0$ (amplitude do modo $\sin(\pi x)$) com Houbolt e compare $U(t)$ a $\cos(c\pi t)$. Relacione com o item (b) no ponto médio.

4.12 Extra: multi-passo AB4 e Houbolt

RK avalia f várias vezes por passo **sem** histórico. Métodos de **múltiplos passos** reutilizam $f_i = f(t_i, u_i)$ passados. Ficam no fim porque a trilha BEM já está completa com Euler/RK/Tsit5; use-os quando quiser menos avaliações de f ou integradores da dinâmica estrutural.

4.12.1 Adams–Bashforth 4 (explícito)

$$u_{i+1} = u_i + h \left(\frac{55}{24} f_i - \frac{59}{24} f_{i-1} + \frac{37}{24} f_{i-2} - \frac{9}{24} f_{i-3} \right).$$

Precisa de 3 valores iniciais (starter: RK4 num trecho curto).

```
function ab4(ivp, n)
    a, b = ivp.tspan
    h = (b - a) / n
    t = [a + i*h for i in 0:n]
    u0 = ivp.u0 isa Number ? float(ivp.u0) : float.(ivp.u0)
    U = Vector{typeof(u0)}(undef, n + 1)
    _, Us = rk4(ODEProblem(ivp.f, ivp.u0, (a, a + 3h), ivp.p), 3)
    U[1:4] .= Us
    σ = [-9, 37, -59, 55] ./ 24
```

```

F = [ivp.f(U[i], ivp.p, t[i]) for i in 1:4]
for i in 4:n
    U[i+1] = U[i] + h * sum(σ[j] * F[j] for j in 1:4)
    F = (F[2], F[3], F[4], ivp.f(U[i+1], ivp.p, t[i+1]))
end
return t, U
end

```

4.12.2 Houbolt (2ª ordem no tempo)

Comum em dinâmica estrutural e em BEM elástico transiente. Aproxima

$$u''_{n+1} = \frac{2u_{n+1} - 5u_n + 4u_{n-1} - u_{n-2}}{h^2},$$

$$u'_{n+1} = \frac{11u_{n+1} - 18u_n + 9u_{n-1} - 2u_{n-2}}{6h}.$$

Implícito: u_{n+1} entra nos dois lados quando a EDO é $u'' = f(t, u, u')$. Amortece altas frequências (rígido).

Esqueleto para o oscilador $u'' + \omega^2 u = 0$ (estado escalar; starter com RK4 na forma 1ª ordem):

```

"""Houbolt para u'' + ω² u = 0, n passos em [0, tf]."""
function houbolt_oscillator(ω, u0, v0, tf, n)
    h = tf / n
    t = collect(range(0, tf; length=n+1))
    u = zeros(n + 1)
    # starter: RK4 no sistema (u,v)
    fsys = (y, p, τ) -> [y[2], -ω² * y[1]]
    _, Ys = rk4(ODEProblem(fsys, [u0, v0], (0.0, 2h), nothing), 2)
    u[1] = u0
    u[2] = Ys[2][1]
    u[3] = Ys[3][1]
    for i in 3:n
        # (2/h²) u_{i+1} + ω² u_{i+1} = (5 u_i - 4 u_{i-1} + u_{i-2}) / h²
        rhs = (5u[i] - 4u[i-1] + u[i-2]) / h²
        u[i+1] = rhs / (2/h² + ω²)
    end
    return t, u
end

ω = 2π
tH, uH = houbolt_oscillator(ω, 1.0, 0.0, 10.0, 400)
tR, UR = rk4(ODEProblem((y,p,τ)->[y[2], -ω²*y[1]], [1.0, 0.0], (0.0, 10.0),
nothing), 400)
plot(tH, uH; label="Houbolt", lw=2)
plot!(tR, [y[1] for y in UR]; label="RK4", ls=:dash, lw=2,
xlabel="t", ylabel="u", title="u'' + ω²u = 0")

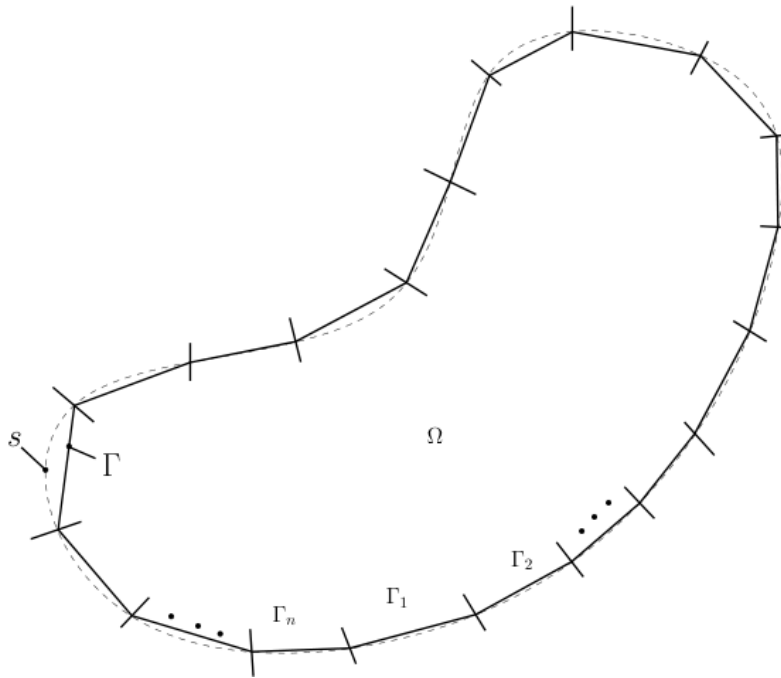
```

Adams–Moulton / preditor–corretor: leitura exterior (não são necessários para a trilha BEM deste capítulo).

5 Indo para 2D

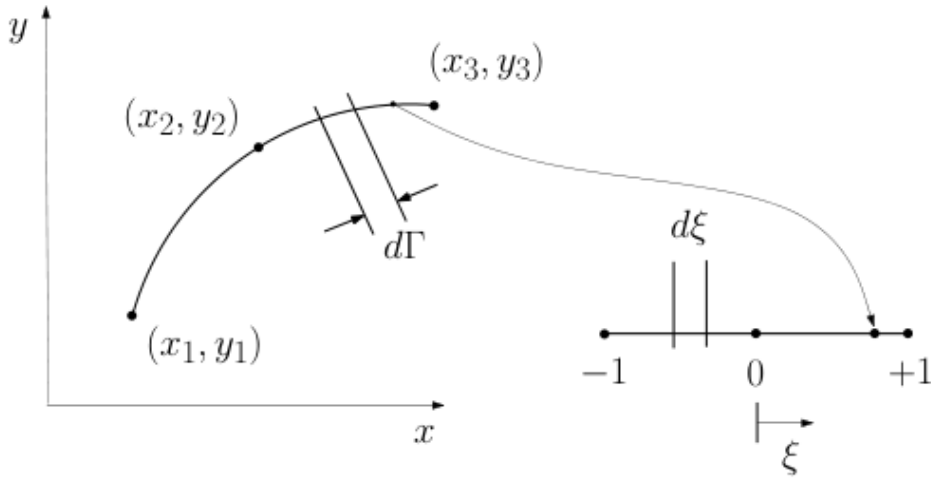
5.1 Cálculo do perímetro de figuras planas

Uma vez que a integral ao longo do contorno no s pode ser bastante difícil, ou mesmo impossível, uma estratégia para o cálculo do perímetro é a divisão do contorno em uma soma de pequenos pedaços $s_1, s_2, \dots, s_n$, ou seja, $s = \sum_{i=1}^n s_i$, onde n é o número de pedaços em que o contorno foi dividido. Uma vez que estes pedaços podem ter uma forma qualquer, cada pedaço s_i será aproximado por uma forma conhecida. Por simplicidade, esta forma é quase sempre dada por um polinômio (linha reta, parábola, etc).



Dessa maneira, cada pedaço $s_1, s_2, \dots, s_n$ é aproximado por formas conhecidas $\Gamma_1, \Gamma_2, \dots, \Gamma_n$, chamados Elementos de Contorno.

Considere agora que os elementos de contorno Γ_i sejam parabólicos, ou seja, são descritos por polinômios de 2ª ordem (equação de uma parábola). Desta forma, são necessários 3 pontos de Γ_i para se definir a parábola. Estes pontos são dados por (x_1, y_1) , (x_2, y_2) e (x_3, y_3) , que correspondem respectivamente a $\xi = -1$, $\xi = 0$ e $\xi = 1$.



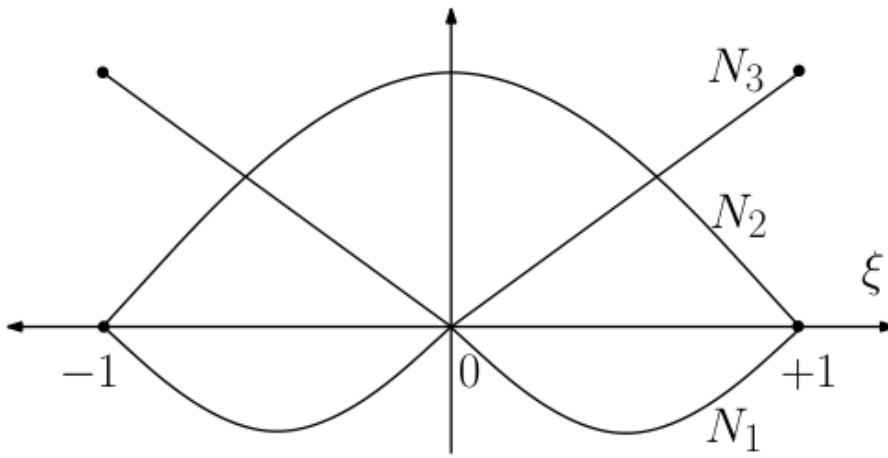
Criando uma função parabólica para relacionar x com ξ , tem-se: $x = a\xi^2 + b\xi + c$ sendo que:

$$x(\xi = -1) = x_1 = a(-1)^2 + b(-1) + c$$

$$x(\xi = 0) = x_2 = a(0)^2 + b(0) + c$$

$$x(\xi = +1) = x_3 = a(+1)^2 + b(+1) + c$$

resolvendo esse sistema obtém-se $x = N_1x_1 + N_2x_2 + N_3x_3$ onde $N_1 = \frac{\xi}{2}(\xi - 1)$, $N_2 = (1 - \xi)(1 + \xi)$ e $N_3 = \frac{\xi}{2}(\xi + 1)$ são as funções de forma quadráticas contínuas.



A derivada de x em relação a ξ é dada por $\frac{dx}{d\xi} = \frac{d[N_1(\xi)]}{d\xi}x_1 + \frac{d[N_2(\xi)]}{d\xi}x_2 + \frac{d[N_3(\xi)]}{d\xi}x_3$ onde $\frac{dN_1}{d\xi} = \xi - \frac{1}{2}$, $\frac{dN_2}{d\xi} = -2\xi$ e $\frac{dN_3}{d\xi} = \xi + \frac{1}{2}$.

O comprimento Γ do perímetro da figura é então dado por:

$$\Gamma = \sum_{i=1}^n \int_{-1}^1 \frac{d\Gamma}{d\xi} d\xi = \sum_{i=1}^n \int_{-1}^1 \sqrt{\left(\frac{dx}{d\xi}\right)^2 + \left(\frac{dy}{d\xi}\right)^2} d\xi = \sum_{i=1}^n \left[\int_{-1}^1 J(\xi) d\xi \right] \text{ em que a função } J(\xi) = \sqrt{\left(\frac{dx}{d\xi}\right)^2 + \left(\frac{dy}{d\xi}\right)^2} \text{ é chamada jacobiano da transformação. Usando quadratura de Gauss com } p \text{ pontos de Gauss, conclui-se que } \Gamma = \sum_{i=1}^n \left[\sum_{k=1}^p w_k J(\xi_k) \right].$$

No BEM_gmsh as propriedades geométricas saem diretamente de um BEMdata (integrais de contorno com as mesmas funções de forma e jacobianos do BEM):

```

using DrWatson
@quickactivate :BEM
include(datadir("Laplace", "Laplace_dad.jl"))

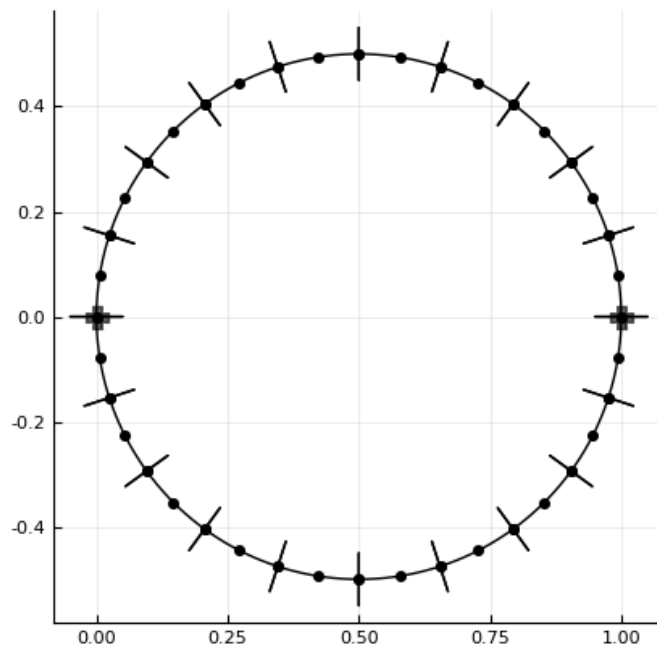
# Malha Gmsh do contorno (e domínio, se houver Surface física)
msh = quadrado(ndiv=20, show=false)
dad = format2d(msh, Laplace(1.0); pontointerno=false)

plot_geo(dad) # visualiza nós, elementos e normais

gp = geometric_props(dad) # ou geometric_props_2d(dad)
println("Perímetro: ", gp.perimeter)
println("Área:      ", gp.area)
println("Centróide: ", gp.centroid)

```

Para uma poligonal simples (sem Gmsh), use `geometric_props_2d_polygon` com a lista de vértices em sentido anti-horário. Material antigo em planilhas/OneDrive permanece como referência histórica: [código legado de propriedade geométrica](#).



5.2 Área

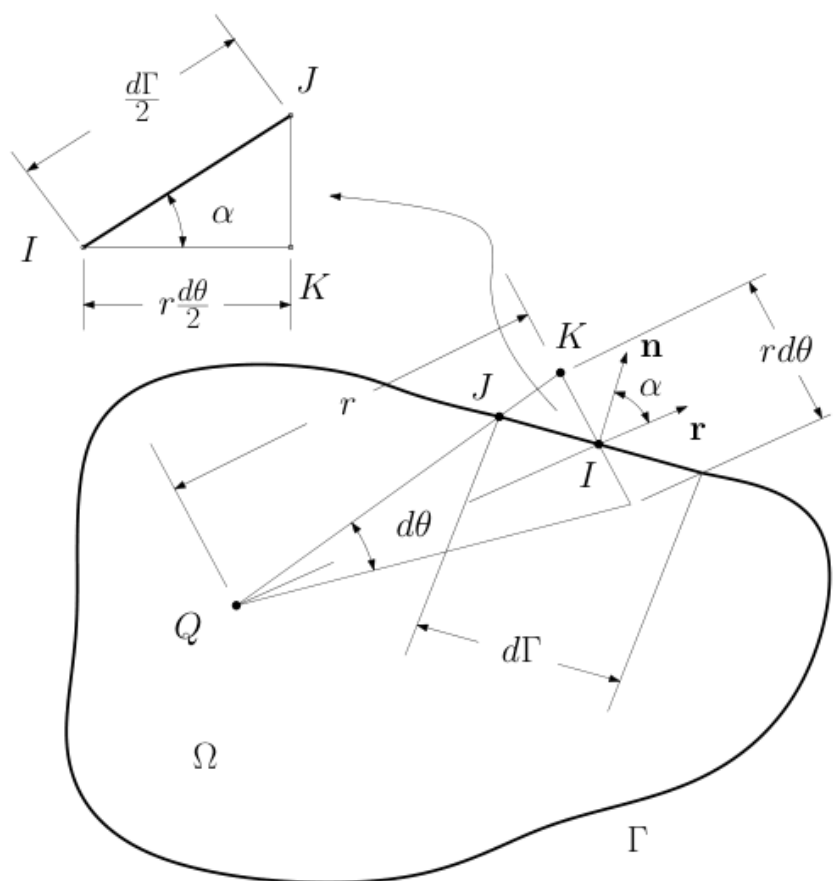
A integral de uma função $f(x, y)$ sobre a área Ω de uma figura plana é dada pela integral $I = \int_{\Omega} f(x, y) dx dy$. A integral pode ser escrita, em coordenadas polares, como: $I = \int_{\Omega} f(x(\rho, \theta), y(\rho, \theta)) \rho d\rho d\theta = \int_{\theta} \int_0^r f(x(\rho, \theta), y(\rho, \theta)) \rho d\rho d\theta$ definindo F como $F(\rho, \theta) = \int_0^r f(x(\rho, \theta), y(\rho, \theta)) \rho d\rho$, pode-se escrever: $I = \int_{\theta} F(\rho, \theta) d\theta$.

como $\cos \alpha = \frac{r \frac{d\theta}{d\Gamma}}{2}$

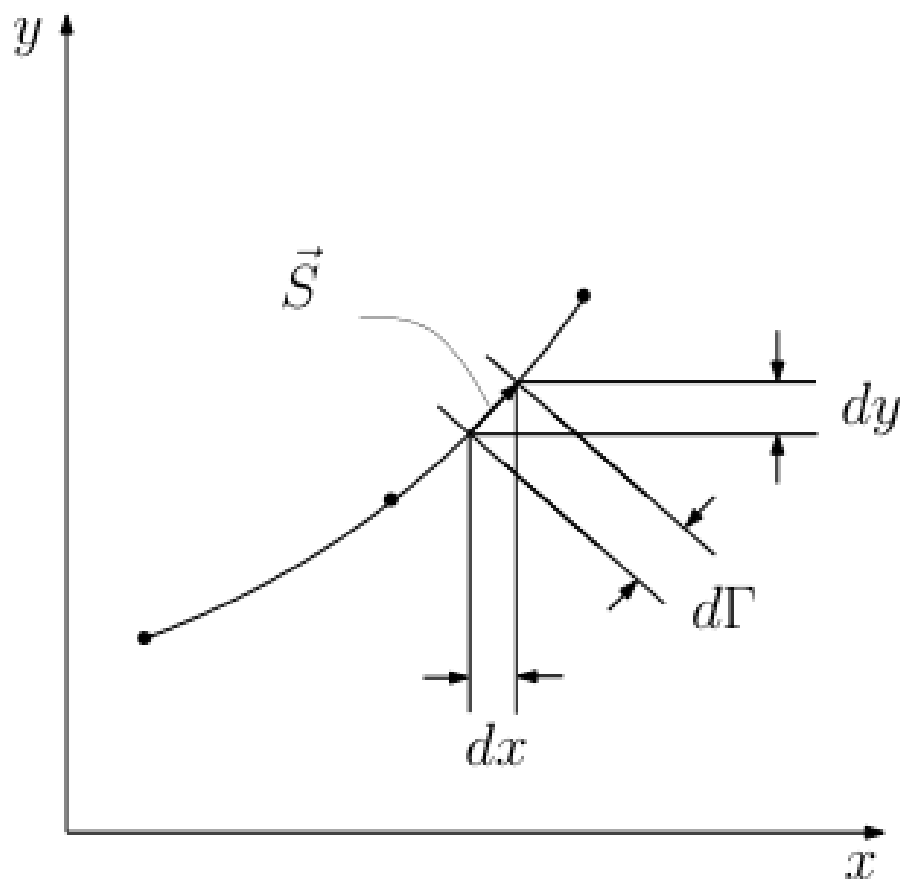
podemos escrever: $d\theta = \frac{\vec{n} \cdot \vec{r}}{r} d\Gamma$ e finalmente $I = \int_{\Gamma} F \frac{\vec{n} \cdot \vec{r}}{r} d\Gamma$.

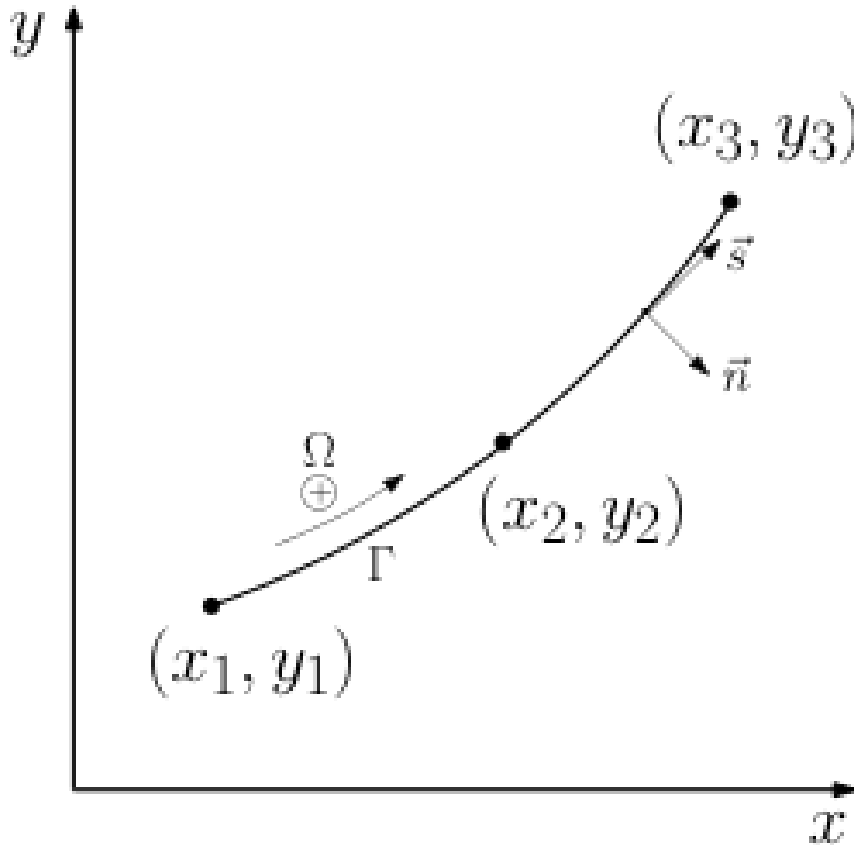
Dividindo Γ em uma soma de pequenos pedaços do contorno:

$$I = \sum_{i=1}^{NE} \int_{\Gamma_i} F \frac{\vec{n} \cdot \vec{r}}{r} d\Gamma.$$



5.2.1 Cálculo do vetor normal $\vec{n}$





$\vec{S}$ = vetor tangente ao contorno Γ . $\vec{S} = dx\vec{i} + dy\vec{j}$ vetor não unitário

$\vec{s}$ = Vetor unitário tangente ao contorno Γ

$$\vec{s} = \frac{\vec{S}}{|\vec{S}|} = \frac{dx\vec{i} + dy\vec{j}}{\sqrt{dx^2 + dy^2}}$$

Dividindo tudo por $d\xi$, tem-se:

$$\vec{s} = \frac{\frac{dx}{d\xi}}{\sqrt{\left(\frac{dx}{d\xi}\right)^2 + \left(\frac{dy}{d\xi}\right)^2}}\vec{i} + \frac{\frac{dy}{d\xi}}{\sqrt{\left(\frac{dx}{d\xi}\right)^2 + \left(\frac{dy}{d\xi}\right)^2}}\vec{j}$$

$$\vec{s} = \frac{\frac{dx}{d\xi}}{\frac{d\Gamma}{d\xi}}\vec{i} + \frac{\frac{dy}{d\xi}}{\frac{d\Gamma}{d\xi}}\vec{j} = s_x\vec{i} + s_y\vec{j}$$

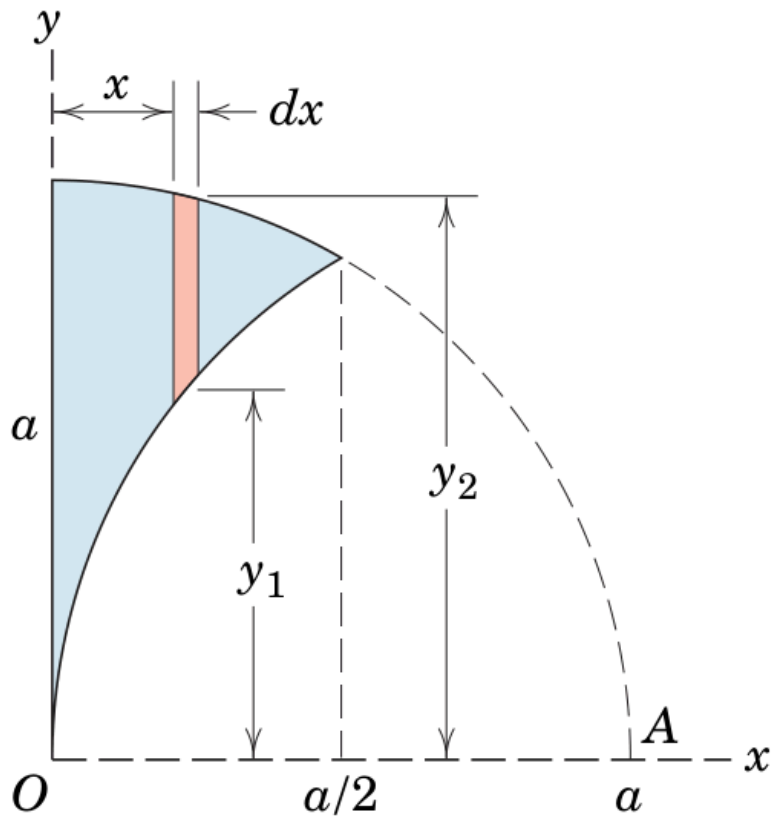
$\vec{n}$ = vetor unitário normal ao contorno Γ apontando para fora do domínio Ω

$\vec{n} = n_x\vec{i} + n_y\vec{j}$ Temos duas possibilidades, uma vez que $\vec{s}$ é unitário:

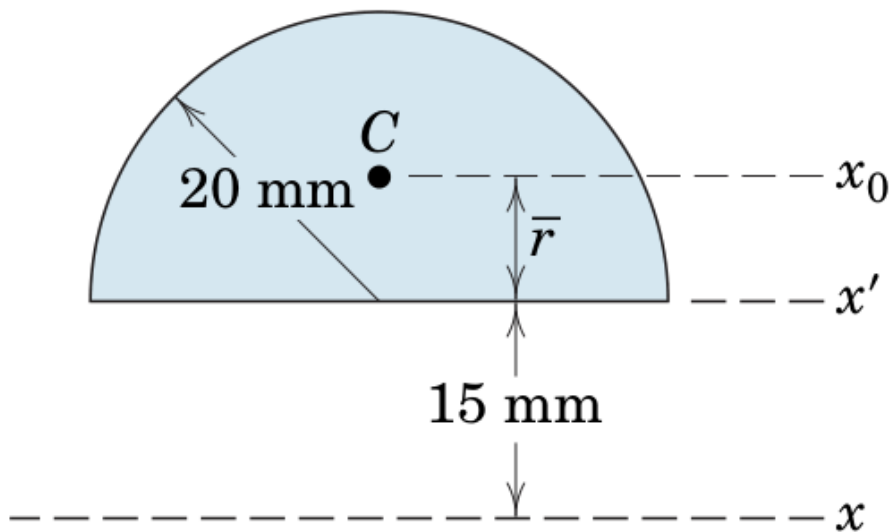
$$n_x = -s_y \quad n_y = s_x \Rightarrow (-s_y)s_x + (s_x)s_y = 0$$

$n_x = s_y \quad n_y = -s_x \Rightarrow (s_y)s_x + (-s_x)s_y = 0$ O vetor $\vec{s}$ é sempre definido percorrendo Γ no sentido anti-horário, se Γ for um contorno externo e horário, caso Γ seja um contorno interno. Portanto, a segunda possibilidade é escolhida de modo que a normal esteja apontada para fora do domínio.

5.2.2 Exercícios



$$I_x = \frac{a^4}{96} (9\sqrt{3} - 2\pi)$$



$$I_x = 2 \cdot 10^4 \pi - \frac{20^2 \cdot \pi}{2} \left(\frac{80}{3 \cdot \pi} \right)^2 + \left(\frac{20^2 \pi}{2} \right) \left(15 + \frac{80}{3 \pi} \right)^2$$

Calcule, estendendo `geometric_props` (ou um script próprio no espírito do `propgeo`), os momentos de inércia em relação ao eixo x para as duas figuras e compare com o resultado analítico. Sugestão: reutilize a malha `Gmsh` + `format2d` e integre y^2 via a mesma redução a contorno apresentada acima.

- Gravações <https://youtu.be/TZvHS2JzKOo> <https://youtu.be/08vADm8JguE> <https://youtu.be/O14y7y9ZCLs> <https://youtu.be/YbKmFAwBbwo>

6 Gmsh e malhas para o BEM

O [Gmsh](#) é um gerador de malhas de código aberto amplamente usado em elementos finitos e, cada vez mais, em elementos de contorno. No curso usamos o repositório [BEM_gmsh](#), que acopla o Gmsh ao fluxo de trabalho em Julia: geometria → grupos físicos (condições de contorno) → malha .msh → format2d / format3d → montagem e solução.

6.1 Por que Gmsh no BEM?

No BEM clássico 2D, a malha é **apenas o contorno** (curvas). Ainda assim, o Gmsh é útil porque:

1. descreve geometrias com furos, arcos, splines e múltiplas regiões de forma robusta;
2. associa **nomes de grupos físicos** às condições de contorno (Dirichlet / Neumann);
3. gera malhas de ordem 1 ou 2 (elementos lineares ou quadráticos);
4. permite refinamento local (campos de tamanho perto de cantos e furos);
5. integra-se ao ONELAB (interface gráfica de parâmetros + pós-processamento).

Para o BEM, o domínio volumétrico da malha 2D do Gmsh serve sobretudo para **pontos internos** e para métodos de domínio (DIBEM). Os elementos de contorno são extraídos das **curvas físicas** (dimensão 1).

6.2 Instalação rápida

No ambiente do projeto BEM_gmsh:

```
using Pkg
Pkg.activate(".") # pasta do BEM_gmsh
Pkg.instantiate()

using DrWatson
@quickactivate :BEM
```

O pacote Julia Gmsh.jl já vem como dependência e embute (ou localiza) o executável do Gmsh. Para a interface gráfica completa, instale também o [Gmsh do sistema](#).

6.3 Anatomia de um arquivo .geo

Um script Gmsh típico para o BEM 2D tem quatro blocos:

1. **Pontos e curvas** — geometria;
2. **Superfície** — domínio (para pontos internos / DIBEM);
3. **Grupos físicos** — nomes que codificam as CDCs;
4. **Geração da malha** — Mesh 2, opcionalmente RecombineMesh e ordem.

Exemplo mínimo (quadrado unitário, campo $T = x$):

```
// quadrado_bem.geo
lc = 0.1;

Point(1) = {0, 0, 0, lc};
Point(2) = {1, 0, 0, lc};
Point(3) = {1, 1, 0, lc};
Point(4) = {0, 1, 0, lc};

Line(1) = {1, 2}; // base
Line(2) = {2, 3}; // direita
```



```

Line(3) = {3, 4}; // topo
Line(4) = {4, 1}; // esquerda

Curve Loop(1) = {1, 2, 3, 4};
Plane Surface(1) = {1};

Transfinite Curve {1, 2, 3, 4} = 12;
Transfinite Surface{1};
Recombine Surface{1};

// Convenção BEM_gmsh (Laplace): "tipo;valor"
// tipo 0 = Dirichlet (T), tipo 1 = Neumann (q = -k ∂T/∂n)
Physical Curve("1;0") = {1, 3}; // isolado
Physical Curve("1;-1") = {2};    // q = -1 (∂T/∂n = +1 se k=1)
Physical Curve("0;0") = {4};     // T = 0
Physical Surface("Domain") = {1};

Mesh 2;

```

Gere a malha no terminal:

```
gmsh -2 quadrado_bem.geo -o quadrado_bem.msh
```

ou, de dentro do Julia / BEM_gmsh, use os geradores prontos (recomendado no curso):

```

using DrWatson
@quickactivate :BEM
include(datadir("Laplace", "Laplace_dad.jl"))

msh = quadrado(ndiv=20, show=false) # grava .msh e devolve o caminho

```

6.4 Convenção de condições de contorno

Os **nomes** dos grupos físicos carregam o tipo e o valor da CDC. Isso evita arquivos auxiliares de contorno.

6.4.1 Laplace / Poisson

Nome do grupo	Significado
"0;T"	Dirichlet: temperatura (ou potencial) prescrita T
"1;q"	Neumann: fluxo $q = -k\partial T/\partial n$

Exemplos: "0;100", "1;0" (isolado), "1;-1".

6.4.2 Elasticidade 2D

Cada aresta usa **quatro** tokens tx;ux;ty;uy:

- token ímpar (tx, ty): 0 = deslocamento prescrita, 1 = tração prescrita;
- token par (ux, uy): valor correspondente.

Nome	Significado
------	-------------

"0;0;0;0"	engaste ($u_x = u_y = 0$)
"1;0;1;0"	livre de tração
"1;P;1;0"	tração $t_x = P, t_y = 0$
"0;0;1;0"	rolo vertical ($u_x = 0, t_y = 0$)

6.5 Do arquivo .msh ao BEMdata

O fluxo canônico no BEM_gmsh é:

```
using DrWatson
@quickactivate :BEM
include(datadir("Laplace", "Laplace_dad.jl"))

props = Laplace(1.0) # k = 1
msh = quadrado(ndiv=20, show=false)
dad = format2d(msh, props) # lê grupos físicos → CDCs

attach_analytical!(dad, ana_laplace_linear(; direction=SA[1.0, 0.0]))
H_G_full_direct(dad, 20) # monta H e G (densa)
solve(dad)

@show rel_error(dad)
plot_geo(dad)
```

Palavras-chave úteis de format2d:

- tipo=1|2|3 — ordem do elemento de contorno (linear, quadrático, cúbico), quando compatível com a malha;
- pontointerno=true|false — usa nós internos da malha de superfície para pós-processamento / DIBEM;
- finalize=false, reopen=false — quando a sessão Gmsh já está aberta (pipelines ONELAB).

Para problemas grandes:

```
H_G_Hmat(dad; atol=1e-6, nmax=32) # montagem hierárquica
solve(dad)
@show compression_ratio(dad.H)
```

6.6 Gerando geometria em Julia (API Gmsh)

Além de arquivos .geo, o BEM_gmsh constrói malhas pela API Julia. Esqueleto de um anel (quarto de coroa):

```
using DrWatson
@quickactivate :BEM

function meu_quarto_circulo(; ndiv=12, ri=1.0, re=2.0, nome="meu_qc")
    gmsh.initialize()
    gmsh.option.setNumber("General.Terminal", 0)
    gmsh.model.add(nome)
    lc = (re - ri) / max(ndiv, 4)
```

```

c = gmsh.model.geo.addPoint(0.0, 0.0, 0.0, lc)
p1 = gmsh.model.geo.addPoint(ri, 0.0, 0.0, lc)
p2 = gmsh.model.geo.addPoint(re, 0.0, 0.0, lc)
p3 = gmsh.model.geo.addPoint(0.0, re, 0.0, lc)
p4 = gmsh.model.geo.addPoint(0.0, ri, 0.0, lc)

lbot = gmsh.model.geo.addLine(p1, p2)
aout = gmsh.model.geo.addCircleArc(p2, c, p3)
lleft = gmsh.model.geo.addLine(p3, p4)
ain = gmsh.model.geo.addCircleArc(p4, c, p1)

cl = gmsh.model.geo.addCurveLoop([lbot, aout, lleft, ain])
s = gmsh.model.geo.addPlaneSurface([cl])
gmsh.model.geo.synchronize()

for crv in (lbot, aout, lleft, ain)
    gmsh.model.mesh.setTransfiniteCurve(crv, ndiv)
end
gmsh.model.mesh.setTransfiniteSurface(s)

gmsh.model.addPhysicalGroup(1, [lbot, lleft], -1, "1;0") # isolado
gmsh.model.addPhysicalGroup(1, [ain], -1, "0;100") # T = 100
gmsh.model.addPhysicalGroup(1, [aout], -1, "1;-200") # q = -200
gmsh.model.addPhysicalGroup(2, [s], -1, "Domain")

gmsh.model.mesh.generate(2)
out = datadir("Laplace", nome * ".msh")
mkpath(dirname(out))
gmsh.write(out)
gmsh.finalize()
return out
end

```

Geradores prontos no repositório (vale estudar o código-fonte):

- data/Laplace/Laplace_dad.jl — quadrado, placa_com_furo, quadrado_elasticity;
- data/Laplace/potencial_problems.jl — Laquini, quarto de círculo, Moulton;
- data/elastico/iso/*.jl — cilindro pressurizado, placa com furo, viga em balanço, trinca central.

6.7 Propriedades geométricas via contorno

O módulo `geometric_props` do `BEM_gmsh` calcula perímetro, área e centróide **só com integrais de contorno** — o mesmo espírito do capítulo “Indo para 2D”:

```

using DrWatson
@quickactivate :BEM
include(datadir("Laplace", "Laplace_dad.jl"))

msh = quadrado(ndiv=30, show=false)
dad = format2d(msh, Laplace(1.0); pontointerno=false)

gp = geometric_props(dad)
@show gp.perimeter gp.area gp.centroid

```

Em 3D (format3d + malha de superfície fechada):

```
# exemplo conceitual – requer um .geo/.msh 3D (ex.: cubo)
# dad3 = format3d(datadir("Laplace", "cubo.msh"), Laplace(1.0))
# gp3 = geometric_props(dad3) # área superficial, volume, centróide
```

6.8 ONELAB: Gmsh como interface do solver

O BEM_gmsh publica um cliente ONELAB. Parâmetros (n_{div} , k , tipo de montagem, ...) ficam no painel do Gmsh; ao clicar em **Run**, a malha é gerada, o BEM resolve e as views de pós-processamento voltam para a GUI.

```
using BEM
dad = bem_onelab_run("data/Laplace/onelab_square.geo"; ndiv=12)
# interativo:
# bem_onelab_gui("data/Laplace/onelab_square.geo")
```

Na linha de comando:

```
julia --project=. scripts/bem_onelab.jl data/Laplace/onelab_square.geo
julia --project=. scripts/bem_onelab.jl data/Laplace/onelab_square.geo --gui
```

Para registrar como solver do Gmsh do sistema (**Tools** → **Options** → **Solver**):

```
julia --project=C:/caminho/BEM_gmsh C:/caminho/BEM_gmsh/scripts/bem_onelab.jl %s
```

6.9 Boas práticas de malha para o BEM

1. **Contorno fechado e orientado** — curvas no sentido anti-horário no contorno externo (normal para fora).
2. **Furos** — contornos internos no sentido horário.
3. **Cantos com CDC mista** — elementos descontínuos (padrão do código) evitam conflito de nós compartilhados.
4. **Refino local** — use Distance + Threshold perto de furos e reentrâncias (placa_com_furo é um bom modelo).
5. **Ordem geométrica = ordem de interpolação** — malha de ordem 2 combina com elementos quadráticos.
6. **Estudo de convergência** — varie n_{div} (ou lc) e reporte erro médio / máximo / L_2 (capítulo de medidas de erro).
7. **Não exagere nos pontos internos** — eles encarecem o DIBEM; comece com malha de contorno e poucos internos.

6.10 Exercícios

1. Reproduza o quadrado unitário com $T = x$ a partir de um .geo próprio (não use quadrado). Compare rel_error com $n_{div} = 8, 16, 32$.
2. Modele uma placa retangular com um furo circular deslocado do centro. Imponha $T = 0$ à esquerda, $T = 1$ à direita e isolamento no restante. Plote o campo com `plot_geo` e exporte com `export_results_to_gmsh`.
3. Calcule perímetro, área e centróide da figura do item 2 com `geometric_props` e confira com a fórmula analítica (retângulo menos círculo).
4. Abra `onelab_square.geo` no modo GUI e varie n_{div} . Registre o tempo de montagem densa vs. `assembly=hmat` para o maior n_{div} que sua máquina permitir.

6.11 Leitura e código de referência

- Documentação do projeto: BEM_gmsh/docs (EN e pt-BR)
- Começando: docs/src/pt-br/getting_started.md
- ONELAB: docs/src/api/onelab.md
- Exemplos: scripts/intro.jl, data/examples/, scripts/bem_onelab.jl
- Manual Gmsh: gmsh.info/doc

7 Laplace 2D

- Código de referência: [BEM_gmsh](#)

Clone o repositório, abra Julia **na pasta do projeto** e prepare o ambiente:

```
using Pkg
Pkg.activate(".")
Pkg.instantiate()

using DrWatson
@quickactivate :BEM
include(datadir("Laplace", "Laplace_dad.jl"))
```

Fluxo mínimo (quadrado unitário, solução exata $T = x$):

```
props = Laplace(1.0)
msh    = quadrado(ndiv=20, show=false)
dad    = format2d(msh, props)

attach_analytical!(dad, ana_laplace_linear(; direction=SA[1.0, 0.0]))
H_G_full_direct(dad, 20) # montagem densa; use H_G_Hmat(dad) se N for grande
solve(dad)

@show rel_error(dad)
plot_geo(dad)
# export_results_to_gmsh(dad, msh, :T; viewer=false)
```

Os grupos físicos da malha já carregam as CDCs ("0;T" Dirichlet, "1;q" Neumann com $q = -k\partial\frac{T}{\partial}n$).
Detalhes no capítulo **Gmsh e malhas**.

8 Mapa do BEM (leia isto antes da álgebra)

Antes das fórmulas, fixe o **pipeline** inteiro. Cada bloco das seções seguintes preenche **um** passo desta cadeia — inclusive o cálculo da diagonal de H , que é só um detalhe técnico do passo 4.

1. PDE $\nabla^2 T = 0$ em Ω , com CDCs em $\Gamma = \partial\Omega$

↓

2. Resíduos ponderados + integração por partes (Green) o operador sai de T e vai para a função peso v

↓

3. Solução fundamental (SF) escolhe-se $v = T^*$ tal que $-\nabla^2 T^* = \delta(x - x_d)$; o domínio some e sobra uma **equação integral só no contorno**

↓

4. Discretização T e $q \approx$ funções de forma nos elementos; integrais $\rightarrow$ matrizes H e G (aqui entram quadratura, singularidades e a **diagonal de H**)

↓

5. Condições de contorno (CDC) em cada nó, T ou q é conhecido $\rightarrow$ reorganiza-se $HT = Gq$ em $Ax = b$

↓

6. Solve $x = A^{-1}b \rightarrow$ contorno completo (T, q) em todos os nós

↓

7. Pontos internos (pós-processamento) com (T, q) no contorno já conhecidos, $T(x_d)$ no interior é só uma integral — **sem** novo sistema linear

Correspondência com o BEM_gmsh:

Passo	No código
1–3 formulação	já embutida em fundamental / montagem
4 H, G	H_G_full_direct(dad, npg) ou H_G_Hmat(dad)
5 CDC	grupos físicos Gmsh + format2d / attach_analytical!
6 solve	solve(dad)
7 internos	automático se pontointerno=true; ver também plot_geo

Ideia-chave. No MEF montamos “rigidez” no **volume**. No BEM montamos H e G no **contorno**; o preço é que essas matrizes são cheias e as integrais podem ser singulares quando o ponto fonte x_d cai no elemento que está sendo integrado.

9 Formulação

9.1 Passos 1–3: da PDE à equação integral

A equação de Laplace é dada por:

$$\nabla^2 T = 0,$$

usando o método dos resíduos ponderados e aplicando a segunda identidade de Green (com $u = T$ e peso v):

$$\int_{\Omega} (v \nabla^2 T - T \nabla^2 v) d\Omega = \int_{\Gamma} \left(v \frac{\partial T}{\partial n} - T \frac{\partial v}{\partial n} \right) ds$$

obtem-se a equação integral de contorno:

$$0.5T(x_d, y_d) = \int_{\Gamma} T q^* ds - \int_{\Gamma} T^* q ds,$$

onde T^* e q^* são as soluções fundamentais:

$$T^* = \frac{-1}{2\pi k} \ln r$$

$$q^* = \frac{1}{2\pi r^2} [(x - x_d)n_x + (y - y_d)n_y].$$

O coeficiente 0.5 à esquerda vale para ponto fonte x_d **sobre** um contorno liso. É o termo de corpo rígido / ângulo sólido: geometricamente, metade do “salto” da solução fundamental fica de cada lado do contorno. (Em cantos o fator não é $\frac{1}{2}$; por isso o código prefere elementos **descontínuos** ou calcula a diagonal de H de forma indireta – passo 4.)

9.2 Passo 4: discretização → matrizes H e G

Podemos representar a temperatura e fluxo em um elemento descontínuo com m nós como: $T = N_1 T_1 + N_2 T_2 + N_3 T_3 + \dots + N_m T_m$ $q = N_1 q_1 + N_2 q_2 + N_3 q_3 + \dots + N_m q_m$ onde estamos aproximando nossa distribuição de temperatura e fluxo no elemento por uma função polinomial de grau $(m - 1)$.

Discretizando em n elementos de contorno descontínuos:

$$0.5T(x_d, y_d) = \sum_{j=1}^{n_{elem}} \left[\int_{\Gamma_j} T q^* d\Gamma \right] - \sum_{j=1}^{n_{elem}} \left[\int_{\Gamma_j} T^* q d\Gamma \right]$$

Usando a representação de T e q na equação acima temos:

$$0.5T(x_d, y_d) = \sum_{j=1}^{n_{elem}} \left\{ \int_{\Gamma_j} [N_1 \ N_2 \ N_3 \ \dots \ N_m] \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ \vdots \\ T_m \end{bmatrix}_j q^* d\Gamma \right\} - \sum_{j=1}^{n_{elem}} \left\{ \int_{\Gamma_j} T^* [N_1 \ N_2 \ N_3 \ \dots \ N_m] \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ \vdots \\ q_m \end{bmatrix}_j d\Gamma \right\}$$

Repetindo essa equação para cada diferente nó-fonte $x_d = x_i$ montamos um sistema matricial:

$$HT = Gq$$

- Coluna ligada a T : integrais de $q^* N_k \rightarrow$ entradas de H (núcleo **mais** singular).
- Coluna ligada a q : integrais de $T^* N_k \rightarrow$ entradas de G (singularidade fraca, integrável).
- A linha i é a equação integral escrita com ponto fonte no nó i .

9.2.1 Diagonal de H : por que é especial?

Quando o ponto fonte x_i está **no mesmo elemento** que o ponto de integração, $r \rightarrow 0$ e $q^* \sim \frac{1}{r}$ (em 2D) deixa de ser uma integral comum. Integrar H_{ii} “na marra” é possível, mas delicado (transformações, partes finitas, etc.).

9.2.2 Método indireto (corpo a temperatura constante)

Em vez de atacar a singularidade forte de frente, usamos uma solução **exata** trivial do problema de Laplace:

$$T(x) \equiv 1, \quad q(x) \equiv 0.$$

Ela satisfaz $\nabla^2 T = 0$ e a equação integral. No sistema discreto isso vira

$$H\{1\} = G\{0\} = \{0\}$$

Ou seja, **cada linha de H soma zero**. Como os termos fora da diagonal H_{ij} ($i \neq j$) são integrais regulares (já calculadas por Gauss), a diagonal sai de graça:

$$H_{ii} = - \sum_{j=1, j \neq i}^N H_{ij}, \quad i = 1, \dots, N.$$

Isso **já inclui** o fator de ângulo sólido (o “0.5” no contorno liso, ou outro valor em cantos).

Ordem prática na montagem (passo 4):

1. zere H e G ;
2. para cada par (nó-fonte i , elemento j), integre e **some** nas colunas certas de H e G — **pule** ou trate à parte o caso singular $i \in$ elemento j em H ;
3. corrija a diagonal: $H_{ii} = - \sum_{j \neq i} H_{ij}$;
4. (opcional) a diagonal de G tem só singularidade fraca e em geral **é** integrada com quadratura adequada — o truque indireto raramente é necessário.

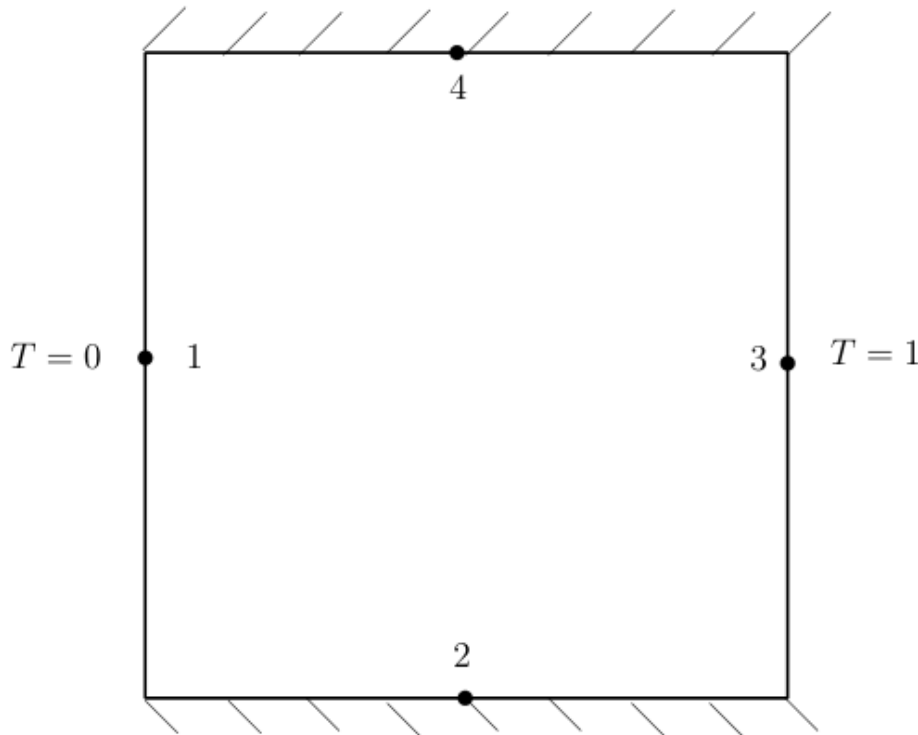
No BEM_gmsh, os passos 2–3 estão dentro de `H_G_full_direct`.

9.3 Passos 5–6: CDCs e o sistema $Ax = b$

Depois de montar $HT = Gq$, **ainda não se resolve nada**: em cada nó falta uma informação. A CDC diz qual coluna vai para o lado esquerdo (incógnita) e qual contribui para o lado direito (dado).

9.3.1 Exemplo (1 elemento por lado)

A fim de ilustrar como se aplica as condições de contorno e se calcula as variáveis desconhecidas será analisado um problema de condução de calor unidirecional com uma discretização de um elemento por lado.



As equações obtidas podem ser escritas na forma matricial, como:

$$\begin{pmatrix} H_{11} & H_{12} & H_{13} & H_{14} \\ H_{21} & H_{22} & H_{23} & H_{24} \\ H_{31} & H_{32} & H_{33} & H_{34} \\ H_{41} & H_{42} & H_{43} & H_{44} \end{pmatrix} \begin{pmatrix} \bar{T}_1 \\ \bar{T}_2 \\ \bar{T}_3 \\ \bar{T}_4 \end{pmatrix} = \begin{pmatrix} G_{11} & G_{12} & G_{13} & G_{14} \\ G_{21} & G_{22} & G_{23} & G_{24} \\ G_{31} & G_{32} & G_{33} & G_{34} \\ G_{41} & G_{42} & G_{43} & G_{44} \end{pmatrix} \begin{pmatrix} q_1 \\ \bar{q}_2 \\ q_3 \\ \bar{q}_4 \end{pmatrix}$$

onde $\bar{T}$ e $\bar{q}$ são termos **conhecidos** (CDC).

Separando os termos conhecidos dos desconhecidos:

$$\begin{pmatrix} -G_{11} & H_{12} & -G_{13} & H_{14} \\ -G_{21} & H_{22} & -G_{23} & H_{24} \\ -G_{31} & H_{32} & -G_{33} & H_{34} \\ -G_{41} & H_{42} & -G_{43} & H_{44} \end{pmatrix} \begin{pmatrix} q_1 \\ T_2 \\ q_3 \\ T_4 \end{pmatrix} = \begin{pmatrix} -H_{11} & G_{12} & -H_{13} & G_{14} \\ -H_{21} & G_{22} & -H_{23} & G_{24} \\ -H_{31} & G_{32} & -H_{33} & G_{34} \\ -H_{41} & G_{42} & -H_{43} & G_{44} \end{pmatrix} \begin{pmatrix} \bar{T}_1 \\ \bar{q}_2 \\ \bar{T}_3 \\ \bar{q}_4 \end{pmatrix}$$

Assim, pode-se escrever $Ax = b$. Resolvendo o sistema linear obtém-se as variáveis que faltavam no contorno. No BEM_gmsh isso é o `solve(dad)`.

Regra prática por nó i :

- se T_i é prescrito (Dirichlet): a incógnita é $q_i \rightarrow$ coluna i de $-G$ entra em A , e $H_{\cdot i}T_i$ vai para b ;
- se q_i é prescrito (Neumann): a incógnita é $T_i \rightarrow$ coluna i de H entra em A , e $G_{\cdot i}q_i$ vai para b .

9.4 Passo 7: pontos internos

A equação integral para pontos **interiores** ($x_d \in \Omega$, fora de Γ) é ligeiramente modificada — o coeficiente à esquerda vira 1, não 0.5:

$$T(x_d, y_d) = \int_{\Gamma} T q^* ds - \int_{\Gamma} T^* q ds$$

Detalhe importante: com T e q já **conhecidos em todo o contorno** (passos 5–6), a temperatura em qualquer ponto interno é só avaliar essas integrais. Não se monta nem se resolve um novo $Ax = b$.

10 Exemplo resolvido ponta a ponta (BEM_gmsh)

Problema: quadrado unitário, $k = 1$, CDCs tais que a solução exata é $T(x, y) = x$ (e portanto $q = -\partial_{\bar{z}} T$). Malha Gmsh com $\text{ndiv}=16$, montagem densa, impressão de números e gráfico.

```
using DrWatson
@quickactivate :BEM
using LinearAlgebra, Printf
include(datadir("Laplace", "Laplace_dad.jl"))

# --- 1. Física e malha ---
props = Laplace(1.0) # k = 1
msh = quadrado(ndiv=16, show=false) # grava .msh e devolve o caminho
dad = format2d(msh, props; pontointerno=true)

println("Arquivo de malha: ", msh)
println("Nós de contorno N = ", dad.n)
println("Pontos internos = ", length(dad.internalNodes))

# --- 2. Campo analítico T = x (bate com as CDCs padrão do quadrado) ---
ana = ana_laplace_linear(; direction=SA[1.0, 0.0])
attach_analytical!(dad, ana)

# --- 3. Montagem H, G (densa) e solução ---
H_G_full_direct(dad, 16) # npg = 16
solve(dad)

# --- 4. Números na tela ---
err = rel_error(dad)
@printf "Erro relativo (rel_error) = %.6e\n" err

println("\nPrimeiros nós do contorno:")
println(" i | x | y | T_num | T_exato | q_num")
for i in 1:min(8, dad.n)
    x, y = dad.Nodes[i]
    T_ex = x # T = x
    @printf "%2d | %6.3f | %6.3f | %8.5f | %8.5f | %9.5f\n" i x y dad.T[i] T_ex dad.q[i]
end

# Erro pontual máximo em T no contorno
eT = maximum(abs(dad.T[i] - dad.Nodes[i][1]) for i in 1:dad.n)
@printf "\nmax |T_num - x| no contorno = %.6e\n" eT

# --- 5. Gráfico ---
fig = plot_geo(dad) # visualização do solver (backend do projeto)
# save("laplace_quadrado.png", fig) # descomente para gravar PNG
fig
```

O que você deve observar:

- `rel_error` cai ao aumentar `ndiv` (faça `ndiv = 8, 16, 32` e anote numa tabela);
- nos lados verticais, $T \approx x$ (constante em cada lado); nos horizontais, $q \approx 0$;
- pontos internos, se houver, seguem $T \approx x$ sem novo sistema linear.

11 Montagem densa vs. H-matriz

Até $N \simeq 3 \cdot 10^3 - 5 \cdot 10^3$ nós, a montagem **densa** (`H_G_full_direct`) costuma ser a escolha didática e prática: implementação simples, fatoração LU direta, ótima para estudos de convergência.

Limitações do BEM denso:

- memória $O(N^2)$ para armazenar H e G ;
- tempo de montagem e de fatoração também $O(N^2) - O(N^3)$;
- em malhas finas (furos, camadas limite geométricas, 3D) o custo explode antes da física ficar interessante.

Quando N cresce, o BEM_gmsh oferece montagem **hierárquica**:

```
msh = quadrado(ndiv=80, show=false, nome="quad_fine")
dad = format2d(msh, Laplace(1.0); pontointerno=false)
attach_analytical!(dad, ana_laplace_linear(; direction=SA[1.0, 0.0]))

H_G_Hmat(dad; atol=1e-6, nmax=32) # blocos comprimidos
@show compression_ratio(dad.H)
solve(dad) # GMRES sobre operador misto
@show rel_error(dad)
```

Critério	Densa <code>H_G_full_direct</code>	H-matriz <code>H_G_Hmat</code>
Memória	$O(N^2)$	tipicamente $O(N \log N)$
Uso no curso	padrão; $N \simeq 10^2 - 10^3$	malhas grandes; projeto avançado
Solver	LU direta	iterativo (GMRES)
Controle	<code>npg</code>	<code>atol</code> , <code>nmax</code> , ...

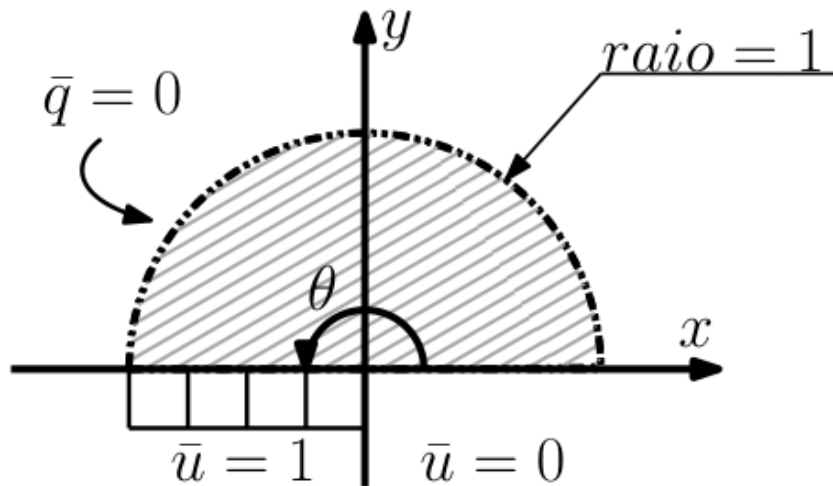
Regra prática: comece **sempre** denso e com malha grossa; só mude para H-matriz quando o denso não couber na RAM ou demorar demais. Detalhes extras no capítulo **Gmsh e malhas** e em `docs/src/pt-br/performance.md` do repositório.

11.1 Exercícios

Use as medidas de erro do capítulo **Medidas de erro** (e/ou `rel_error(dad)` do BEM_gmsh). Notação: potencial T , fluxo $q = -k \partial \frac{T}{\partial n}$.

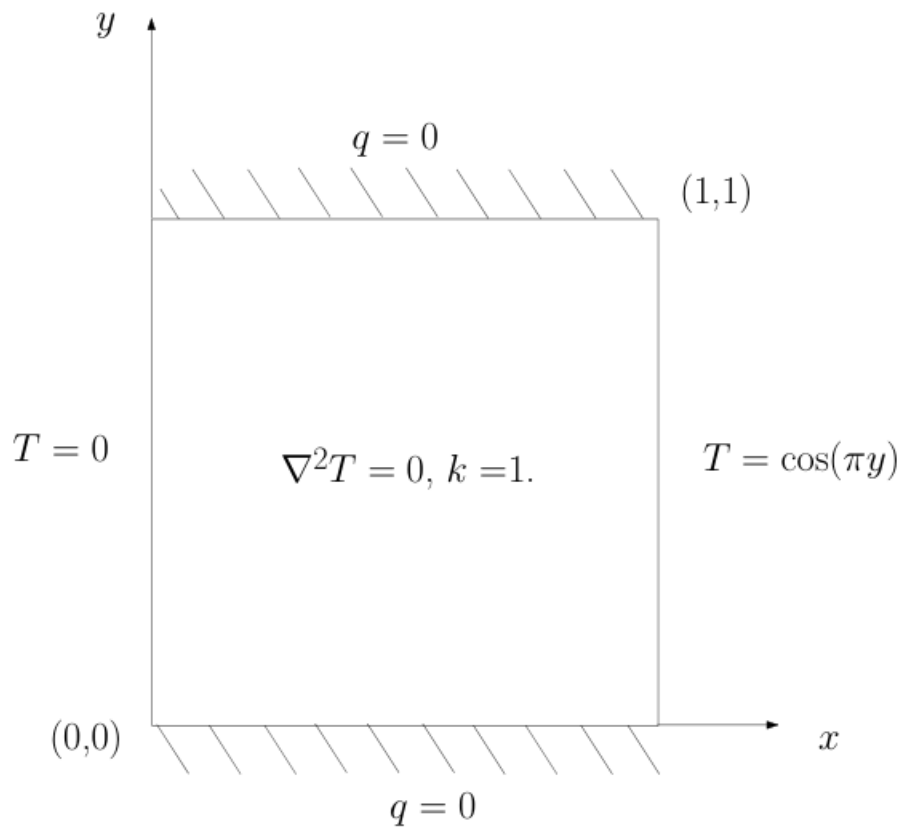
1. Resolva esse problema com elementos lineares, quadráticos e cúbicos e calcule o erro médio, o erro máximo e a norma L_2 do erro no contorno para diferentes discretizações. Faça um gráfico (Plots) comparando a convergência dos três tipos de elementos. A solução analítica é dada por:

$$T(\theta) = \frac{\theta}{\pi}$$
$$q(x) = -\frac{1}{\pi x} \quad \text{em} \quad y = 0$$



1. Analise o seguinte problema de placa com condições de contorno mistas:

- $T(x=0) = 0$
- $T(x=1) = \cos(\pi y)$
- $q(y=0) = q(y=1) = 0$

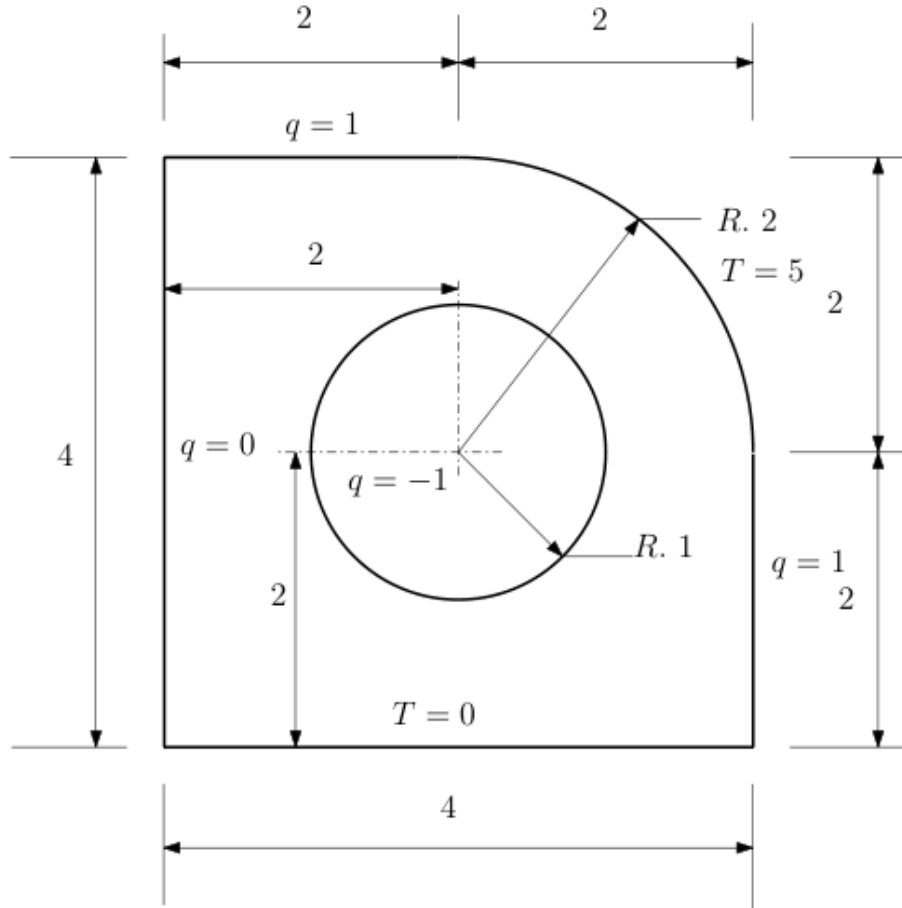


A solução analítica é:

$$T^{\text{an}} = \sinh(\pi x) \frac{\cos(\pi y)}{\sinh(\pi)}$$

Varie o número de elementos e faça uma tabela com o erro percentual no ponto interno $(x, y) = \left(\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2}\right)$.

1. Faça um mapa de cor da distribuição de temperatura em uma placa com dimensões e condições de contorno mostradas na figura.



11.2 Desafio

Baseado nesse [artigo](#) calcule o coeficiente de sustentação de um perfil NACA usando o BEM.

[gravação](#)

12 Medidas de erro

Nas comparações com solução analítica use sempre **erro relativo** normalizado. Seja $\mathbf{u}$ o vetor numérico e $\mathbf{u}^{\text{exact}}$ a referência nos mesmos pontos.

12.1 Erro médio

$$\|\mathbf{u}\|_{\text{med}} = \sum_i |u_i|$$

$$\varepsilon_u = \frac{\|\mathbf{u} - \mathbf{u}^{\text{exact}}\|_{\text{med}}}{\|\mathbf{u}^{\text{exact}}\|_{\text{med}}}$$

12.2 Erro máximo

$$\|\mathbf{u}\|_{\text{max}} = \max_i |u_i|$$

$$\varepsilon_u = \frac{\| \mathbf{u} - \mathbf{u}^{\text{exact}} \|_{\max}}{\| \mathbf{u}^{\text{exact}} \|_{\max}}$$

12.3 Norma L_2 do erro

$$\| \mathbf{u} \|_{L_2} = \sqrt{\sum_{i=1}^{N_e} (u_i)^2}$$

$$\varepsilon_u = \frac{\| \mathbf{u} - \mathbf{u}^{\text{exact}} \|_{L_2}}{\| \mathbf{u}^{\text{exact}} \|_{L_2}}$$

12.4 No BEM_gmsh

Com solução analítica anexada (attach_analytical! ou apply_analytical_bc!):

```
err = rel_error(dad)    # erro relativo agregado no contorno
@show err
```

Reporte sempre as três medidas (médio, máximo, L_2) em estudos de convergência e indique se o erro é medido no contorno, em pontos internos ou em ambos.

13 Poisson 2D

14 MECID / DIBEM

A equação de Poisson é dada por:

$$\nabla^2 T = f(x, y),$$

um novo termo aparece na equação integral de contorno:

$$0.5T(x_d, y_d) = \int_{\Gamma} T q^* ds - \int_{\Gamma} T^* q ds + \int_{\Omega} T^* f d\Omega,$$

esse termo precisa de um tratamento extra. Vamos aproximar $T^* f$ por funções de base radial:

$$T^* f = \sum_{j=1}^n \phi(r_j) \alpha_j.$$

ϕ é uma função de base radial que será tomada aqui por $r^2 \log(r)$. α são coeficientes desconhecidos que podem ser obtidos aplicando essa equação n vezes e montando um sistema matricial:

$$\begin{bmatrix} \phi(\mathbf{X}_1 - \mathbf{X}_1) & \phi(\mathbf{X}_1 - \mathbf{X}_2) & \cdots & \phi(\mathbf{X}_1 - \mathbf{X}_N) \\ \phi(\mathbf{X}_2 - \mathbf{X}_1) & \phi(\mathbf{X}_2 - \mathbf{X}_2) & \cdots & \phi(\mathbf{X}_2 - \mathbf{X}_N) \\ \vdots & \vdots & \ddots & \vdots \\ \phi(\mathbf{X}_N - \mathbf{X}_1) & \phi(\mathbf{X}_N - \mathbf{X}_2) & \cdots & \phi(\mathbf{X}_N - \mathbf{X}_N) \end{bmatrix} \begin{Bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_N \end{Bmatrix} = \begin{Bmatrix} f(\mathbf{X}_1) T^*(\mathbf{X}_d, \mathbf{X}_1) \\ f(\mathbf{X}_2) T^*(\mathbf{X}_d, \mathbf{X}_2) \\ \vdots \\ f(\mathbf{X}_N) T^*(\mathbf{X}_d, \mathbf{X}_N) \end{Bmatrix} = \begin{Bmatrix} T^*(\mathbf{X}_d, \mathbf{X}_1) & \cdots & 0 \\ 0 & T^*(\mathbf{X}_d, \mathbf{X}_2) & 0 \\ \vdots & \ddots & \vdots \\ 0 & 0 & T^*(\mathbf{X}_d, \mathbf{X}_N) \end{Bmatrix} \begin{Bmatrix} f(\mathbf{X}_1) \\ f(\mathbf{X}_2) \\ \vdots \\ f(\mathbf{X}_N) \end{Bmatrix}$$

escrito de maneira sintética:

$$\alpha = F^{-1} D f.$$

Voltando a integral de domínio:

$$\int_{\Omega} T^* f d\Omega = \int_{\Omega} \sum_{j=1}^n \phi(r_j) \alpha_j d\Omega = \sum_{j=1}^n \int_{\Omega} \phi(r_j) d\Omega F^{-1} Df = s F^{-1} Df = s_f Df$$

$$= \begin{Bmatrix} s_1 & s_2 & \dots & s_n \end{Bmatrix} F^{-1} \begin{Bmatrix} T^*(X_d, X_1) & \dots & 0 \\ 0 & T^*(X_d, X_2) & 0 \\ \vdots & \ddots & \vdots \\ 0 & 0 & T^*(X_d, X_N) \end{Bmatrix} \begin{Bmatrix} f(X_1) \\ f(X_2) \\ \vdots \\ f(X_N) \end{Bmatrix} = \begin{Bmatrix} s_{f1} T^*(X_d, X_1) & s_{f2} T^*(X_d, X_2) & \dots & s_{fN} T^*(X_d, X_N) \end{Bmatrix}$$

Fazendo isso para os n pontos fontes:

$$\begin{Bmatrix} s_{f1} T^*(X_1, X_1) & s_{f2} T^*(X_1, X_2) & \dots & s_{fN} T^*(X_1, X_N) \\ s_{f1} T^*(X_2, X_1) & s_{f2} T^*(X_2, X_2) & \dots & s_{fN} T^*(X_2, X_N) \\ \vdots & \vdots & \ddots & \vdots \\ s_{f1} T^*(X_N, X_1) & s_{f2} T^*(X_N, X_2) & \dots & s_{fN} T^*(X_N, X_N) \end{Bmatrix} \begin{Bmatrix} f(X_1) \\ f(X_2) \\ \vdots \\ f(X_N) \end{Bmatrix} = M f$$

a diagonal dessa matriz não é bem definida devido a singularidade da solução fundamental. Ela será calculada de maneira indireta como feito com a matriz H .

Esse procedimento deve ser capaz de calcular essa integral quando a função f for constante e igual a 1.

$$I_{1d} = \int_{\Omega} T^*(X_d, X) d\Omega$$

Igualando as equações:

$$M[1] = [I_1]$$

a diagonal da matriz M deve ser dada por

$$M_{ii} = I_{1i} - \sum_{j=1}^N M_{ij}, \text{ com } i \neq j, \text{ para } i = 1, 2, \dots, N,$$

No BEM_gmsh o termo de domínio é montado por **DIBEM** (funções de base radial) e fica disponível após a montagem de H e G :

```
using DrWatson
@quickactivate :BEM
include(datadir("Laplace", "Laplace_dad.jl"))

props = Laplace(1.0)
msh = quadrado(ndiv=24, show=false)
dad = format2d(msh, props; pontointerno=true) # internos entram no DIBEM

H_G_full_direct(dad, 16)
DIBEM(dad) # monta a matriz de domínio (tipo massa) M
# DIBEM(dad; rbf=PHS(3; poly_deg=0)) # variante de RBF

solve(dad) # estacionário sem fonte
# Transiente (difusão) – o termo M ù é tratado pelo integrador:
# sol = solve_transient(dad, 0.01, 1.0) # 1ª ordem (calor)
# solve_Houbolt(dad, 0.05, 2.0) # 2ª ordem (onda)
# sol = solve_transient_o2(dad, 0.05, 2.0) # 2ª ordem via DifferentialEquations

plot_geo(dad)
```

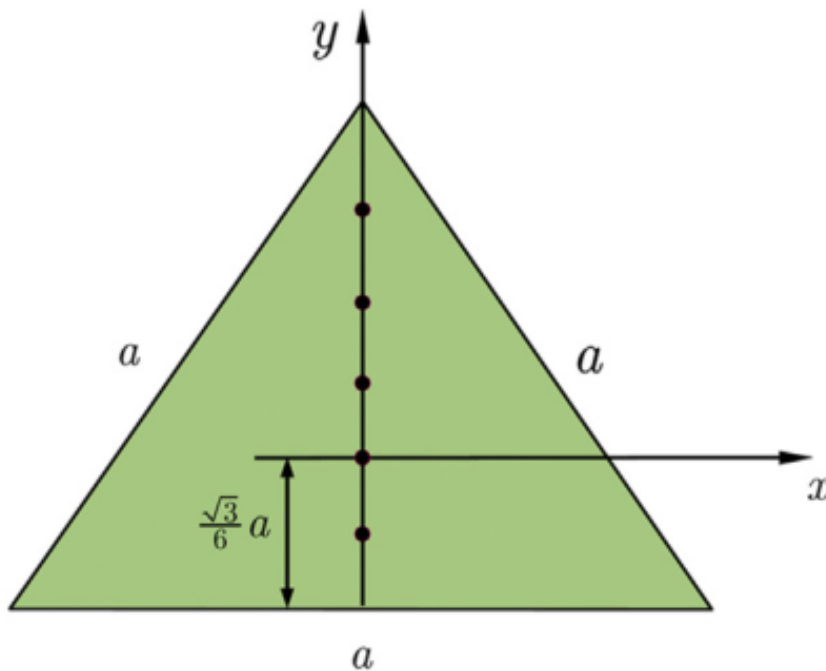
Para carga de domínio estacionária, a contribuição Mf entra no lado direito do sistema após a aplicação das CDCs (veja `Domain.jl` e os exemplos em `docs/src/pt-br/examples.md`). Os pontos internos da malha Gmsh (`Physical Surface`) são reutilizados como centros das RBFs — não é necessário gerar uma malha volumétrica de elementos finitos.

14.1 Exercícios

Todos os exercícios têm de ser resolvidos com diferentes discretizações.

1. Determine a superfície de deflexão de uma membrana elástica em forma de um triângulo equilátero com comprimento lateral $a = 5,0$ m. A membrana está fixa ao longo de sua borda e é submetida a uma carga distribuída uniformemente $f = 10$ kN/m² e uma tensão $S = 1$ kN/m. Os eixos coordenados são tomados como mostrado na figura.

A equação que rege esse problema é: $S\nabla^2 w = -f$

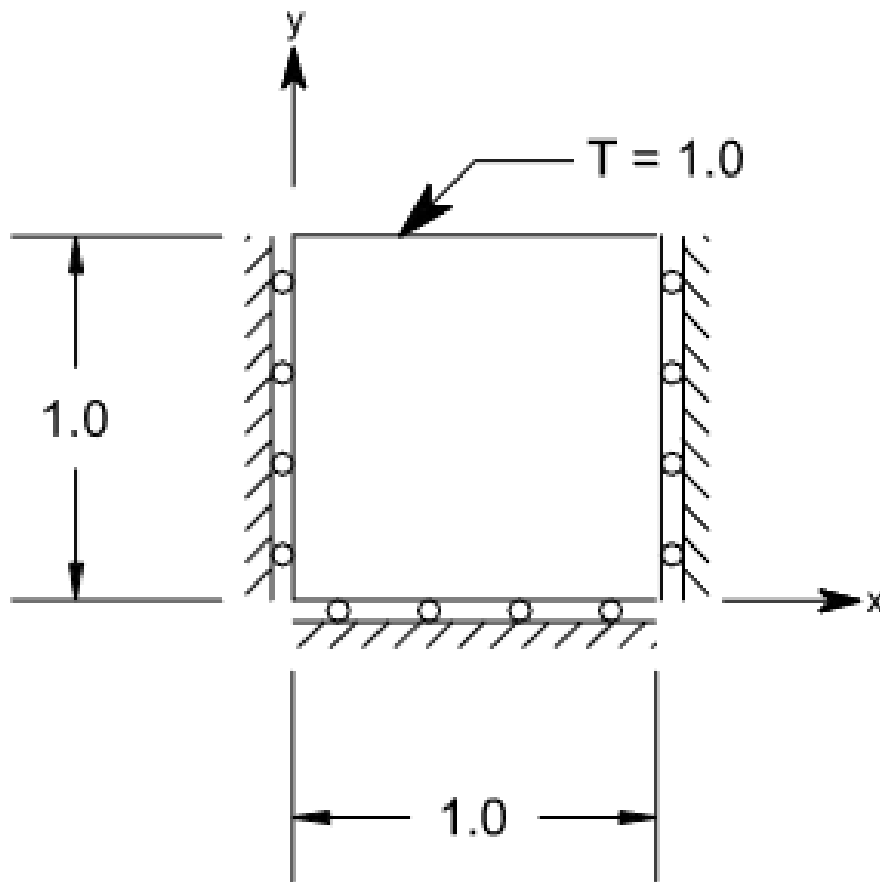


A solução analítica é dada por:

$$w = -\frac{f}{2S} \left[\frac{1}{2}(x^2 + y^2) - \frac{1}{a\sqrt{3}}(y^3 - 3x^2y) - \frac{1}{18}a^2 \right]$$

Calcule o erro médio e a norma L2 do erro em 100 pontos internos. Faça um gráfico com a distribuição de deflexão.

1. Considere um cubo unitário inicialmente a temperatura zero e submetido a um aquecimento súbito em uma de suas faces.



A face aquecida é elevada e mantida a uma temperatura unitária. Supõe-se que as propriedades do material sejam unitárias. A solução analítica para este problema de exemplo pode ser encontrada como:

$$T(Y, t) = 1 - \frac{4}{\pi} \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1} \exp\left\{-\frac{(2n+1)^2 \pi^2 k t}{4L^2}\right\} \cos \frac{(2n+1)\pi Y}{2L}$$

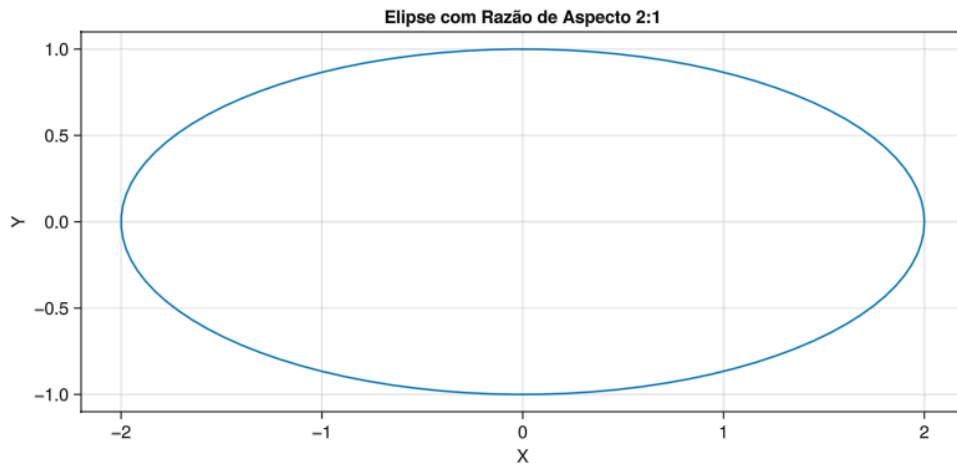
Usando `solve_transient` (ou `Houbolt / solve_transient_o2` quando couber) no `BEM_gmsh`, resolva esse problema e compare com a solução analítica.

1. Considere uma elipse que é governada por: $\nabla^2 u = 4 - x^2$

a solução analítica é dada por:

$$u = \left[1.6 - \frac{1}{246}(50x^2 - 8y^2 + 33.6)\right] \left(\frac{x^2}{4} + y^2 - 1\right) e^{q = 0.4(x^2 + 8y^2) + \frac{1}{246}(-50x^3 - 96xy^2 + 83.2x)\frac{x}{2} + \frac{1}{246}(-96x^2y + 32y^3 - 83.2y)y}$$

calcule os erros do potencial nos pontos internos e do fluxo no contorno.



extra

Um cilindro oco com temperatura inicial zero é considerado. O raio interno $a=1$ e o raio externo $b = 2$. A superfície interna do cilindro é mantida a uma temperatura=1. Nesse caso, são impostas restrições de simetria. As propriedades do material são unitárias. A solução analítica é dada por:

$$T(r, t) = \frac{\ln(b/r)}{\ln(b/a)} T_1 + \pi \sum_{n=1}^{\infty} \frac{J_0(b\alpha_n) J_0(a\alpha_n)}{J_0^2(a\alpha_n) - J_0^2(b\alpha_n)} \{J_0(r\alpha_n) Y_0(b\alpha_n) - J_0(b\alpha_n) Y_0(r\alpha_n)\} e^{-i\alpha_n^2},$$

onde T_i é a temperatura constante na superfície interna, J_0 e Y_0 são as funções de Bessel de primeira e segunda espécies, respectivamente, e α é a raiz de:

$$J_0(ax)Y_0(bx) - J_0(bx)Y_0(ax) = 0.$$

you pode usar a função `fzeros` para encontrar α

14.2 Gravação

https://youtu.be/uSREar_ejnM

15 Elasticidade 2D

Notação (ver glossário): deslocamentos u_i , trações t_i , Poisson do material ν (não confundir com a função peso v da formulação integral).

16 Equações importantes

A relação de equilíbrio de tensão pode ser escrita como:

$$\frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \sigma_{xy}}{\partial y} + f_x = 0$$

$$\frac{\partial \sigma_{yx}}{\partial x} + \frac{\partial \sigma_{yy}}{\partial y} + f_y = 0$$

$$\frac{\partial \sigma_{ij}}{\partial x_j} + f_i = 0$$

Relação deformação-deslocamento pode ser escrita como:

$$\varepsilon_{xx} = \frac{\partial u_x}{\partial x}; \quad \varepsilon_{yy} = \frac{\partial u_y}{\partial y}; \quad \varepsilon_{xy} = \frac{1}{2} \left(\frac{\partial u_x}{\partial y} + \frac{\partial u_y}{\partial x} \right)$$

$$\varepsilon_{ij} = \frac{1}{2} \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} \right)$$

Relação tensão-deformação pode ser escrita como:

$$\varepsilon_{xx} = \left(\frac{1}{E} \right) \sigma_{xx} + \left(\frac{-v}{E} \right) \sigma_{yy}$$

$$\varepsilon_{yy} = \left(\frac{-v}{E} \right) \sigma_{xx} + \left(\frac{1}{E} \right) \sigma_{yy}$$

$$\varepsilon_{xy} = \frac{1}{2\mu} \sigma_{xy}$$

$$\mu = \frac{E}{2(1+v)}$$

Essas três relações podem ser usadas para chegar na seguinte equação diferencial de deslocamentos:

$$\frac{\partial^2 u_x}{\partial x^2} + \frac{\partial^2 u_x}{\partial y^2} + \frac{1}{1-2v} \left(\frac{\partial^2 u_x}{\partial x^2} + \frac{\partial^2 u_y}{\partial x \partial y} \right) = \frac{-f_x}{\mu}$$

$$\frac{\partial^2 u_y}{\partial x^2} + \frac{\partial^2 u_y}{\partial y^2} + \frac{1}{1-2v} \left(\frac{\partial^2 u_y}{\partial y^2} + \frac{\partial^2 u_x}{\partial x \partial y} \right) = \frac{-f_y}{\mu}$$

$$\frac{\partial^2 u_i}{\partial x_j \partial x_j} + \left(\frac{1}{1-2v} \right) \frac{\partial^2 u_j}{\partial x_i \partial x_j} = \frac{-f_i}{\mu}$$

A solução fundamental dessa equação de deslocamento é dada por:

$$U_{ij}(p, Q) = \frac{1}{8\pi\mu(1-v)} \left\{ (3-4v) \ln \left[\frac{1}{r(p, Q)} \right] \delta_{ij} + \frac{\partial r(p, Q)}{\partial x_i} \frac{\partial r(p, Q)}{\partial x_j} \right\}$$

e para a tração:

$$\begin{aligned} T_{ij}(p, Q) &= \frac{-1}{4\pi(1-\nu)r(p, Q)} \left(\frac{\partial r(p, Q)}{\partial n} \right) \\ &\times \left[(1-2\nu)\delta_{ij} + 2 \frac{\partial r(p, Q)}{\partial x_i} \frac{\partial r(p, Q)}{\partial x_j} \right] \\ &+ \frac{1-2v}{4\pi(1-v)r(p, Q)} \left[\frac{\partial r(p, Q)}{\partial x_j} n_i - \frac{\partial r(p, Q)}{\partial x_i} n_j \right] \end{aligned}$$

Usando um procedimento semelhante ao apresentado para o problema potencial obtemos a equação integral de contorno:

$$\begin{aligned} \begin{bmatrix} u_x(p) \\ u_y(p) \end{bmatrix} &+ \int_{\Gamma} \begin{bmatrix} T_{xx}(p, Q) & T_{xy}(p, Q) \\ T_{yx}(p, Q) & T_{yy}(p, Q) \end{bmatrix} \begin{bmatrix} u_x(Q) \\ u_y(Q) \end{bmatrix} d\Gamma(Q) \\ &= \int_{\Gamma} \begin{bmatrix} U_{xx}(p, Q) & U_{xy}(p, Q) \\ U_{yx}(p, Q) & U_{yy}(p, Q) \end{bmatrix} \begin{bmatrix} t_x(Q) \\ t_y(Q) \end{bmatrix} d\Gamma(Q) \end{aligned}$$

$$u_i(p) + \int_{\Gamma} T_{ij}(p, Q) u_j(Q) \, d\Gamma(Q) = \int_{\Gamma} U_{ij}(p, Q) t_j(Q) \, d\Gamma(Q)$$

Essa equação pode ser derivada e junto com as relações tensão deformação obtém-se a equação que pode ser usada para calcular a tensão:

$$\begin{aligned} & \sigma_{ij}(p) + \int_{\Gamma} \left\{ \frac{(2\mu\nu)}{1-2\nu} \delta_{ij} \frac{\partial T_{mk}(p, Q)}{\partial x_m} \right. \\ & \quad \left. + \mu \left[\frac{\partial T_{ik}(p, Q)}{\partial x_j} + \frac{\partial T_{jk}(p, Q)}{\partial x_i} \right] \right\} u_k(Q) \, d\Gamma(Q) \\ &= \int_{\Gamma} \left\{ \frac{(2\mu\nu)}{1-2\nu} \delta_{ij} \frac{\partial U_{mk}(p, Q)}{\partial x_m} + \mu \left[\frac{\partial U_{ik}(p, Q)}{\partial x_j} + \frac{\partial U_{jk}(p, Q)}{\partial x_i} \right] \right\} t_k(Q) \, d\Gamma(Q) \end{aligned}$$

que pode ser reescrita como:

$$\sigma_{ij}(p) + \int_{\Gamma} S_{kij}(p, Q) u_k(Q) \, d\Gamma(Q) = \int_{\Gamma} D_{kij}(p, Q) t_k(Q) \, d\Gamma(Q)$$

onde os tensores S_{kij} e D_{kij} são dados por:

$$\begin{aligned} S_{kij}(p, Q) = & \frac{\mu}{2\pi(1-\nu)} \left(\frac{1}{r^2} \right) n_i \left[2\nu \frac{\partial r}{\partial x_j} \frac{\partial r}{\partial x_k} + (1-2\nu) \delta_{jk} \right] \\ & + \frac{\mu}{2\pi(1-\nu)} \left(\frac{1}{r^2} \right) n_j \left[2\nu \frac{\partial r}{\partial x_i} \frac{\partial r}{\partial x_k} + (1-2\nu) \delta_{ik} \right] \\ & + \frac{\mu}{2\pi(1-\nu)} \left(\frac{1}{r^2} \right) n_k \left[2(1-2\nu) \frac{\partial r}{\partial x_i} \frac{\partial r}{\partial x_j} - (1-4\nu) \delta_{ij} \right] \\ & + \frac{\mu}{\pi(1-\nu)} \left(\frac{1}{r^2} \right) \left(\frac{\partial r}{\partial n} \right) \left[(1-2\nu) \delta_{ij} \frac{\partial r}{\partial x_k} + \nu \left(\delta_{jk} \frac{\partial r}{\partial x_i} + \delta_{ik} \frac{\partial r}{\partial x_j} \right) \right. \\ & \quad \left. - 4 \frac{\partial r}{\partial x_i} \frac{\partial r}{\partial x_j} \frac{\partial r}{\partial x_k} \right] \\ D_{kij}(p, Q) = & \frac{1}{4\pi(1-\nu)} \left(\frac{1}{r} \right) \left[(1-2\nu) \left(\delta_{jk} \frac{\partial r}{\partial x_i} + \delta_{ik} \frac{\partial r}{\partial x_j} - \delta_{ij} \frac{\partial r}{\partial x_k} \right) \right. \\ & \quad \left. + 2 \frac{\partial r}{\partial x_i} \frac{\partial r}{\partial x_j} \frac{\partial r}{\partial x_k} \right] \end{aligned}$$

16.1 Código (BEM_gmsh)

```
using DrWatson
@quickactivate :BEM
include(datadir("Laplace", "Laplace_dad.jl"))

# --- verificação: patch de Dirichlet (ε_xx = 0.01) ---
msh = quadrado_elasticity(ndiv=15, show=false)
dad = format2d(msh, Elasticity(1.0, 0.3, 1.0); pontointerno=false)
```

```

ana = ana_elasticity_patch(; E=1.0, v=0.3, εxx=0.01)
apply_analytical_bc!(dad, ana) # impõe  $u = \varepsilon \cdot x$  em todo o contorno

H_G_full_direct(dad, 16)
solve(dad)
@show rel_error(dad)
plot_geo(dad)

```

Problemas clássicos com malha pronta em data/elastico/iso/:

```

include(datadir("elastico", "iso", "pressurized_tube.jl"))
msh = mesh_pressurized_tube(; ndiv=16, show=false)
dad = format2d(msh, Elasticity(E, v, 1.0); pontointerno=true)
H_G_full_direct(dad, 16)
solve(dad)
# compare com  $u_{\text{tube}}$  /  $\sigma_r_{\text{tube}}$  /  $\sigma_\theta_{\text{tube}}$  no mesmo arquivo

```

Convenção de CDC nos grupos físicos: "tx;ux;ty;uy" (ver capítulo **Gmsh e malhas**).

16.2 Exercícios

16.2.1 Cilindro pressurizado

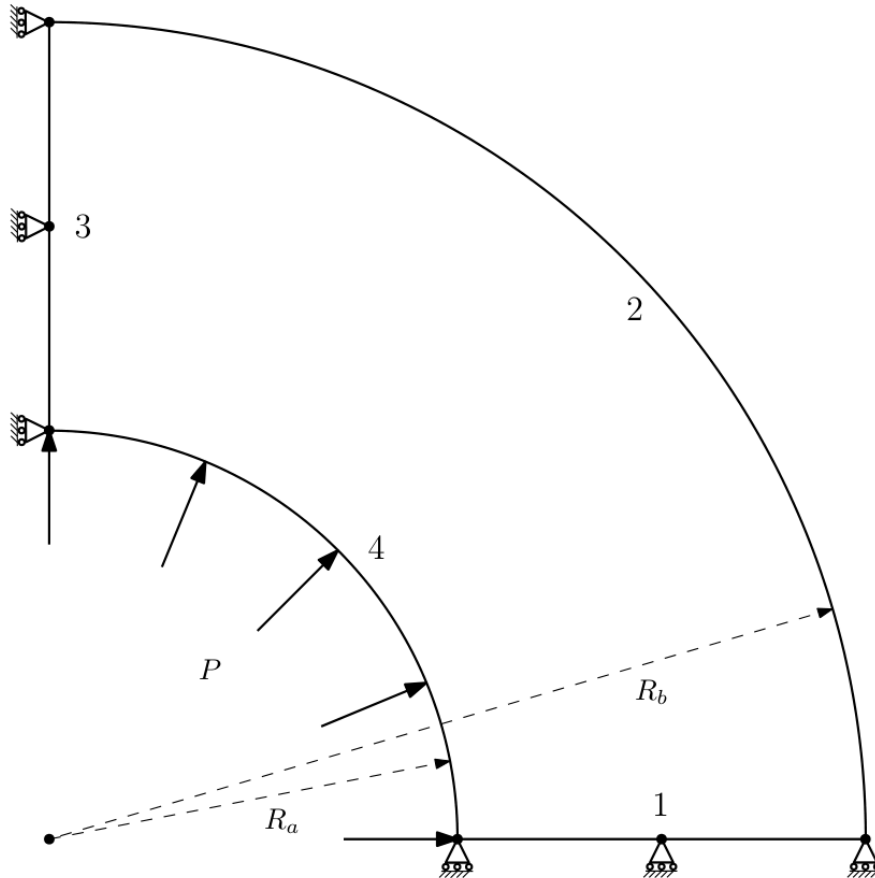
Raio interno $R_a = 50mm$

Raio externo $R_b = 100mm$

Modulo de elasticidade $E = 200GPa$

Poisson $\nu = 0.32$

Pressão $P = 100N/mm$



$$u_r = \frac{(1 + \nu)pr_a^2}{(r_b^2 - r_a^2)E} \left[(1 - 2\nu)r + \frac{r_b^2}{r} \right]$$

$$\sigma_r = \frac{pr_a^2}{r_b^2 - r_a^2} - \frac{r_a^2 r_b^2 p}{r_b^2 - r_a^2} \frac{1}{r^2}$$

$$\sigma_h = \frac{pr_a^2}{r_b^2 - r_a^2} + \frac{r_a^2 r_b^2 p}{r_b^2 - r_a^2} \frac{1}{r^2}$$

16.2.2 Placa com furo

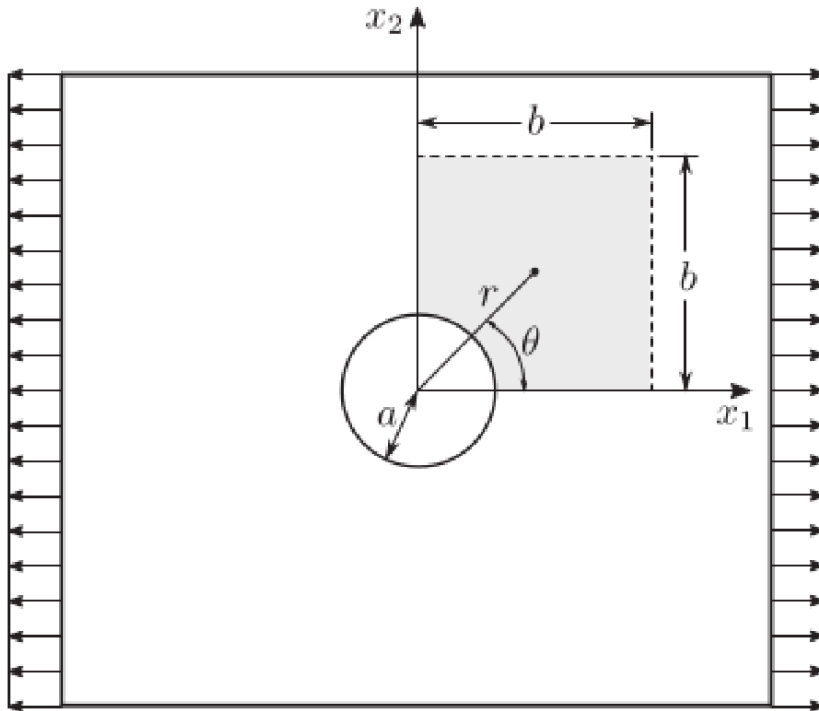
Raio $R = 50mm$

Modulo de elasticidade $E = 100GPa$

Poisson $\nu = 0.25$

Pressão $P = 1N/mm$

Esse problema é de estado plano de tensão. Para ser tratado pelas soluções fundamentais de estado plano de deformação as propriedades deve ser ajustadas:



$$\nu' = \frac{\nu}{1 + \nu}$$

$$E' = E \left[1 - \frac{\nu'^2}{(1 + \nu')^2} \right]$$

A solução analítica é dada por:

$$\sigma_r = \frac{\sigma}{2} \left(1 - \frac{a^2}{r^2} \right) + \frac{\sigma}{2} \left(1 + \frac{3a^4}{r^2} - \frac{4a^2}{r^2} \right) \cos(2\theta)$$

$$\sigma_\theta = \frac{\sigma}{2} \left(1 + \frac{a^2}{r^2} \right) - \frac{\sigma}{2} \left(1 + \frac{3a^4}{r^4} \right) \cos(2\theta)$$

$$\tau_{r\theta} = -\frac{\sigma}{2} \left(1 - \frac{3a^4}{r^4} + \frac{4a^2}{r^2} \right) \sin(2\theta)$$

16.2.3 Viga

Uma viga com um carregamento parabólico $t_2(y) = -\frac{P}{2I} \left(\frac{D^2}{4} - y^2 \right)$ tem solução analítica:

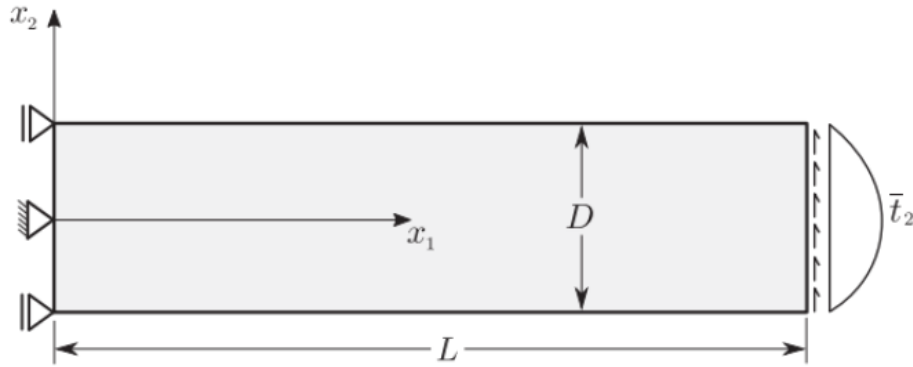
Comprimento $L = 48mm$

Altura $D = 12mm$

Modulo de elasticidade $E = 300GPa$

Poisson $\nu = 0.3$

Pressão $P = 1000N$



$$u_1(x, y) = -\frac{Py}{6EI} \left[(6L - 3x)x + (2 + \nu) \left(y^2 - \frac{D^2}{4} \right) \right]$$

$$u_2(x, y) = \frac{P}{6EI} \left[(3\nu)y^2(L - x) + (4 + 5\nu)\frac{D^2x}{4} + (3L - x)x^2 \right]$$

$$\sigma_{xx}(x, y) = -\frac{P(L - x)y}{I}$$

$$\tau_{xy}(x, y) = -\frac{P}{2I} \left(\frac{D^2}{4} - y^2 \right)$$

Resolva os três exercícios com o BEM_gmsh (malhas em data/elastico/iso/ ou gerador próprio via Gmsh). Reporte erros em deslocamentos e tensões para pelo menos três refinamentos.

gravação

17 Propgeo 3D

A mesma ideia do capítulo **Indo para 2D** se estende a sólidos: volume, área superficial e centróide podem ser obtidos por integrais **apenas na superfície** fechada $\Gamma = \partial\Omega$.

No BEM_gmsh:

```
using DrWatson
@quickactivate :BEM

# Requer malha de superfície fechada (ex.: cubo em data/Laplace)
# dad = format3d(datadir("Laplace", "cubo.msh"), Laplace(1.0))
# gp = geometric_props(dad) # GeometricProps3D
# @show gp.surface_area gp.volume gp.centroid
```

O despacho é automático: `dad.dimension == 3` seleciona `geometric_props_3d`. A integração usa as funções de forma 2D dos elementos de superfície e as normais já armazenadas no BEMdata.

17.1 Exercício

1. Gere um cubo (ou esfera faceted) no Gmsh, leia com `format3d` e compare área, volume e centróide com o valor analítico.
2. Introduza um furo cilíndrico (superfície interna orientada para dentro do material) e repita o cálculo.

17.2 Material complementar

- [Arquivos legados propgeo 3D](#)
- [gravação](#)
- Código atual: `src/Core/GeometricProperties.jl` no BEM_gmsh

18 Trabalhos finais

Curso de **30 horas** — Introdução ao Método dos Elementos de Contorno.

Código: `BEM_gmsh`.

Estas propostas **não** são exercícios de “rodar o exemplo e plotar”. Cada uma exige **leitura de formulação, extensão de malha/CDC, estudo quantitativo e discussão crítica** no nível de uma monografia curta. O núcleo do BEM já está no repositório — o trabalho é **usar a biblioteca como plataforma de pesquisa**, não como caixa-preta.

18.1 Regras gerais

Equipes: 1–2 alunos. Uma proposta por equipe (combinar com o docente se quiser fusão parcial).

Carga esperada: 20–40 h extraclasse depois do núcleo Laplace/elasticidade. Começar cedo.

Entregáveis obrigatórios (todas as propostas):

1. **Relatório** (12–20 páginas, PDF) com:
 - formulação matemática do problema **no formalismo BEM** (não só a PDE);
 - estado da arte curto (3–8 referências relevantes, não genéricas);
 - descrição da malha Gmsh e das CDCs (tipos físicos);
 - o que foi herdado do BEM_gmsh vs. o que a equipe implementou/adaptou;
 - resultados com **números**, tabelas e figuras;
 - análise de erro / convergência / custo;
 - limitações e trabalho futuro.
2. **Repositório ou pasta reproduzível:** scripts Julia, `.geo`/geradores, README com comandos exatos (`julia --project=... script.jl`), seeds e versões.
3. **Contribuição própria** (pelo menos **dois** itens da lista):
 - gerador de malha **novo** (geometria que não está pronta em `data/`);
 - CDC ou pós-processamento não trivial (`export Gmsh`, sensor em linha, SIF, ...);
 - comparação sistemática entre **dois** métodos/solvers/discretizações;
 - estudo de parâmetro físico com curva e interpretação;
 - implementação auxiliar (RBF alternativa, critério de parada, remalhagem simples, ...).
4. **Defesa oral** de 10–15 min (perguntas sobre formulação e sobre o código).

O que não conta como trabalho final: copiar `scripts/intro.jl`, mudar `ndiv` e entregar um gráfico de $T = x$.

Rubrica sugerida:

Critério	Peso
Formulação e domínio do BEM no problema escolhido	20 %
Originalidade / contribuição além dos scripts prontos	25 %
Rigor numérico (convergência, verificação, analítico/literatura)	25 %
Discussão crítica (limites, custo, o que falhou)	15 %
Reprodutibilidade e clareza do relatório/código	15 %

18.2 Proposta A — Mecânica da fratura com Dual BEM (trinca + K_I + propagação)

Nível. Avançado.

Âncoras no código. `src/Crack/`, `data/elastico/iso/center_crack.jl`, `scripts/crack_central.jl`, `data/examples/crack_feddersen.jl`, grupos físicos tipo "5;..." (faces duais).

18.2.1 Problema científico

Placa finita com trinca central (ou trinca de borda / geometria **nova** proposta pela equipe) sob tração remota. Calcular fatores de intensidade de tensão (K_I , e K_{II} se houver modo misto) pelo **Dual BEM** (BIE de deslocamento em uma face + BIE de tração na face coincidente) e confrontar com solução de referência (Feddersen / Tada / handbook).

Em seguida, dar **pelo menos um passo de propagação** com critério MTS (e, se couber, estimativa de vida via Paris já esboçada no script) — discutindo o que o código faz e o que ainda é simplificado.

18.2.2 O que deve ir além do script pronto

Escolha **no mínimo duas** frentes:

- Variar $\frac{a}{W}$ em uma curva $\frac{K_I(\frac{a}{W})}{K_I^\infty}$ e comparar com a correção de Feddersen (ou outra da literatura) em **vários** pontos — não um único a .
- Estudo de convergência em n_{crack} , ordem do elemento e n_{pg} ; extrair taxa observada e discutir singularidade $r^{-\frac{1}{2}}$ na ponta.
- Geometria alternativa: trinca de borda, trinca inclinada (modo misto), ou dois furos + trinca — malha Gmsh **da equipe**.
- Pós-processamento: perfil de COD (abertura) ao longo da trinca; estimativa de K_I por extrapolação de COD vs. por tensão; comparar.
- Um experimento numérico de **caminho**: vários passos MTS com remalhagem manual ou semi-automática (mesmo que tosca), documentando falhas.

18.2.3 Entregas específicas

- Tabela K_I^{num} vs. K_I^{ref} com erro relativo para ≥ 4 razões $\frac{a}{W}$ ou ≥ 4 malhas.
- Figura da malha com faces duais destacadas e normais opostas.
- Discussão: por que o BEM dual evita a degeneração de faces coincidentes; o que acontece se a face B for mal orientada.
- Seção “o que o Dual BEM ainda não faz neste trabalho” (contato entre faces, 3D, plástico, ...).

18.2.4 Pontos de partida (não são o trabalho)

```
using DrWatson
@quickactivate :BEM
using .Crack
include(datadir("elastico", "iso", "center_crack.jl"))
# ver scripts/crack_central.jl e a API CrackTip / propagate!
```

18.3 Proposta B — Multirregião, interfaces e materiais contrastantes

Nível. Avançado.

Âncoras no código. `src/MultiRegion/`, `data/Laplace/two_regions.jl`, `scripts/two_regions_interface.jl`; elasticidade anisotrópica em `data/elastico/aniso/` (Lekhnitskii); DI-BEM se houver fonte por região.

18.3.1 Problema científico

Sistemas em que **um único** contorno exterior não basta: duas ou mais sub-regiões com condutividades (ou módulos) diferentes, acopladas por condições de interface (continuidade de T e equilíbrio de q , ou analogamente em elasticidade).

A equipe formula o bloco do sistema global, implementa/adapta um caso com **contraste forte** de propriedades e quantifica:

- salto numérico na interface ($|T_a - T_b|$, $|q_a + q_b|$);
- erro vs. solução analítica **por região** (quando existir) ou vs. solução de referência monodomínio equivalente;
- sensibilidade ao contraste $\frac{k_1}{k_2}$ (ou $\frac{E_1}{E_2}$) em vários logaritmos de razão.

18.3.2 O que deve ir além do script pronto

Pelo menos duas frentes:

- Geometria **não** retângulo-retângulo: inclusão circular/elíptica, parede compostas, ou três regiões; malha Gmsh própria com grupos por região.
- Caso com solução analítica clássica (ex.: cilindro composto sob pressão / condução radial em coroas concêntricas com k diferentes) **ou** verificação por refino cruzado + conservação de fluxo global.
- Comparar montagem multirregião vs. “truque” de um só domínio quando o contraste $\rightarrow 1$ (regressão).
- Extensão: interface com resistência de contato térmico ($q = h(T_a - T_b)$) — mesmo que implementada de forma simplificada no sistema de interface — **ou** um caso elástico anisotrópico (AnisotropicElasticity / Lekhnitskii) com verificação de patch ou furo.
- Perfil de q normal ao longo da interface e balanço integral $\int_{\Gamma_{\text{ext}}} q \, ds \approx 0$ (estacionário sem fonte).

18.3.3 Entregas específicas

- Diagrama de blocos do sistema algébrico multirregião (incógnitas por região + multiplicadores/condições de interface).
- Curva erro e/ou fluxo residual vs. contraste e vs. N .
- Discussão de condicionamento: o que acontece com $\frac{k_1}{k_2} = 10^3$ ou 10^{-3} .

18.3.4 Pontos de partida

```
using DrWatson
@quickactivate :BEM
include(datadir("Laplace", "two_regions.jl"))
# scripts/two_regions_interface.jl
# data/elastico/aniso/*.jl para a variante mecânica
```

18.4 Proposta C — Dinâmica no contorno: ondas / calor transiente com análise de esquema

Nível. Avançado.

Âncoras no código. DIBEM, solve_transient, solve_transient_o2, solve_Houbolt, data/Laplace/wave_propagation.jl, scripts/wave_propagation.jl, catálogo :bar_sudden, :annulus, :membrane_v0, :membrane_forced, :ricker, ...

18.4.1 Problema científico

Acoplar as matrizes de contorno H, G a uma matriz de domínio tipo massa M (DIBEM) e integrar no tempo um problema **hiperbólico** (onda escalar) **ou parabólico** (difusão) com solução de referência (série de Fourier/Bessel ou d'Alembert em barra).

O foco **não** é “gerar um vídeo”. É responder com números:

- qual a relação entre Δt , h_{elem} , velocidade c (ou difusividade) e estabilidade/precisão;
- como Houbolt, integrador de 2ª ordem (solve_transient_o2) e, se aplicável, 1ª ordem se comportam em **dispersão, dissipação numérica** e custo;
- se a escolha de RBF / densidade de pontos internos domina o erro espacial.

18.4.2 O que deve ir além do script pronto

Pelo menos duas frentes:

- Comparação **head-to-head** Houbolt vs. solve_transient_o2 (e/ou vs. referência modal) na **mesma** malha, com tabelas de erro em sensores e tempo de CPU.
- Estudo de CFL prático: malha fixa, varrer Δt ; depois Δt fixo, varrer n_{div} e n_{int} .
- Caso forçado ou com pulso Ricker: espectro ou tempos de chegada vs. teoria.
- Um caso **novo** de domínio (L-shape, coroa, membrana com obstáculo) com malha própria + condição inicial/no contorno documentada.
- Extra de peso: acoplar exportação temporal para views Gmsh / animação reproduzível + métrica $L_2(\Omega)$ aproximada via pontos internos ao longo do tempo.

18.4.3 Entregas específicas

- Formulação discreta: de $HT = Gq + Mf$ (ou $M\ddot{u}$) ao marchador usado.
- Gráficos espaço-tempo ou snapshots em instantes teóricos de reflexão.
- Tabela de erros vs. Δt e vs. N com ordem observada.
- Discussão honesta: onde o DIBEM polui a solução em relação ao erro de tempo.

18.4.4 Pontos de partida

```
using DrWatson
@quickactivate :BEM
include(datadir("Laplace", "Laplace_dad.jl"))
include(datadir("Laplace", "wave_propagation.jl"))
dad, meta = wave_problem(:bar_sudden; ndiv=12, n_int=6)
# scripts/wave_propagation.jl (WAVE_CASE, WAVE_SOLVER=houbolt|diffeq|both)
```

18.5 Proposta D — H-matrizes, custo e fidelidade em malhas grandes

Nível. Avançado.

Âncoras no código. H_G_Hmat, src/Hmat/, scripts/compare_hbs_hss.jl, scripts/profile_assembly.jl, scripts/plate_large.jl, docs de performance.

18.5.1 Problema científico

Para um problema **verificável** (Laplace $T = x$ ou elasticidade patch / placa com furo com solução de Kirsch), construir a **fronteira de Pareto** precisão $\times$ custo. Não basta um run com H-matriz: o trabalho é **instrumentar**, varrer N e responder com dados **quando** comprimir vale a pena.

18.5.2 O que deve ir além do script pronto

Pelo menos duas frentes:

- Montagem densa vs. H-matriz (e, se possível, mais de um formato: HODLR / HSS / parâmetros atol, nmax) na **mesma** geometria.
- Curvas de rel_error (ou erro em sensores) vs. N ; tempo de montagem, tempo de solve, memória / compression_ratio; N em que o denso se torna inviável na máquina usada.
- Geometria não trivial com N alto (malha própria) **ou** script de benchmark automatizado com tabela/CSV reprodutível.
- Estudo de threads e/ou falha controlada (afrouxar atol até o erro saturar) com interpretação; perfil (@time, TimerOutputs) e gargalo (integração vs. álgebra).

18.5.3 Entregas específicas

- Gráfico único “erro $\times$ tempo” com famílias densa / H-matriz.
- Tabela N , memória, compression_ratio, tempos, erro.
- Texto explícito: **em que regime** a equipe recomenda H-matriz neste hardware.

18.5.4 Pontos de partida

```
julia --project=. scripts/compare_hbs_hss.jl
julia --project=. scripts/profile_assembly.jl
julia --project=. scripts/plate_large.jl
```

18.6 Proposta E — Contato elástico por BEM de semi-espaço / semi-plano

Nível. Avançado.

Âncoras no código. `src/Contact/`, `scripts/hertz_line_2d.jl`, `scripts/contact_pohrt_li.jl`, `scripts/compare_contact_acceleration.jl`, dados de fretting / Cattaneo–Mindlin em `data/elastic/iso/`.

18.6.1 Problema científico

Reproduzir e **estender** um contato clássico (Hertz linha 2D ou Cattaneo–Mindlin / fretting) no formalismo de semi-espaço ou semi-plano do BEM_gmsh. O centro é a física do contato (zona ativa, pressão, deslizamento), não só chamar um script.

18.6.2 O que deve ir além do script pronto

Pelo menos duas frentes:

- Pressão de contato vs. solução analítica de Hertz (erro em p_0 , a_{contato}) e convergência com refino da malha de potencial contato.
- Estudo paramétrico (carga, raio, módulo efetivo) colapsado na forma adimensional de Hertz.
- Discussão do algoritmo de região de contato (ativo/inativo) e comparação de **duas** estratégias de aceleração do repositório, se aplicável.
- Extensão: superfície com rugosidade simples ou indentador não circular; **ou** carga em “ciclo” com análise de deslizamento parcial; **ou** visualização da zona de contato evolutiva + comparação de CPU entre variantes.

18.6.3 Entregas específicas

- Figura $\frac{p(x)}{p_0}$ vs. $\frac{x}{a}$ sobreposta à elipse de Hertz.
- Tabela de erros (p_0 , semi-largura, força resultante) vs. malha.
- Seção sobre limites do modelo de semi-espaço / semi-plano.

18.6.4 Pontos de partida

```
julia --project=. scripts/hertz_line_2d.jl
julia --project=. scripts/contact_pohrt_li.jl
julia --project=. scripts/compare_contact_acceleration.jl
```

18.7 Cronograma sugerido (a partir da 2ª metade do curso)

Semana	Marco
W1	Escolha da proposta + leitura da API/scripts âncora + 1 página de plano
W2	Malha/CDC mínimas rodando; primeiro número comparável à referência
W3	Estudo paramétrico ou de convergência a meio caminho; texto da formulação
W4	Congelar resultados; relatório + README; ensaio da defesa

18.8 Integridade e uso de IA

- Cite o BEM_gmsh, as notas e **todas** as soluções analíticas/handbooks.
- Podem usar assistentes de código, mas a equipe deve explicar **qualquer** trecho na defesa. Código que a equipe não entende = trecho inválido na nota.
- Não entregue resultados que não conseguiu reproduzir do zero no ambiente da disciplina.

18.9 Escolha orientada

Se você curte...	Prefira	Cuidado com
Mecânica dos sólidos / K_I	A — fratura dual	Singularidade na ponta; malha dual
Materiais / interfaces	B — multirregião	Condicionamento com contraste alto
Dinâmica / sinais	C — ondas/transiente	CFL escondido; erro do DIBEM
HPC / algoritmos	D — H-matrizes	Medir mal tempo/memória
Tribologia / contato	E — Hertz / fretting	Algoritmo de zona ativa

Em dúvida, fale com o docente **antes** de W2: trocar de proposta depois do primeiro marco custa caro.

Material extra

Tópicos avançados / aprofundamento opcional

19 Viga de Euler (extra)

Material extra / standalone. Este capítulo aprofunda o BEM 1D em vigas (Euler–Bernoulli, transiente, Timoshenko). **Não** faz parte da trilha obrigatória de 30 h.

O repositório [BEM_gmsh](#) concentra-se em **Laplace/Poisson e elasticidade 2D** (malhas Gmsh, DIBEM, H-matrizes, contato, ondas escalares). **Não** há, no núcleo atual, um solver de viga de Euler–Bernoulli pronto como os de Laplace / Elasticity.

Os códigos deste capítulo são **pedagógicos e autônomos**: rode-os em um script Julia simples (com Plots se for plotar), sem esperar um `Viga(...)` + `format2d` no `BEM_gmsh`. Servem para fixar SF de ordem alta, dualidade deslocamento/rotação e, se desejar, um projeto de monografia.

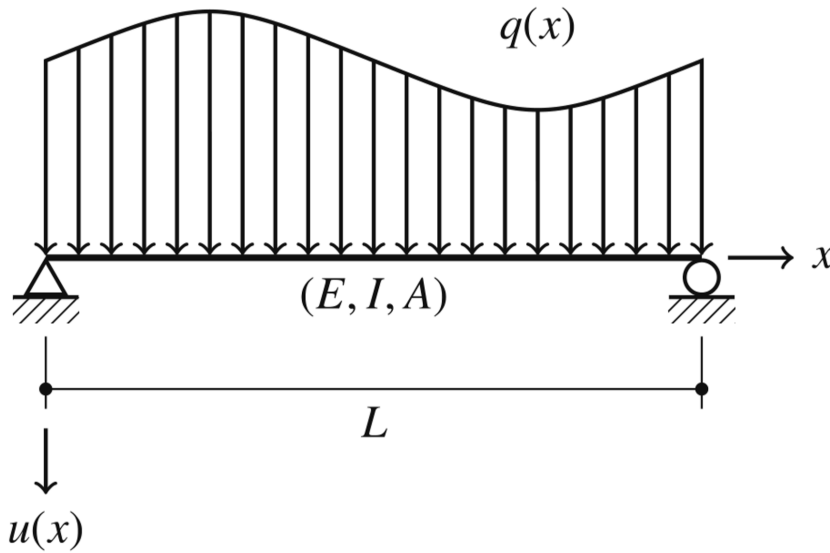
19.1 Teoria de vigas

Considerando a viga representada, a equação governante pode ser escrita como:

$$EI \frac{d^4 u}{dx^4} = q(x)$$

A partir de u definimos a rotação, o momento fletor e a cortante:

$$\varphi = \frac{du}{dx}, \quad M = -EI \frac{d^2 u}{dx^2}, \quad Q = -EI \frac{d^3 u}{dx^3}.$$



Usando essas definições, o método dos resíduos ponderados e integração por partes quatro vezes, obtém-se:

$$\begin{aligned} u(\xi) = & +u^*(\xi, x)Q(x) \big|_{x=L} - u^*(\xi, x)Q(x) \big|_{x=0} \\ & -\varphi^*(\xi, x)M(x) \big|_{x=L} + \varphi^*(\xi, x)M(x) \big|_{x=0} \\ & +M^*(\xi, x)\varphi(x) \big|_{x=L} - M^*(\xi, x)\varphi(x) \big|_{x=0} \\ & -Q^*(\xi, x)u(x) \big|_{x=L} + Q^*(\xi, x)u(x) \big|_{x=0} \\ & + \int_0^L u^*(\xi, x)q(x) dx, \end{aligned}$$

onde:

$$u^*(\xi, x) = \frac{r^3}{12EI},$$

$$\varphi^*(\xi, x) = \frac{r^2}{4EI} \left(\frac{\partial r}{\partial x} \right),$$

$$M^*(\xi, x) = -\frac{r}{2} \left(\frac{\partial r}{\partial x} \right)^2,$$

$$Q^*(\xi, x) = -\frac{1}{2} \left(\frac{\partial r}{\partial x} \right)^3 \text{ e}$$

$$r = |x - \xi|$$

onde $\frac{\partial r}{\partial x} = -1$ se $x < \xi$ e $\frac{\partial r}{\partial x} = 1$ se $x > \xi$.

Para esse problema precisamos de definir mais uma equação para conseguirmos montar um sistema de equação adequado. Isso é obtido derivando a equação anterior em relação à ξ :

$$\begin{aligned} \varphi(\xi) = & + \frac{\partial u^*(\xi, x)}{\partial \xi} Q(x) \Big|_{x=L} - \frac{\partial u^*(\xi, x)}{\partial \xi} Q(x) \Big|_{x=0} \\ & - \frac{\partial \varphi^*(\xi, x)}{\partial \xi} M(x) \Big|_{x=L} + \frac{\partial \varphi^*(\xi, x)}{\partial \xi} M(x) \Big|_{x=0} \\ & + \frac{\partial M^*(\xi, x)}{\partial \xi} \varphi(x) \Big|_{x=L} - \frac{\partial M^*(\xi, x)}{\partial \xi} \varphi(x) \Big|_{x=0} \\ & + \int_0^L \frac{\partial u^*(\xi, x)}{\partial \xi} q(x) dx, \end{aligned}$$

onde:

$$\begin{aligned} \frac{\partial u^*(\xi, x)}{\partial \xi} &= \frac{r^2}{4EI} \left(\frac{\partial r}{\partial \xi} \right), \\ \frac{\partial \varphi^*(\xi, x)}{\partial \xi} &= \frac{r}{2EI} \left(\frac{\partial r}{\partial \xi} \right) \left(\frac{\partial r}{\partial x} \right), \\ \frac{\partial M^*(\xi, x)}{\partial \xi} &= \frac{1}{2} \left(\frac{\partial r}{\partial \xi} \right) \left(\frac{\partial r}{\partial x} \right)^2, \end{aligned}$$

onde $\frac{\partial r}{\partial \xi} = 1$ se $x < \xi$ e $\frac{\partial r}{\partial \xi} = -1$ se $x > \xi$.

```
function solfundviga(ξ, x,E,I,L)
    r=abs(x-ξ)
    if x==ξ
        if ξ==0
            drdx=1
        elseif ξ==L
            drdx=-1
        else
            drdx=0
        end
    else
        drdx=(x-ξ)/r
    end
end
```

```

drdξ=-drdx

u_star = r^3 / (12 * E * I)
phi_star = (r^2 / (4 * E * I)) * drdx
M_star = -r/2 * drdx^2
Q_star = -1/2 * drdx^3

du_star_dxi = (r^2 / (4 * E * I)) * drdξ
dphi_star_dxi = (r / (2 * E * I)) * drdξ * drdx
dM_star_dxi = 1/2 * drdξ * drdx^2

u_star ,phi_star ,M_star ,Q_star ,du_star_dxi ,dphi_star_dxi ,dM_star_dxi
end

```

Considere uma viga com $L = 4$ m, $E = 50$ GPa, $I = 0.0036\text{m}^2$ e uma carga pontual no centro da viga de valor 10KN.

```

L=4
E=50e9
Iv=0.0036

#solfundviga(ξ, x,E,I,L)
u_00 ,phi_00 ,M_00 ,Q_00 ,du_00 ,dphi_00,dM_00= solfundviga(0,0,E,Iv,L)
u_0L ,phi_0L ,M_0L ,Q_0L ,du_0L ,dphi_0L,dM_0L= solfundviga(0,L,E,Iv,L)
u_L0 ,phi_L0 ,M_L0 ,Q_L0 ,du_L0 ,dphi_L0,dM_L0= solfundviga(L,0,E,Iv,L)
u_LL ,phi_LL ,M_LL ,Q_LL ,du_LL ,dphi_LL,dM_LL= solfundviga(L,L,E,Iv,L)
#multiplica U e phi
H=-[Q_00 -M_00 -Q_0L M_0L
    0 dM_00 0 -dM_0L
    Q_L0 -M_L0 -Q_LL M_LL
    0 dM_L0 0 -dM_LL]
#multiplica V e M
G=[-u_00 phi_00 u_0L -phi_0L
    -du_00 dphi_00 du_0L -dphi_0L
    -u_L0 phi_L0 u_LL -phi_LL
    -du_L0 dphi_L0 du_LL -dphi_LL]

uc0 ,phic0 ,Mc0 ,Qc0 ,duc0 ,dphic0,dMc0=solfundviga(0,L/2,E,Iv,L)
ucL ,phicL ,McL ,QcL ,ducL ,dphicL,dMcL=solfundviga(L,L/2,E,Iv,L)
b=1e4*[uc0;duc0 ;ucL;ducL]

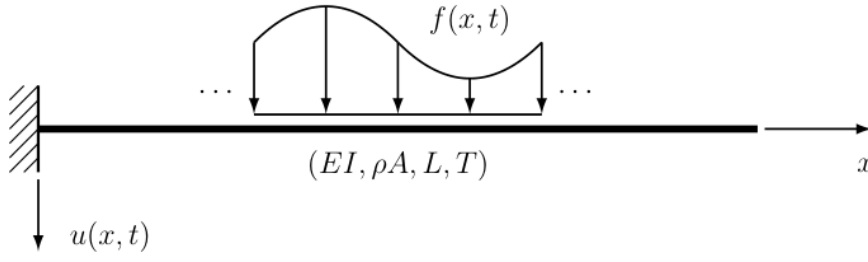
A=[-G[:,1] H[:,2] -G[:,3] H[:,4]]
x=A\b

```

19.1.1 Exercício

- 1- Generalize esse código para qualquer condição de contorno. Ele é capaz de resolver problemas hiperestáticos? Como?
- 2- Generalize esse código para qualquer carga distribuída e compare com a solução analítica.

20 Efeitos transientes



$$EI \frac{\partial^4 u}{\partial x^4} + \rho A \frac{\partial^2 u}{\partial t^2} = f(x, t)$$

Dividindo a equação por ρA e definindo

$$c = \sqrt{\frac{EI}{\rho A}}$$

obtem-se a expressão:

$$c^2 \frac{\partial^4 u}{\partial x^4} + \frac{\partial^2 u}{\partial t^2} = \frac{f(x, t)}{\rho A}$$

a solução fundamental para esse problema é dada por:

$$u^*(x, \xi, t, \tau) = \frac{1}{c} \left\{ \frac{r}{2} \left[S\left(\frac{r}{\sqrt{2\pi a}}\right) - C\left(\frac{r}{\sqrt{2\pi a}}\right) \right] + \frac{\sqrt{a}}{\sqrt{2\pi}} \left[\sin\left(\frac{r^2}{4a}\right) + \cos\left(\frac{r^2}{4a}\right) \right] \right\}$$

onde as funções S e C são denominadas integrais de Fresnel, $r = |x - \xi|$ e $a = c(t - \tau)$.

$$\begin{aligned} \theta^*(x, \xi, t, \tau) &= +\frac{1}{c} \left\{ \frac{1}{2} \left[S\left(\frac{r}{\sqrt{2\pi a}}\right) - C\left(\frac{r}{\sqrt{2\pi a}}\right) \right] \right\} \left(\frac{\partial r}{\partial x} \right), \\ M^*(x, \xi, t, \tau) &= -\frac{EI}{c} \left\{ \frac{1}{2\sqrt{2\pi a}} \left[\sin\left(\frac{r^2}{4a}\right) - \cos\left(\frac{r^2}{4a}\right) \right] \right\} \left(\frac{\partial r}{\partial x} \right)^2, \\ Q^*(x, \xi, t, \tau) &= -\frac{EI}{c} \left\{ \frac{r}{4\sqrt{2\pi a^3}} \left[\sin\left(\frac{r^2}{4a}\right) + \cos\left(\frac{r^2}{4a}\right) \right] \right\} \left(\frac{\partial r}{\partial x} \right)^3, \end{aligned}$$

A equação integral para esse problema é dada por:

$$\begin{aligned} u(\xi, t) &= -\frac{1}{\rho A} \int_0^t [+u^*Q - \theta^*M + M^*\theta - Q^*u]_{x=0} d\tau \\ &+ \frac{1}{\rho A} \int_0^t [+u^*Q - \theta^*M + M^*\theta - Q^*u]_{x=L} d\tau \\ &+ \frac{1}{\rho A} \int_0^t \int_0^L u^* f dx d\tau. \end{aligned}$$

A solução do problema, que envolve quatro incógnitas de contorno, requer ao menos duas equações integrais distintas. Assim, escreve-se, para a rotação:

$$\begin{aligned}\theta(\xi, t) = & -\frac{1}{\rho A} \left\{ \int_0^t \left[\frac{\partial u^*}{\partial \xi} Q - \frac{\partial \theta^*}{\partial \xi} M + \frac{\partial M^*}{\partial \xi} \theta - \frac{\partial Q^*}{\partial \xi} u \right]_{x=0} d\tau \right\} \\ & + \frac{1}{\rho A} \left\{ \int_0^t \left[\frac{\partial u^*}{\partial \xi} Q - \frac{\partial \theta^*}{\partial \xi} M + \frac{\partial M^*}{\partial \xi} \theta - \frac{\partial Q^*}{\partial \xi} u \right]_{x=L} d\tau \right\} \\ & + \frac{1}{\rho A} \left\{ \int_0^t \int_0^L \frac{\partial u^*}{\partial \xi} f dx d\tau \right\},\end{aligned}$$

20.0.1 Exercício

3 - Faça um gráfico 3D de u^* e $\partial \frac{u^*}{\partial \xi}$. Considere ξ e τ iguais a zero, tempo de 0 a 10 s e x de 0 a L . Em Plots use surface ([docs Plots](#)).

21 Desafio

1- Implemente um código que usando a formulação transiente e resolva um problema de vibração livre.

2-Esse problema também pode ser resolvido usando a solução fundamental estática e uma integral de domínio como feito para o problema de Poisson. Faça isso usando Houbolt e compare.

- Compare com uma solução analítica disponível no [Rao](#)

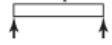
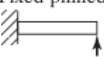
End Conditions of Beam	Frequency Equation	Mode Shape (Normal Function)	Value of $\beta_n l$
 Pinned-pinned	$\sin \beta_n l = 0$	$W_n(x) = C_n [\sin \beta_n x]$	$\beta_1 l = \pi$ $\beta_2 l = 2\pi$ $\beta_3 l = 3\pi$ $\beta_4 l = 4\pi$
 Free-free	$\cos \beta_n l \cdot \cosh \beta_n l = 1$	$W_n(x) = C_n [\sin \beta_n x + \sinh \beta_n x$ $+ \alpha_n (\cos \beta_n x + \cosh \beta_n x)]$ where $\alpha_n = \left(\frac{\sin \beta_n l - \sinh \beta_n l}{\cosh \beta_n l - \cos \beta_n l} \right)$	$\beta_1 l = 4.730041$ $\beta_2 l = 7.853205$ $\beta_3 l = 10.995608$ $\beta_4 l = 14.137165$ ($\beta l = 0$ for rigid-body mode)
 Fixed-fixed	$\cos \beta_n l \cdot \cosh \beta_n l = 1$	$W_n(x) = C_n [\sinh \beta_n x - \sin \beta_n x$ $+ \alpha_n (\cosh \beta_n x - \cos \beta_n x)]$ where $\alpha_n = \left(\frac{\sinh \beta_n l - \sin \beta_n l}{\cos \beta_n l - \cosh \beta_n l} \right)$	$\beta_1 l = 4.730041$ $\beta_2 l = 7.853205$ $\beta_3 l = 10.995608$ $\beta_4 l = 14.137165$
 Fixed-free	$\cos \beta_n l \cdot \cosh \beta_n l = -1$	$W_n(x) = C_n [\sin \beta_n x - \sinh \beta_n x$ $- \alpha_n (\cos \beta_n x - \cosh \beta_n x)]$ where $\alpha_n = \left(\frac{\sin \beta_n l + \sinh \beta_n l}{\cos \beta_n l + \cosh \beta_n l} \right)$	$\beta_1 l = 1.875104$ $\beta_2 l = 4.694091$ $\beta_3 l = 7.854757$ $\beta_4 l = 10.995541$
 Fixed-pinned	$\tan \beta_n l - \tanh \beta_n l = 0$	$W_n(x) = C_n [\sin \beta_n x - \sinh \beta_n x$ $+ \alpha_n (\cosh \beta_n x - \cos \beta_n x)]$ where $\alpha_n = \left(\frac{\sin \beta_n l - \sinh \beta_n l}{\cos \beta_n l - \cosh \beta_n l} \right)$	$\beta_1 l = 3.926602$ $\beta_2 l = 7.068583$ $\beta_3 l = 10.210176$ $\beta_4 l = 13.351768$
 Pinned-free	$\tan \beta_n l - \tanh \beta_n l = 0$	$W_n(x) = C_n [\sin \beta_n x + \alpha_n \sinh \beta_n x]$ where $\alpha_n = \left(\frac{\sin \beta_n l}{\sinh \beta_n l} \right)$	$\beta_1 l = 3.926602$ $\beta_2 l = 7.068583$ $\beta_3 l = 10.210176$ $\beta_4 l = 13.351768$ ($\beta l = 0$ for rigid-body mode)

FIGURE 8.15 Common boundary conditions for the transverse vibration of a beam.

3-Implemente a formulação da viga de Timoshenko e de BICKFORD-REDDY e compare as três para diferentes tamanhos de viga.